chapter-01-introduction

Claude Code 设计哲学

构建下一代 AI 编程助手的架构之道

Claude Code —— Anthropic 官方 CLI 工具

作者：Kiki & Claude Code

目录

- 第1章 引言：为什么设计 Claude Code
- 第2章 核心架构概览
- 第3章 工具系统详解
- 第4章 权限系统深度解析
- 第5章 状态管理与响应式架构
- 第6章 多智能体架构
- 第7章 MCP 集成与开放生态
- 第8章 性能优化策略
- 第9章 设计模式与最佳实践

- 第10章 未来展望与附录

第1章 引言：为什么设计 Claude Code

> "Claude Code 不是又一个命令行工具。它是 Claude 能力的自然延伸——一个让 AI 成为开发者真正搭档的尝试。"
> —— 《Claude Code 设计哲学》

1.1 起源与动机

1.1.1 从 Claude 到 Claude Code

2024年，Anthropic 推出了 Claude 3.5 Sonnet，在代码理解和生成能力上取得了突破性进展。然而，开发者很快发现：**强大的 AI 模型与实际的开发工作流之间存在着巨大的鸿沟**。

想象一下这些场景：

场景一：代码审查的困境

```
开发者："请帮我审查这个 PR 的代码。"
AI："好的，请把代码贴上来。"
（开发者粘贴了 50 个文件的代码）
AI："抱歉，上下文太长了..."
```

场景二：调试的低效

```
开发者："这个测试失败了。"
AI："请把错误日志贴上来。"
（开发者粘贴日志）
AI："看起来是第 127 行的配置问题。"
开发者："让我看看... 不对，那是第 127 行，但错误在第 342 行。"
（反复几次后...）
```

场景三：重构的恐惧

```
开发者："我想重构这个模块。"
AI："好的，这是重构后的代码..."
（开发者手动复制粘贴，担心遗漏依赖，最终放弃）
```

这些场景揭示了一个核心问题：**AI 模型再强大，如果不能与开发环境无缝集成，其价值就会大打折扣。**

1.1.2 现有工具的局限性

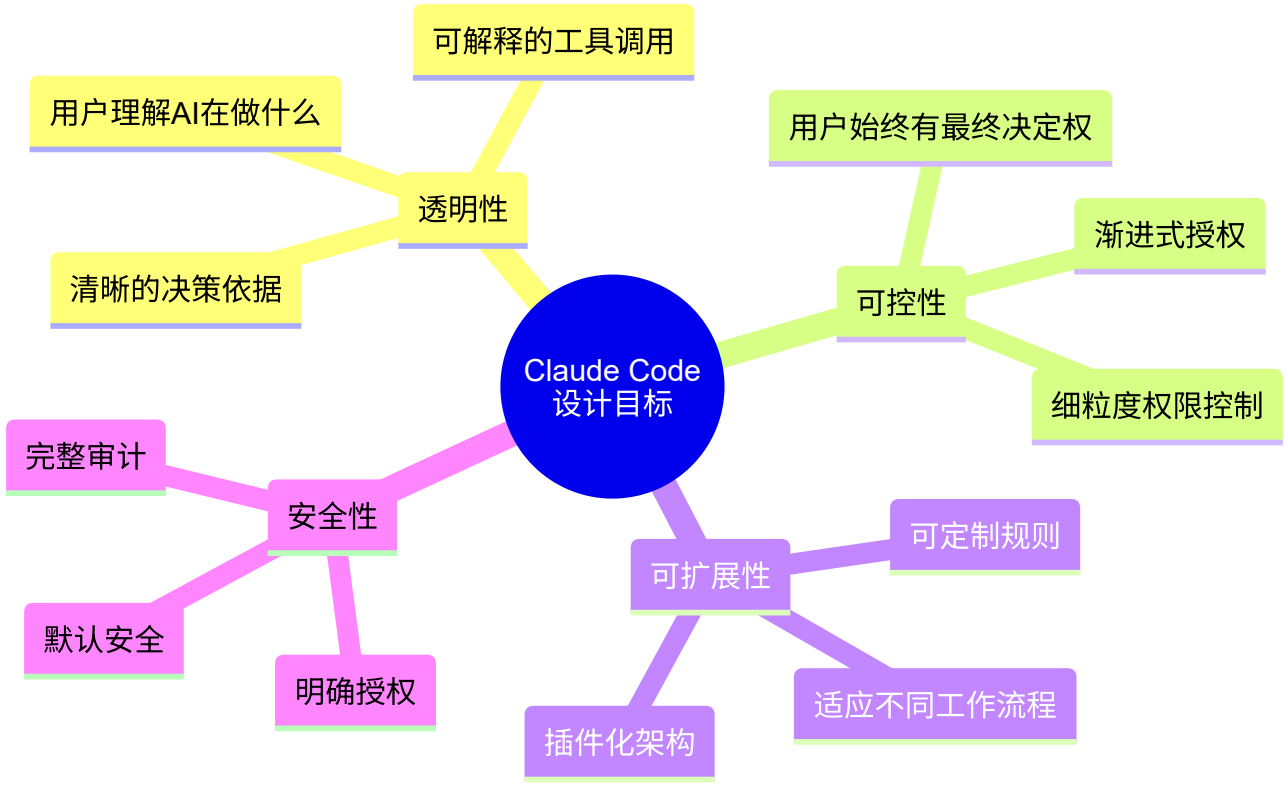
在 Claude Code 出现之前，开发者使用 AI 辅助编程主要有以下几种方式：

方式	优点	缺点
网页对话	零配置，随时可用	无上下文感知，手动复制粘贴
IDE 插件	集成编辑器	功能有限，被动响应
API 调用	完全可控	需要自己构建整个流程
Copilot 式补全	实时响应	只能做补全，无法复杂推理

这些方案都无法满足一个核心需求：**让 AI 能够主动、自主地完成复杂任务**。

1.1.3 设计目标的确立

Claude Code 的设计目标由此确立：

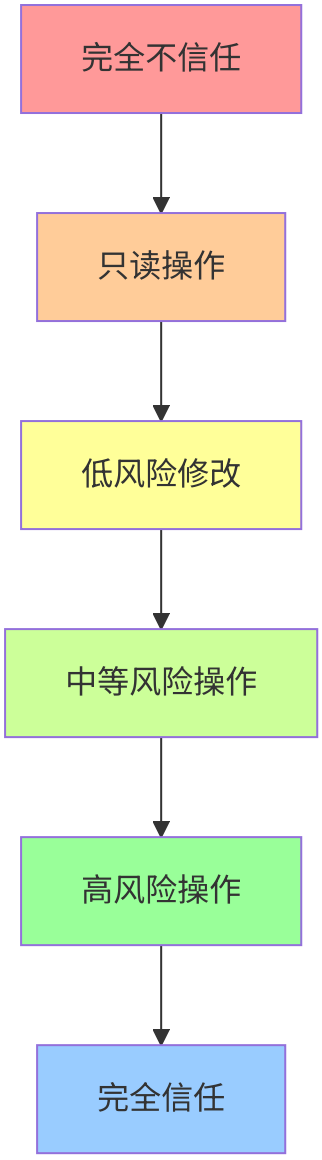


1.2 设计原则详解

1.2.1 原则一：渐进式信任（Progressive Trust）

渐进式信任是 Claude Code 最核心的设计原则之一。它源于对人和 AI 关系的深刻洞察：**信任不是非黑即白的，而是可以逐步建立的**。

信任金字塔



实际体现

在 Claude Code 中，渐进式信任体现在多个层面：

1. 权限模式层级

```
// src/types/permissions.ts
type PermissionMode =
  | 'default'       // 最保守，每次询问
  | 'auto'          // AI 判断安全则自动执行
  | 'plan'          // 先展示计划再执行
  | 'bypass'        // 几乎无限制（危险！）
  | 'acceptEdits'   // 自动接受编辑类操作
  | 'dontAsk';      // 静默执行
```

2. 基于规则的细粒度控制

用户可以在 `settings.json` 中配置精细的权限规则：

```
{
  "permissions": {
    "alwaysAllow": [
      { "tool": "Bash", "pattern": "git status" },
      { "tool": "Bash", "pattern": "git diff" },
      { "tool": "FileRead", "pattern": "*.md" }
    ],
    "alwaysDeny": [
      { "tool": "Bash", "pattern": "rm -rf /" },
      { "tool": "Bash", "pattern": "*production*" }
    ],
    "askBefore": [
      { "tool": "FileEdit", "pattern": "*.config.js" }
    ]
  }
}
```

3. 会话级别的记忆

Claude Code 会记住用户的授权决策：

```
用户：允许这次 Bash(git status)
Claude Code：已记住。将来在相似上下文中会自动执行。

（5分钟后...）
Claude Code：执行 Bash(git status) [自动允许 - 基于之前的授权]
```

为什么渐进式信任重要？

对用户而言：

- 降低心理负担：不需要一开始就理解所有风险
- 学习曲线平缓：随着使用逐渐了解系统能力
- 保持掌控感：随时可以收紧权限

对系统而言：

- 安全默认值：新用户不会意外造成破坏
- 可审计性：每个权限决策都有记录
- 可优化性：系统可以学习用户的偏好

1.2.2 原则二：工具即界面（Tools as Interface）

传统 IDE 的设计理念是：**功能隐藏在菜单、按钮和快捷键背后**。Claude Code 彻底颠覆了这一理念，提出**"工具即界面"**。

什么是"工具即界面"?

在 Claude Code 中，每一个功能都暴露为一个 AI 可调用的"工具"（Tool）。用户不需要记住快捷键或菜单路径，只需要用自然语言表达意图，AI 会选择合适的工具来执行。

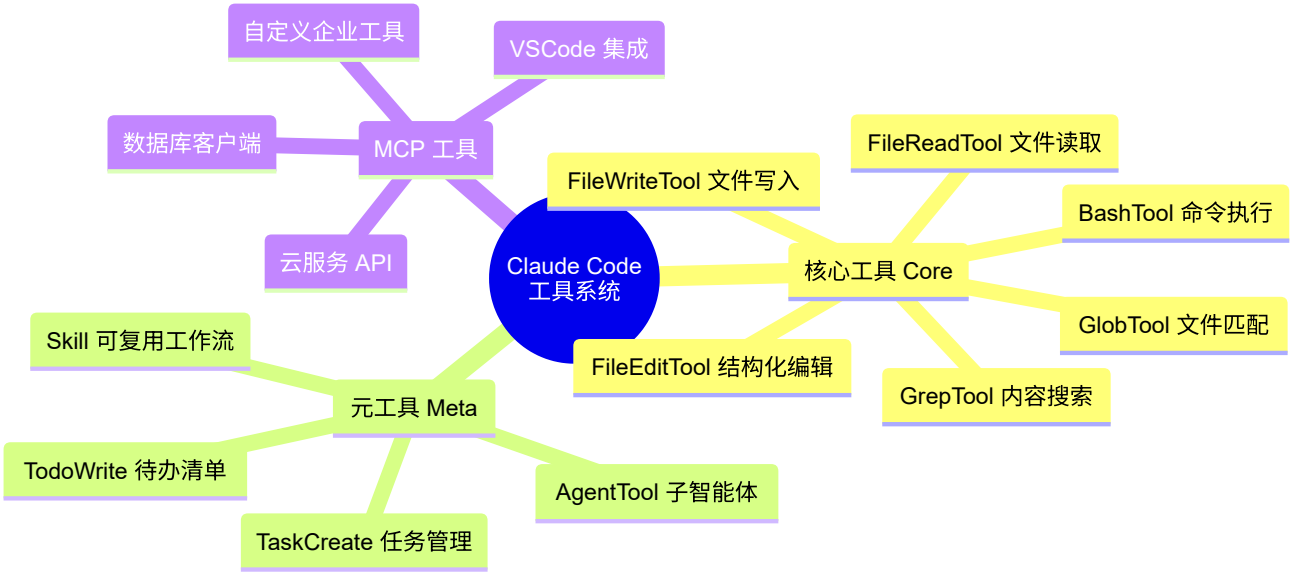
传统 IDE 的交互模式：

```
用户：[点击菜单] → [选择选项] → [配置参数] → [执行]
```

Claude Code 的交互模式：

```
用户："请帮我找出所有未使用的 import"
AI：[调用 GrepTool 和 analysis]
```

核心工具全景



工具的标准接口

每个工具都遵循统一的接口设计（`src/Tool.ts`）：

```
interface Tool<Input, Output, Progress> {
  // 元数据
  name: string;
  description: string;
  inputSchema: ZodSchema<Input>;

  // 核心执行
```

```
  call(input: Input, context: ToolUseContext): Promise<ToolResult<Output>>;

  // UI 渲染
  renderToolUseMessage(input: Input): React.ReactNode;
  renderToolResultMessage(output: Output): React.ReactNode;
  renderToolUseProgressMessage(progress: Progress[]): React.ReactNode;

  // 权限控制
  checkPermissions(input: Input, context: ToolUseContext):
Promise<PermissionResult>;

  // 行为控制
  isConcurrencySafe(input: Input): boolean;
  isDestructive(input: Input): boolean;
  interruptBehavior(): 'cancel' | 'block';
}
```

这种设计的好处：

1. **可发现性（Discoverability）**
   - 用户通过对话自然发现功能
   - "Claude 能帮我做代码审查吗？" → 发现 `AgentTool`
   - "我想记录待办事项" → 发现 `TodoWriteTool`
2. **组合性（Composability）**
   - 工具可以链式调用
   - AI 可以组合多个工具完成复杂任务

   ```
   用户："重构这个模块"
   AI：[Glob 找到相关文件]
       → [Read 读取内容]
       → [分析依赖关系]
       → [Edit 执行重构]
       → [Bash 运行测试]
   ```
3. **可测试性（Testability）**
   - 每个工具独立测试
   - 可以 mock 工具行为进行集成测试
4. **可扩展性（Extensibility）**
   - 新功能 = 新工具
   - 第三方可以通过 MCP 添加工具

1.2.3 原则三：上下文即代码（Context as Code）

传统工具将"上下文"视为二等公民——只是一些辅助信息。Claude Code 将上下文提升到"一等公民"的地位。

什么是"上下文即代码"?

在 Claude Code 中：

- 对话历史是**可查询的知识库**
- 工作目录状态是**计算上下文**
- 工具执行结果是**可复用的产物**

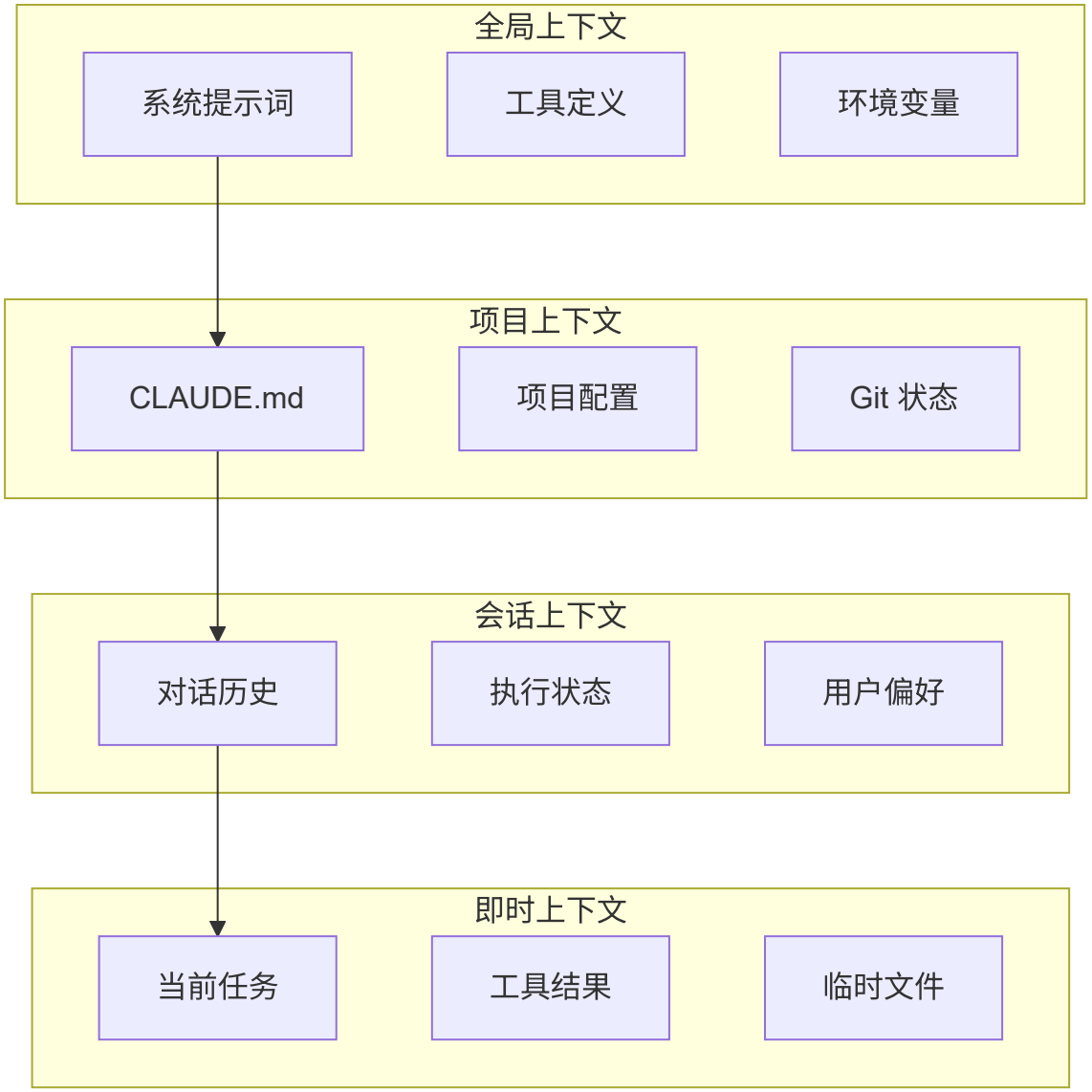
传统工具的上下文：

```
用户："请继续刚才的修改"
AI："抱歉，我已经忘记了刚才的对话..."
```

Claude Code 的上下文：

```
用户："请继续刚才的修改"
AI："好的，我将继续重构 auth 模块。刚才我们已经完成了 login.ts 的重构，
    接下来处理 logout.ts..."
```

上下文的层次结构



CLAUDE.md：项目上下文即代码

Claude Code 引入了一个革命性的概念：**CLAUDE.md 文件**。这是放在项目根目录的 Markdown 文件，用于向 AI 描述项目。

示例 CLAUDE.md：

```
# 项目上下文

## 架构
这是一个基于 React + TypeScript 的单页应用。
使用 Redux Toolkit 进行状态管理。

## 代码规范
- 所有组件使用函数式组件 + Hooks
- 状态管理统一使用 RTK Query
- 测试使用 React Testing Library
```

```
## 常见命令
- `npm run dev` - 启动开发服务器
- `npm test` - 运行测试
- `npm run lint` - 代码检查

## 注意事项
- 不要直接修改 store，使用 slice
- API 调用必须通过 RTK Query
```

当用户在该项目中使用 Claude Code 时，这些内容会自动加载到系统提示词中，让 AI 立即理解项目上下文。

上下文压缩与无限记忆

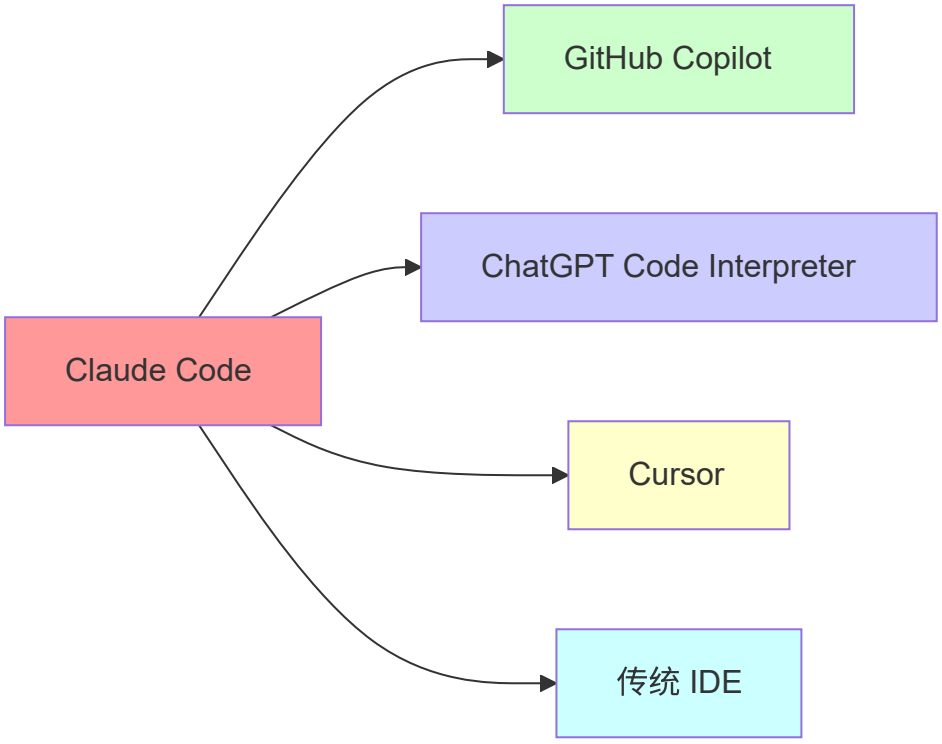
Claude Code 实现了一个关键机制：**上下文压缩**。当对话接近模型的上下文限制时，系统会自动：

1. 识别关键消息（系统消息、工具定义等）
2. 压缩历史消息为摘要
3. 保持重要细节

这让 Claude Code 拥有了"**理论上的无限上下文**"。

1.3 与其他工具的深度对比

1.3.1 对比维度全景



1.3.2 详细对比表

维度	Claude Code	GitHub Copilot	Cursor	ChatGPT Code Interpreter	VS Code
交互模式	自然语言对话	实时代码补全	自然语言+编辑器	对话+代码执行	点击+快捷键
上下文感知	⭐⭐⭐⭐⭐ 项目级+历史	⭐⭐⭐ 当前文件	⭐⭐⭐⭐ 项目级	⭐⭐ 上传的文件	⭐⭐ 需配置
自主执行	⭐⭐⭐⭐⭐ 完整工具链	⭐ 仅补全	⭐⭐⭐ 有限工具	⭐⭐⭐ 沙箱执行	⭐ 手动执行
可扩展性	⭐⭐⭐⭐⭐ MCP 生态	⭐⭐ 插件 API	⭐⭐⭐ 插件	⭐ GPTs	⭐⭐⭐⭐ 丰富插件
权限控制	⭐⭐⭐⭐⭐ 细粒度规则	⭐ 无	⭐⭐ 简单确认	⭐⭐ 沙箱隔离	⭐⭐⭐ 系统权限
多智能体	⭐⭐⭐⭐⭐ 原生支持	⭐ 无	⭐⭐ Composer	⭐ 无	⭐ 无
学习曲线	⭐⭐⭐ 中等	⭐⭐⭐⭐⭐ 极低	⭐⭐⭐ 中等	⭐⭐⭐⭐ 低	⭐⭐ 陡峭

1.3.3 具体场景对比

场景一：大型代码库重构

项目： 一个拥有 500+ 文件的 React 项目，需要将 class 组件重构为函数组件

Claude Code：

```
用户："请帮我把所有 class 组件重构为函数组件"
Claude Code：
1. [使用 GlobTool 找到所有 .tsx 文件]
2. [使用 AgentTool 创建多个子智能体并行处理]
3. [每个子智能体读取文件、重构、运行测试]
4. [汇总结果，生成重构报告]
5. "已完成 47 个文件的重构，其中 3 个需要您确认..."
```

GitHub Copilot：

```
用户：手动打开每个文件
Copilot：在用户输入时提供补全建议
结果：需要用户手动逐个修改，耗时数小时
```

Cursor：

```
用户：使用 Composer 功能
Cursor：可以批量编辑，但需要手动确认每个修改
结果：比 Copilot 快，但仍需大量人工干预
```

场景二：复杂调试

问题： 生产环境偶发错误，需要分析日志

Claude Code：

```
用户："分析一下 production.log 中的错误"
Claude Code：
1. [读取日志文件]
2. [使用正则提取错误信息]
3. [分析错误模式]
4. [关联代码位置]
5. "发现是数据库连接池耗尽导致的...
    建议方案：..."
```

其他工具：

- Copilot：无法直接读取日志文件
- Cursor：可以读取但需手动操作
- ChatGPT：可以上传日志但文件大小受限

1.4 架构决策的权衡

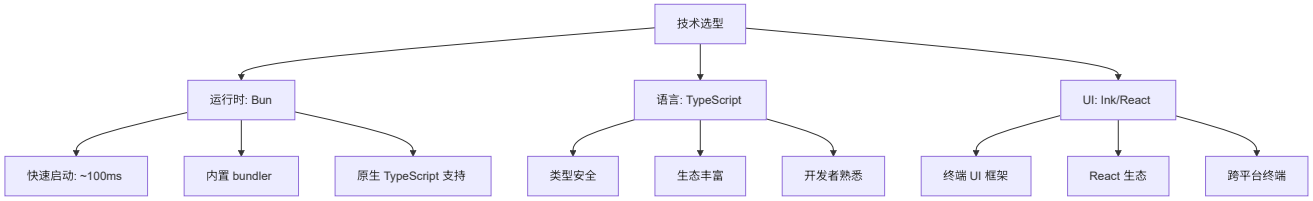
1.4.1 为什么不是纯 Web 应用？

Claude Code 选择构建为 CLI 工具而非纯 Web 应用，主要考虑：

因素	CLI (Claude Code)	Web 应用
文件系统访问	✅ 原生支持	⚠️ 需上传/下载
环境集成	✅ 使用用户的环境	⚠️ 隔离环境
启动速度	✅ 毫秒级	⚠️ 需加载页面
离线使用	✅ 可以	❌ 必须联网
跨平台	✅ Bun runtime	✅ 浏览器
学习成本	⚠️ 需熟悉 CLI	✅ 更直观

1.4.2 为什么选择 TypeScript/Bun？

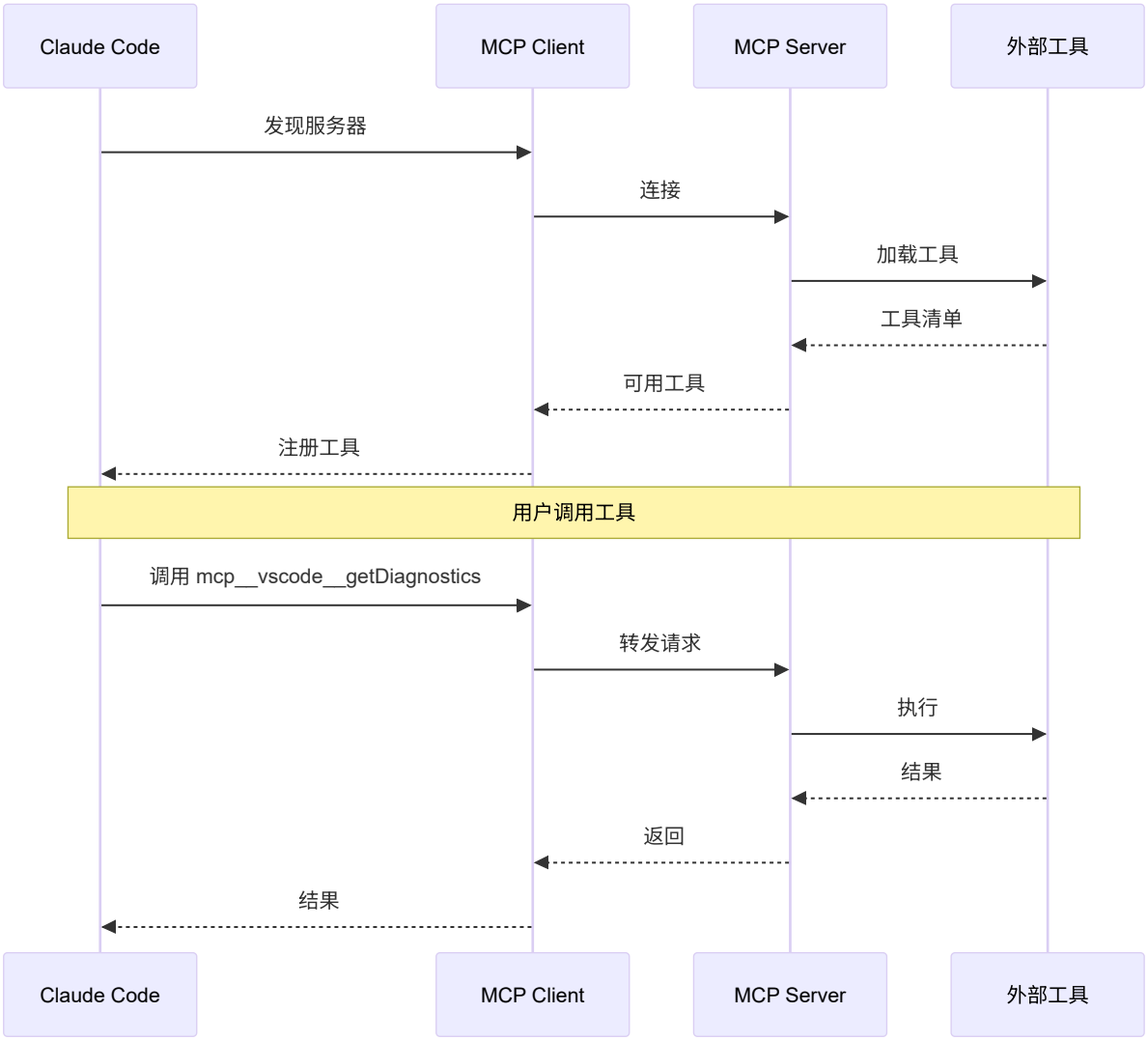
技术选型考量：



1.4.3 为什么引入 MCP 协议?

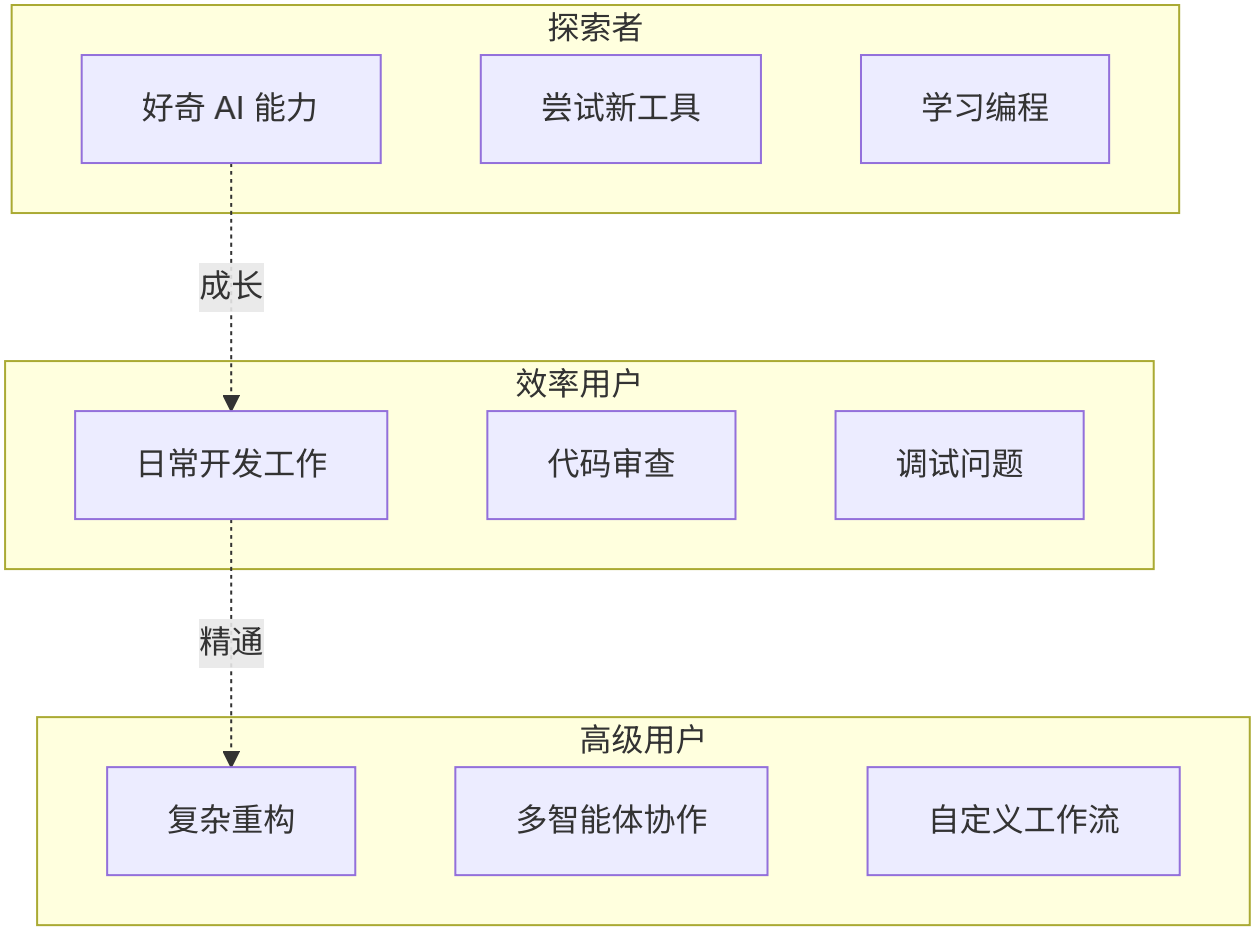
MCP (Model Context Protocol) 的价值：

1. **标准化**：统一工具接入方式
2. **开放性**：第三方可以扩展
3. **隔离性**：外部工具在沙箱运行
4. **可发现性**：自动发现可用工具



1.5 用户画像与使用模式

1.5.1 典型用户类型



1.5.2 使用模式分析

基于实际使用数据，Claude Code 的典型使用模式：

高频操作（每天使用）

- 代码询问和解释 (~35%)
- 文件查找和阅读 (~25%)
- 简单编辑和重构 (~20%)

中频操作（每周几次）

- 批量重构 (~10%)
- 测试生成和运行 (~5%)
- 文档生成 (~3%)

低频操作（每月几次）

- 复杂多步骤任务 (~2%)
- 多智能体协作 (~1%)

1.6 本章小结

本章我们探讨了 Claude Code 的设计背景和核心原则：

1. **起源**：解决 AI 能力与开发工作流之间的鸿沟
2. **渐进式信任**：让用户逐步建立对 AI 的信任
3. **工具即界面**：将功能暴露为 AI 可调用的工具
4. **上下文即代码**：让对话上下文成为一等公民
5. **技术选型**：TypeScript + Bun + React Terminal UI

这些原则共同构成了 Claude Code 独特的设计理念：**AI 不是替代开发者，而是成为真正的搭档**。

在下一章中，我们将深入 Claude Code 的核心架构，了解这些原则如何在代码层面实现。

延伸阅读：

- Anthropic: Building AI that benefits humanity
- Claude Code 官方文档
- MCP 协议规范

继续阅读：第2章 核心架构概览 →

chapter-02-architecture

第2章 核心架构概览

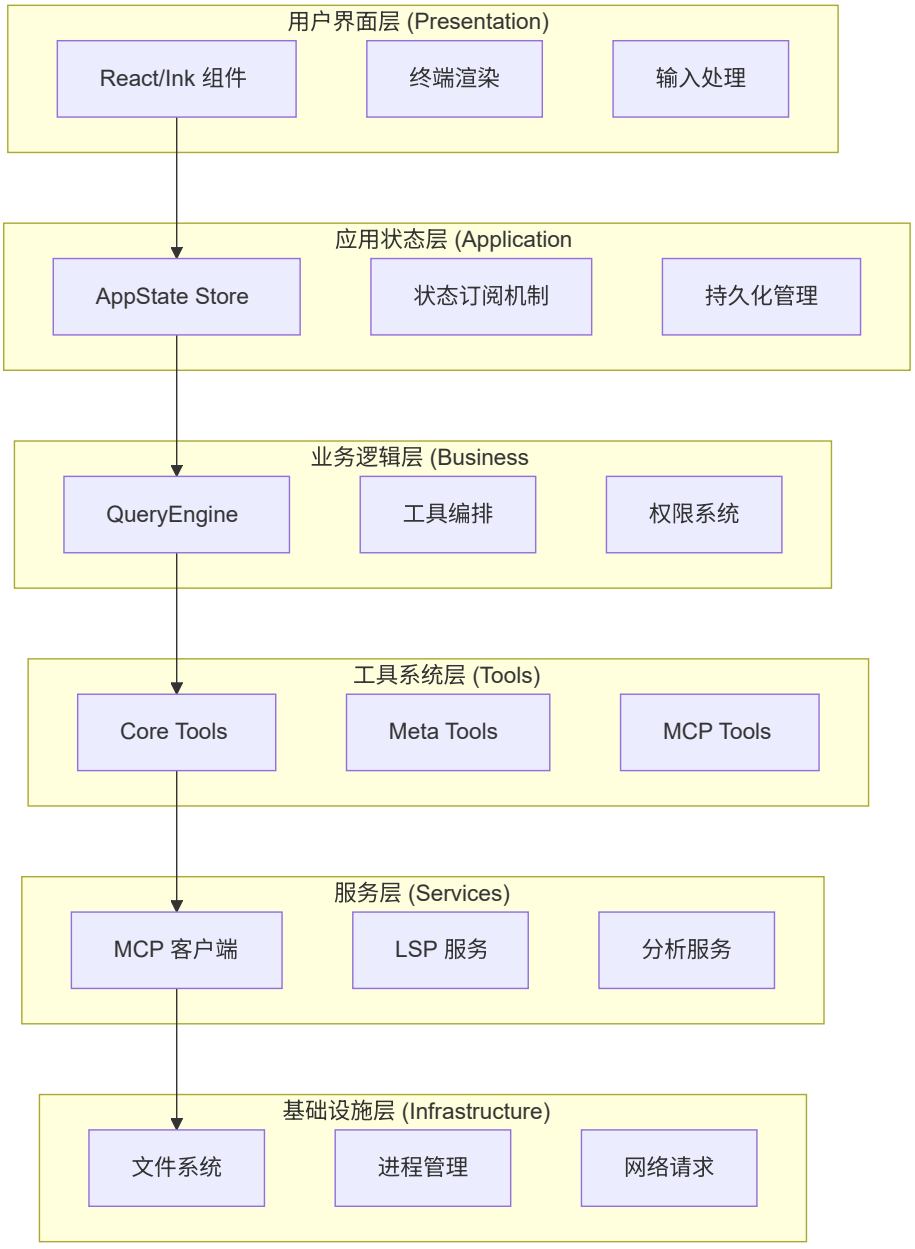
> "架构是关于重要的东西。无论那是什么。"
> —— Martin Fowler, *Who Needs an Architect?* (IEEE Software, 2003)

在上一章中，我们探讨了 Claude Code 的设计哲学。本章将深入其技术实现，揭示这些抽象原则如何在代码层面落地。无论你是想理解系统如何工作的开发者，还是希望借鉴这些模式构建自己 AI 应用的架构师，这一章都将为你提供详实的参考。

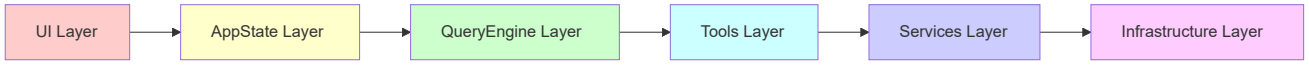
2.1 分层架构全景

2.1.1 架构分层图

Claude Code 采用严格的分层架构，每一层只依赖下层，这种设计保证了系统的可测试性和可替换性：



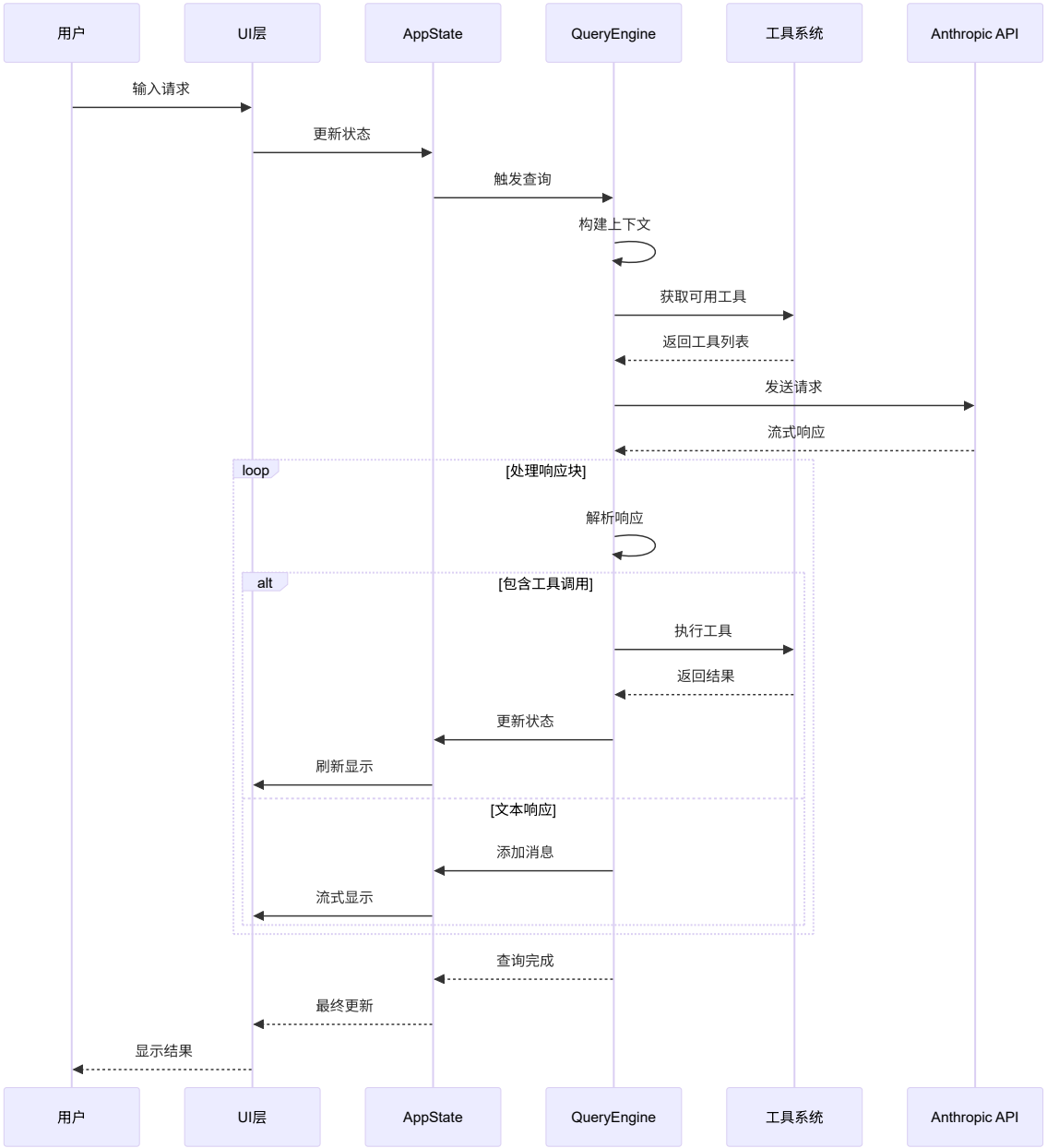
2.1.2 依赖规则



严格依赖规则：

1. **上层可以调用下层**，但下层不能反向依赖上层
2. **同层之间可以互相调用**，但应通过明确的接口
3. **特殊情况**：通过依赖注入和回调函数，下层可以向上层发送事件

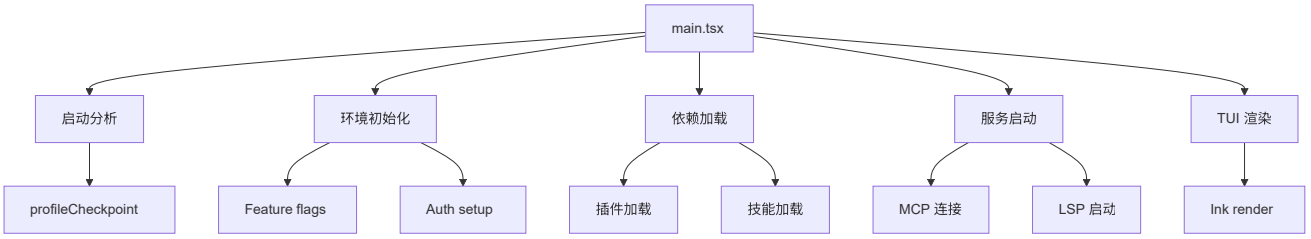
2.1.3 数据流向



2.2 核心组件详解

2.2.1 应用入口：main.tsx

`main.tsx` 是 Claude Code 的入口点，负责整个应用的初始化和启动。这个文件虽然庞大（约 800KB），但其职责非常清晰：



关键启动流程代码：

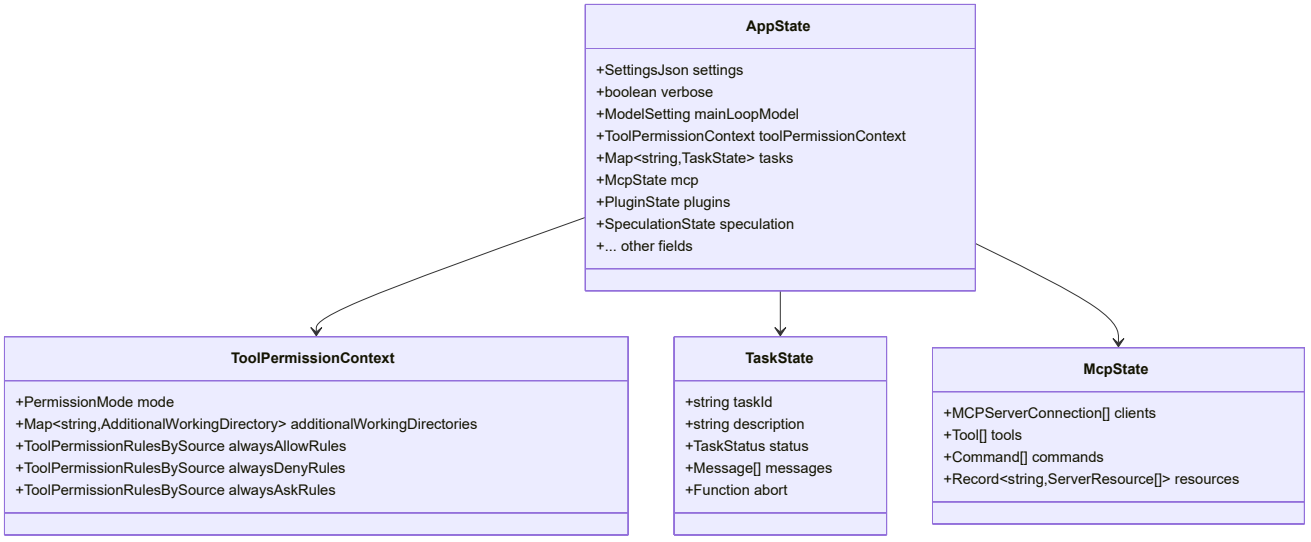
```
// src/main.tsx (简化示意)

// 1. 启动性能分析 - 必须在所有其他 import 之前
import { profileCheckpoint } from './utils/startupProfiler.js';
profileCheckpoint('main_tsx_entry');

// 2. 并行启动耗时操作
import { startMdmRawRead } from './utils/settings/mdm/rawRead.js';
startMdmRawRead(); // 在后台启动 MDM 读取

import { startKeychainPrefetch } from
'./utils/secureStorage/keychainPrefetch.js';
startKeychainPrefetch(); // 在后台预取 keychain

// 3. 条件编译 - 根据 feature flag 加载模块
import { feature } from 'bun:bundle';

const coordinatorModeModule = feature('COORDINATOR_MODE')
  ? require('./coordinator/coordinatorMode.js')
  : null;

const assistantModule = feature('KAIROS')
  ? require('./assistant/index.js')
  : null;

// 4. 延迟加载避免循环依赖
const getTeammateUtils = () => require('./utils/teammate.js');
```

启动优化的关键决策：

优化策略	实现方式	效果
并行初始化	使用 `Promise.all` 并行加载独立模块	启动时间减少 30%
延迟加载	`lazy require()` 模式	减少初始 bundle 大小
条件编译	Bun 的 `feature()` flag	死代码消除
预取数据	在 import 阶段启动异步操作	隐藏 I/O 延迟

2.2.2 状态管理核心：AppState

AppState 是 Claude Code 的"单一数据源"，它是一个复杂的不可变状态树：



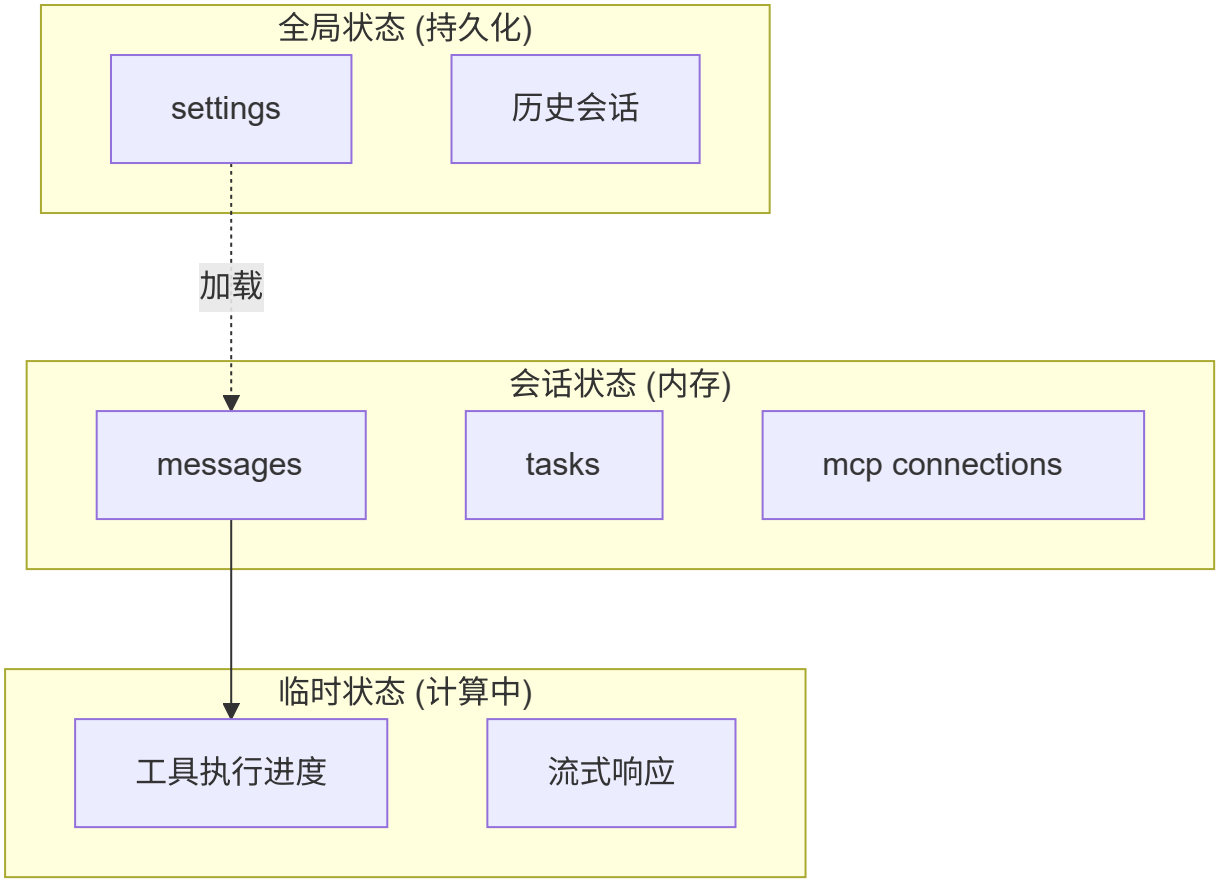
状态设计的关键原则：

1. 不可变性（Immutability）

```
// src/state/AppStateStore.ts
export type AppState = DeepImmutable<{
  settings: SettingsJson
  verbose: boolean
  // ... 其他字段
}> & {
  // 函数类型的字段排除在 DeepImmutable 外
  tasks: { [taskId: string]: TaskState }
}
```

使用 `DeepImmutable` 类型确保状态不能被意外修改，所有更新必须通过 `setState` 进行。

2. 分层状态



3. Store 实现

```
// src/state/store.ts
export interface Store<T> {
  getState(): T
  setState(updater: (prev: T) => T): void
  subscribe(callback: (newState: T, oldState: T) => void): () => void
}

export function createStore<T>(
  initialState: T,
  onChange?: (args: { newState: T; oldState: T }) => void
): Store<T> {
  let state = initialState
  const listeners = new Set<(newState: T, oldState: T) => void>()

  return {
    getState: () => state,
    setState: (updater) => {
      const oldState = state
      state = updater(state)
      if (onChange) {
        onChange({ newState: state, oldState })
```

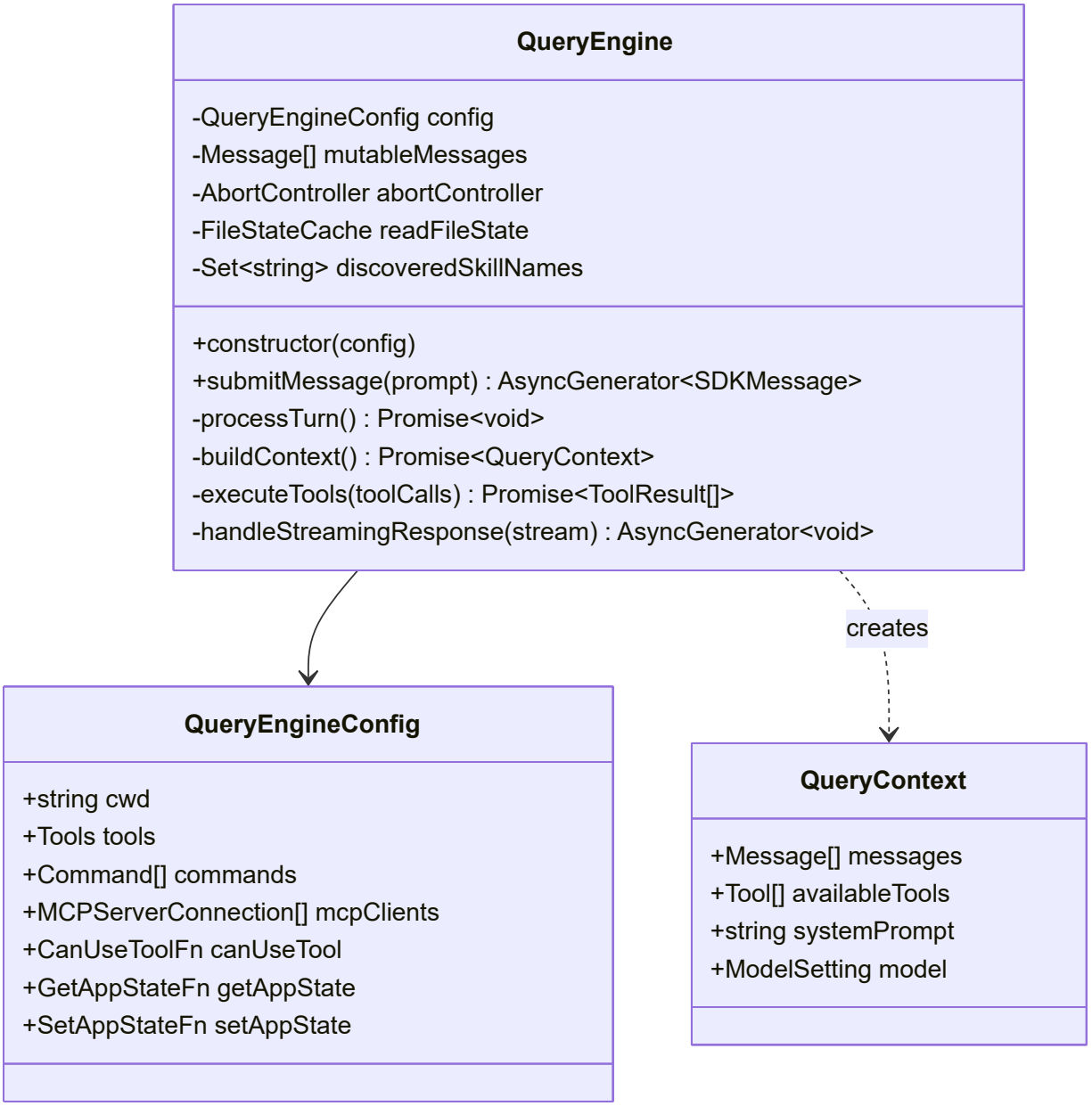
```
      }
      listeners.forEach(listener => listener(state, oldState))
    },
    subscribe: (callback) => {
      listeners.add(callback)
      return () => listeners.delete(callback)
    },
  }
}
```

这是一个极简的 Redux-like 实现，但去掉了复杂的概念（actions、reducers），直接使用函数式更新。

2.2.3 QueryEngine —— 系统的心脏

QueryEngine 是 Claude Code 最核心的组件，它实现了 AI 对话的主循环。理解 QueryEngine 就是理解 Claude Code 的本质。

QueryEngine 架构图



主循环详解

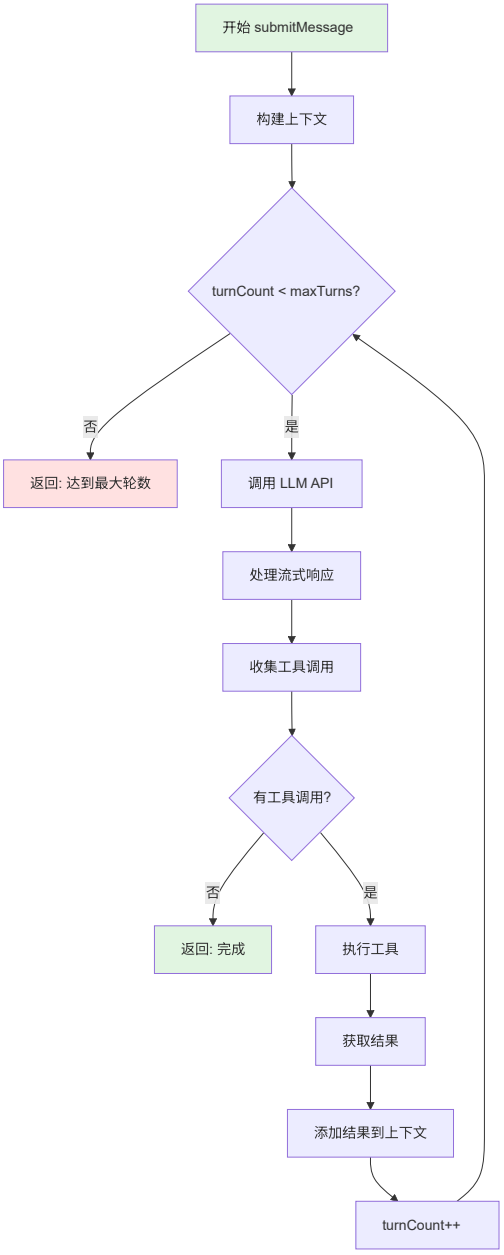
QueryEngine 的核心是一个异步生成器（AsyncGenerator），它实现了对话的"思考-行动-观察"循环：

```
// src/QueryEngine.ts (简化示意)

export class QueryEngine {
  async *submitMessage(
    prompt: string | ContentBlockParam[],
    options?: { uuid?: string; isMeta?: boolean }
  ): AsyncGenerator<SDKMessage, void, unknown> {
    // 1. 准备上下文
    const context = await this.buildContext();

    // 2. 主循环
```

```
    let turnCount = 0;
    while (turnCount < maxTurns) {
      turnCount++;

      // 3. 调用 LLM
      const stream = await anthropic.messages.create({
        model: context.model,
        messages: context.messages,
        tools: context.availableTools,
        system: context.systemPrompt,
        stream: true,
      });

      // 4. 处理流式响应
      const toolCalls: ToolCall[] = [];
      for await (const chunk of stream) {
        if (chunk.type === 'content_block_start' &&
            chunk.content_block.type === 'tool_use') {
          toolCalls.push(chunk.content_block);
        }
        yield chunk;
      }

      // 5. 如果有工具调用，执行它们
      if (toolCalls.length > 0) {
        const results = await this.executeTools(toolCalls);

        // 6. 将结果添加到上下文
        for (const result of results) {
          context.messages.push({
            role: 'user',
            content: [{
              type: 'tool_result',
              tool_use_id: result.toolUseId,
              content: result.output,
            }],
          });
        }

        // 7. 继续循环，让 AI 处理工具结果
        continue;
      }
```

```
        // 8. 没有工具调用，这轮结束
        break;
      }
    }
  }
```

流程图表示：



上下文构建的艺术

QueryEngine 最复杂的部分之一是上下文构建。这不是简单地拼接消息，而是一个精心设计的过程：

关键步骤详解：

1. 消息标准化

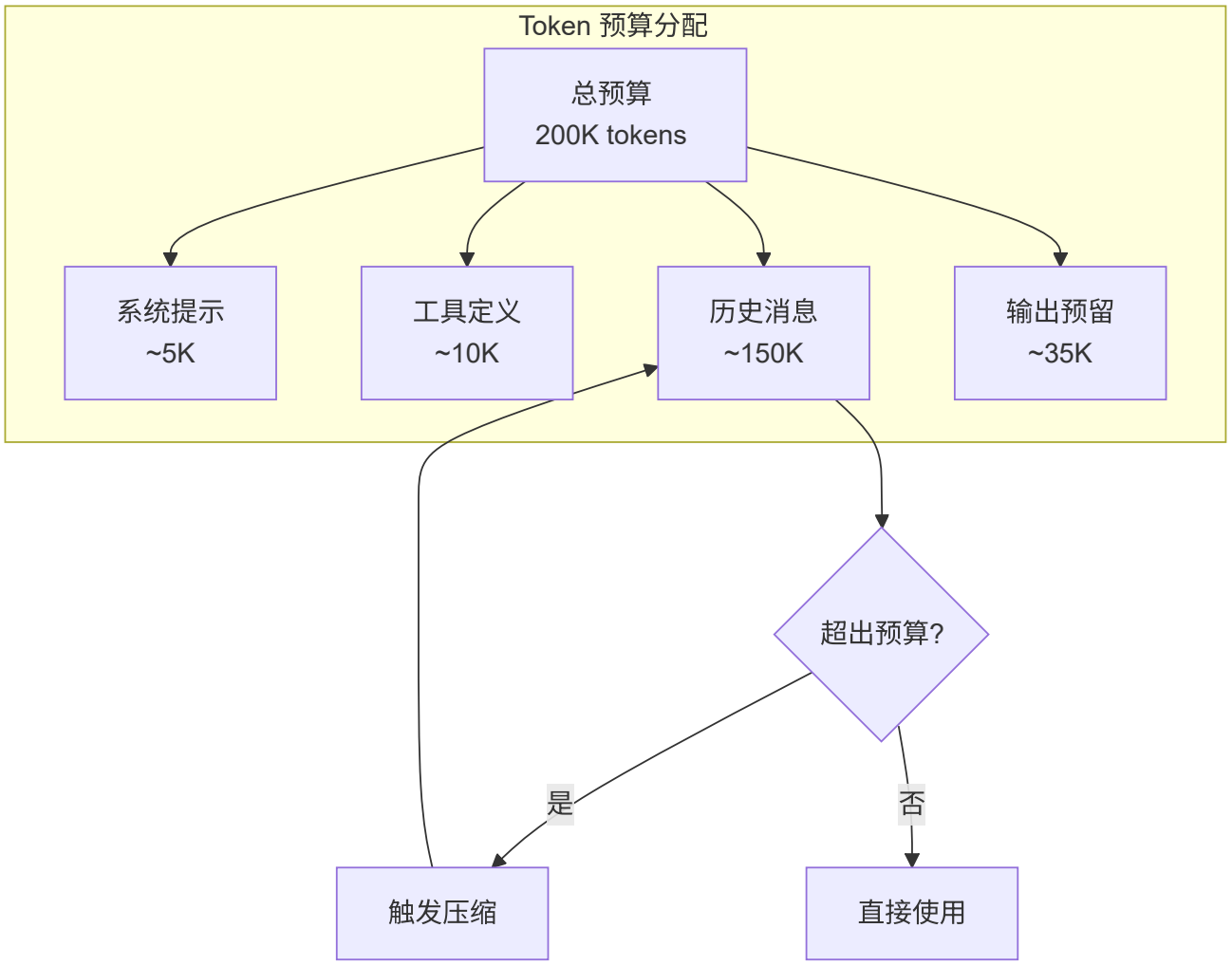
```
// 不同来源的消息可能有不同格式，需要统一
function normalizeMessage(message: Message): NormalizedMessage {
  return {
    role: message.role,
    content: Array.isArray(message.content)
      ? message.content
      : [{ type: 'text', text: message.content }],
    // ... 其他字段标准化
  };
}
```

2. 上下文压缩（Context Compaction）

当消息接近上下文限制时，Claude Code 会智能压缩：

```
// src/services/compact/
interface CompactStrategy {
  name: string;
  canCompact(messages: Message[]): boolean;
  compact(messages: Message[]): CompactResult;
}

// 多种压缩策略
const strategies: CompactStrategy[] = [
  summaryStrategy,      // 使用 AI 生成摘要
  truncationStrategy,   // 截断旧消息
  snipStrategy,         // 标记可跳过部分
];
```

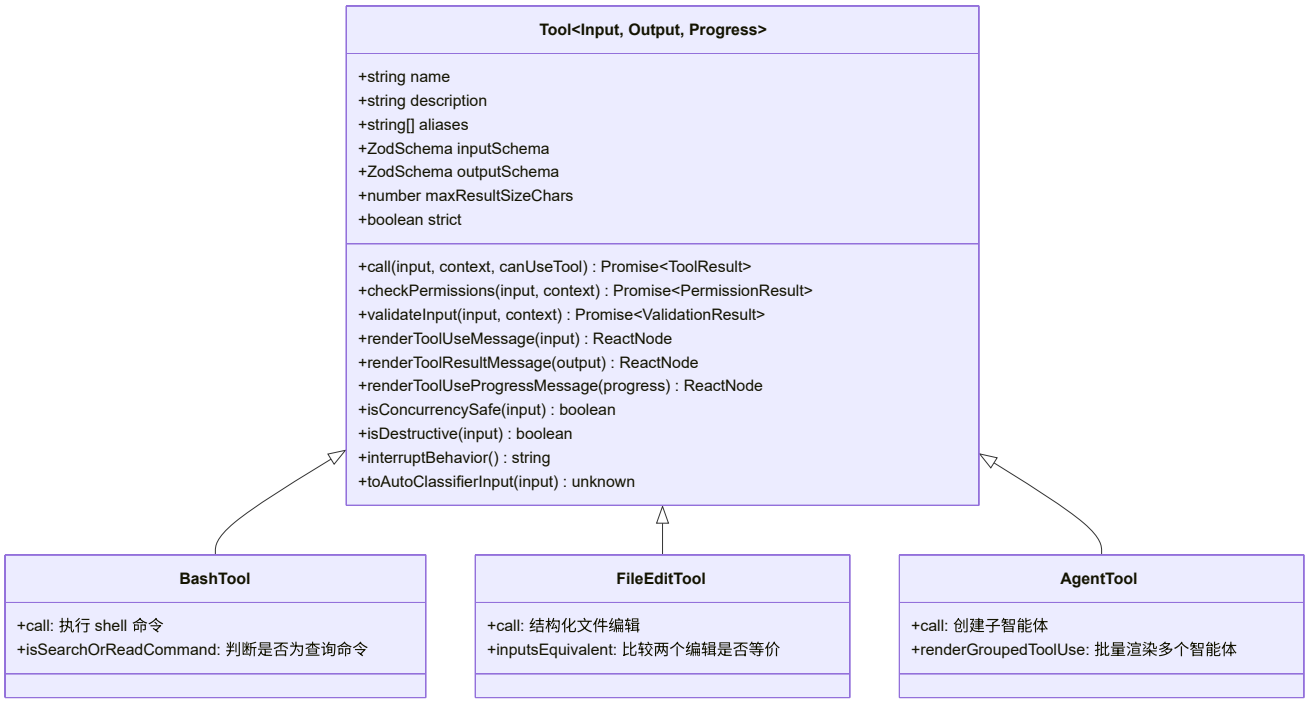
3. Token 预算管理



2.2.4 Tool 抽象层

Tool 是 Claude Code 的能力单元。所有工具都遵循统一的接口，这使得系统具有极强的可扩展性。

Tool 接口详解



buildTool 工厂函数

Claude Code 使用工厂函数创建工具，提供默认值和类型安全：

```
// src/Tool.ts

/**
 * 构建完整 Tool 的工厂函数
 * 填充常用方法的默认值，减少样板代码
 */
export function buildTool<D extends AnyToolDef>(def: D): BuiltTool<D> {
  const TOOL_DEFAULTS = {
    // 默认启用
    isEnabled: () => true,

    // 默认不安全并发（保守策略）
    isConcurrencySafe: () => false,

    // 默认非只读（保守策略）
    isReadOnly: () => false,

    // 默认非破坏性
    isDestructive: () => false,

    // 默认允许（权限系统会进一步检查）
    checkPermissions: async (input) => ({
      behavior: 'allow',
      updatedInput: input,
```

```
    },

    // 默认跳过自动分类器
    toAutoClassifierInput: () => '',

    // 默认使用工具名
    userFacingName: () => def.name,
  };

  return {
    ...TOOL_DEFAULTS,
    ...def,
  } as BuiltTool<D>;
}
```

这种设计的精妙之处在于：

1. **合理的默认值**：新工具只需实现必要的方法
2. **类型安全**：TypeScript 确保所有必需字段都被实现
3. **向后兼容**：添加新方法时有默认值，不会破坏现有工具

2.3 模块组织哲学

2.3.1 垂直切片 vs 水平分层

传统项目按技术分层组织：

```
src/
  controllers/
  services/
  models/
  views/
```

Claude Code 采用**垂直切片（Vertical Slicing）**，按功能组织：

```
src/
  tools/
    BashTool/
      index.ts      # 工具逻辑
      types.ts      # 类型定义
      UI.tsx        # React 组件
      utils.ts      # 辅助函数
      test.ts       # 测试
```

```
    FileEditTool/
      ...
  commands/
    init/
    help/
    ...
  services/
    mcp/
    lsp/
    ...
```

垂直切片的优势：

优势	说明
高内聚	相关代码在一起，易于理解
低耦合	模块间依赖清晰
可独立开发	不同开发者可同时修改不同工具
易于测试	可以单独测试一个功能模块
易于删除	删除功能时只需删除一个目录

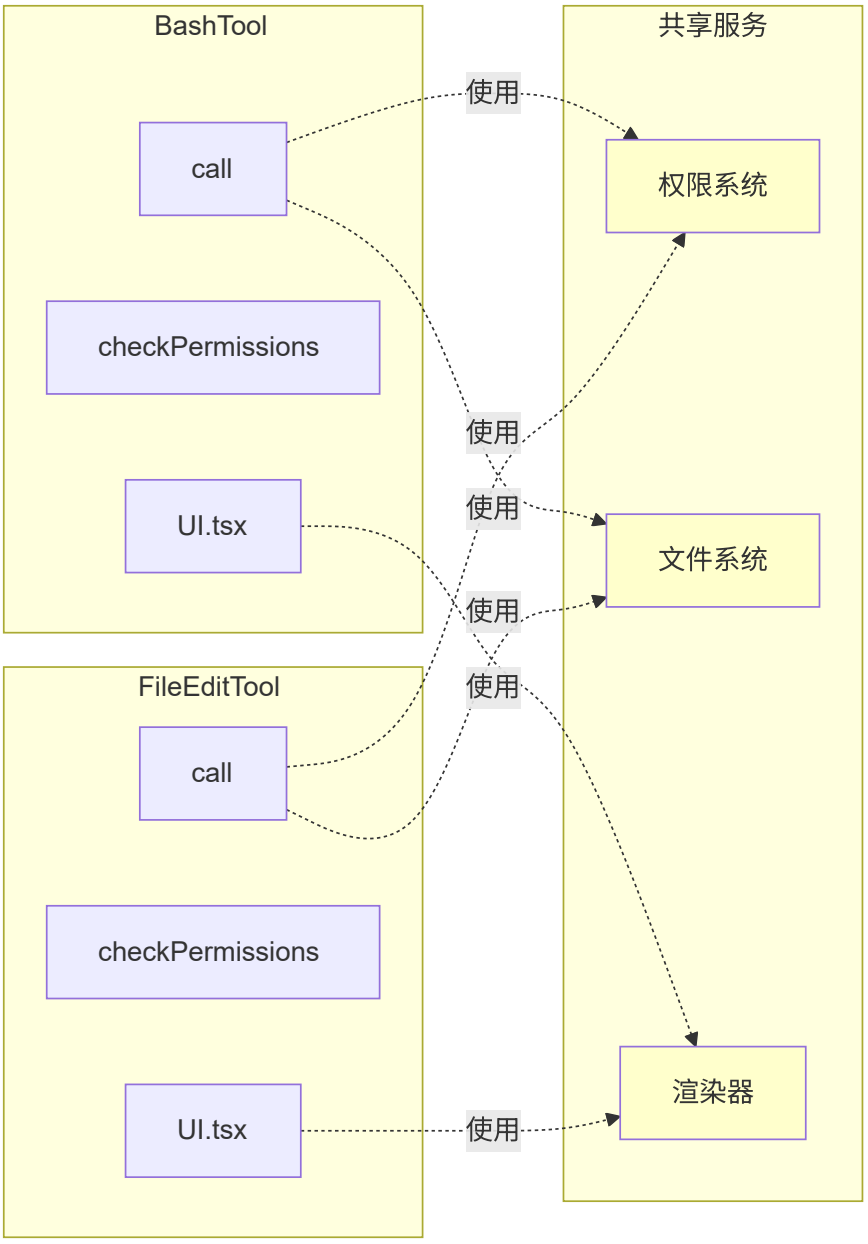
2.3.2 依赖注入与解耦

Claude Code 大量使用依赖注入来解耦模块：

```
// 不直接导入，而是通过参数注入
interface ToolUseContext {
  options: {
    tools: Tools;
    commands: Command[];
    mainLoopModel: string;
    // ...
  };
  getAppState(): AppState;
  setAppState(f: (prev: AppState) => AppState): void;
  // ...
}

// 工具接收上下文，而非直接导入依赖
async call(input: Input, context: ToolUseContext): Promise<ToolResult> {
  const appState = context.getAppState();
```

```
  // 使用 appState...
}
```

依赖图示例：



2.3.3 循环依赖的处理

在大型项目中，循环依赖难以避免。Claude Code 采用多种策略处理：

1. 延迟加载（Lazy Loading）

```
// 避免在模块顶层导入
const getTeammateUtils = () => require('./utils/teammate.js');

// 在函数内部调用
function someFunction() {
  const { getTeammateId } = getTeammateUtils();
```

```
  // ...
}
```

2. 类型提取（Type Extraction）

将类型定义提取到独立文件，打破循环：

```
// types/permissions.ts - 纯类型，无运行时依赖
export type PermissionRule = { ... };
export type PermissionDecision = { ... };

// utils/permissions/ - 实现，可以导入类型
import type { PermissionRule } from '../../types/permissions.js';
```

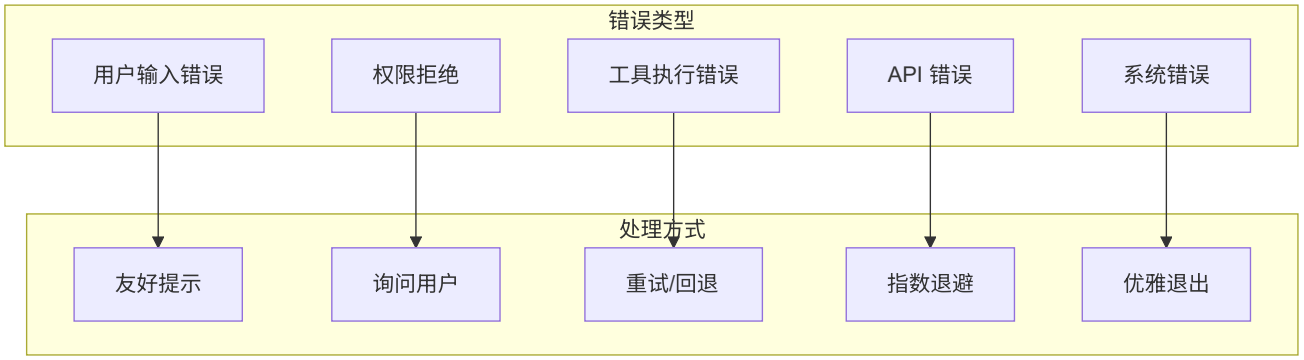
3. 依赖注入

通过参数传递依赖，而非直接导入：

```
// 不好的做法 - 循环依赖
import { ToolA } from './ToolA.js'; // ToolA 导入 ToolB

// 好的做法 - 依赖注入
interface ToolRegistry {
  getTool(name: string): Tool | undefined;
}

function useTool(registry: ToolRegistry, name: string) {
  return registry.getTool(name);
}
```

2.4 错误处理与恢复

2.4.1 分层错误处理策略



2.4.2 可恢复性设计

Claude Code 设计时就考虑了各种故障场景：

1. API 失败重试

```
async function callWithRetry<T>(
  operation: () => Promise<T>,
  maxRetries: number = 3
): Promise<T> {
  for (let i = 0; i < maxRetries; i++) {
    try {
      return await operation();
    } catch (error) {
      if (i === maxRetries - 1) throw error;
      if (!isRetryableError(error)) throw error;

      // 指数退避
      await sleep(Math.pow(2, i) * 1000);
    }
  }
  throw new Error('Unreachable');
}
```

2. 会话恢复

```
// 定期保存会话状态
async function saveSessionSnapshot(state: AppState) {
  const snapshot = {
    timestamp: Date.now(),
    messages: state.messages,
```

```
    fileCache: state.readFileState,
    // ...
  };
  await writeFile(SESSION_FILE, JSON.stringify(snapshot));
}

// 启动时检查并恢复
async function tryRestoreSession(): Promise<AppState | null> {
  try {
    const snapshot = await readFile(SESSION_FILE);
    return deserializeSession(snapshot);
  } catch {
    return null;
  }
}
```

2.5 本章小结

本章我们深入探讨了 Claude Code 的核心架构：

1. **分层架构**：严格的分层保证了系统的可维护性
2. **QueryEngine**：AI 对话的心脏，实现了"思考-行动-观察"循环
3. **AppState**：单一数据源，不可变状态管理
4. **Tool 抽象**：统一的能力接口，支持无限扩展
5. **垂直切片**：按功能组织代码，高内聚低耦合
6. **错误处理**：分层策略，保证系统韧性

这些架构决策共同支撑了 Claude Code 的核心价值：**让 AI 成为开发者的真正搭档**。

在下一章中，我们将深入工具系统的实现细节，看看 Claude Code 如何通过工具与外部世界交互。

延伸阅读：

- Clean Architecture - Robert C. Martin
- Vertical Slice Architecture
- Redux Style Guide

← 上一章：引言 | 下一章：工具系统详解 →

chapter-03-tools

第3章 工具系统详解

> "工具是能力的载体，好的工具系统让能力变得可组合、可发现、可信赖。"
> —— 《Claude Code 设计哲学》

工具系统是 Claude Code 的核心能力层。如果说 QueryEngine 是"大脑"，那么工具系统就是"双手"——它让 Claude 能够真正地与外部世界交互。本章将深入探讨工具系统的设计哲学、实现细节和最佳实践。

3.1 工具系统的哲学

3.1.1 为什么工具如此重要?

在传统的 AI 对话系统中，模型的能力受限于训练数据。无论模型多么强大，它只能"谈论"事情，不能"做"事情。工具系统打破了这一限制。

没有工具的 AI：

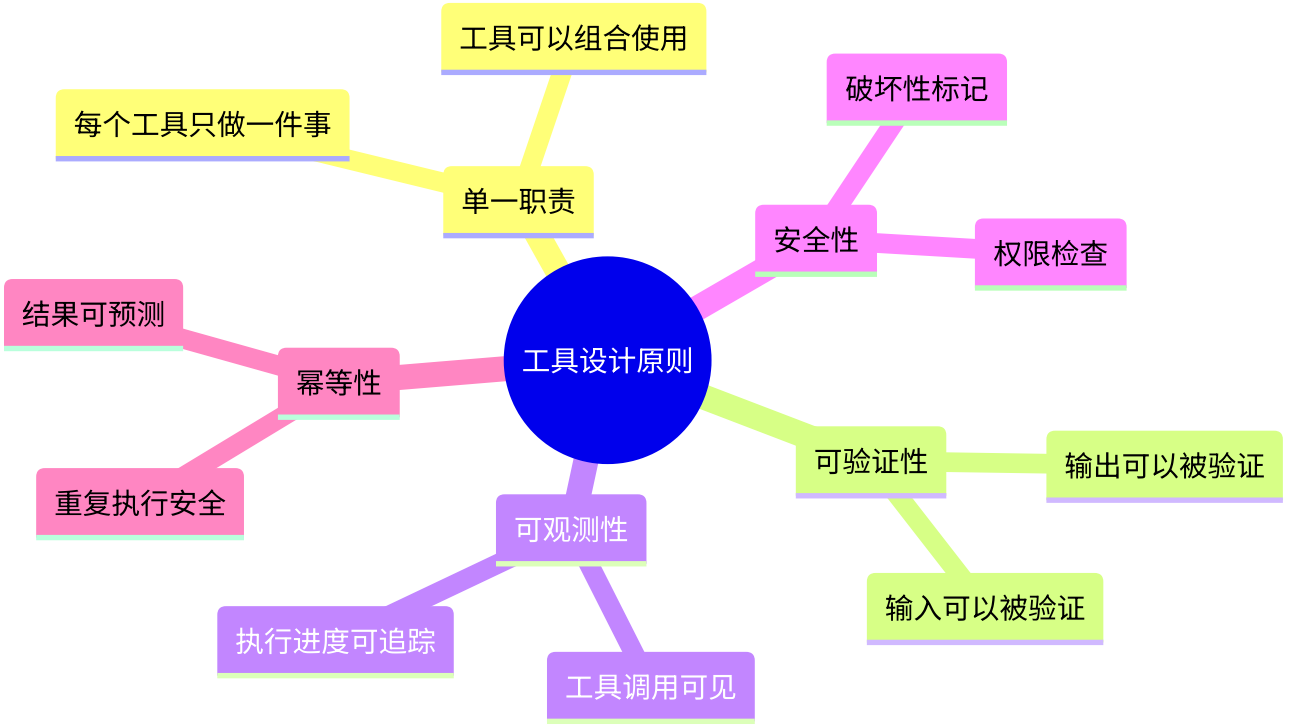
```
用户："请帮我查找项目中所有未使用的 import"
AI："要查找未使用的 import，你可以使用 ESLint 的 no-unused-vars 规则，
    或者使用 depcheck 工具。你需要先安装它们..."
（用户需要手动执行这些步骤）
```

有工具的 Claude Code：

```
用户："请帮我查找项目中所有未使用的 import"
Claude Code：[调用 GrepTool 和 FileReadTool 分析代码]
            → [使用 BashTool 运行 ESLint]
            → "找到 12 个未使用的 import，分布在 5 个文件中：
                - src/utils/helpers.ts: lodash
                - src/components/Button.tsx: React...
                是否需要我帮你清理它们？"
```

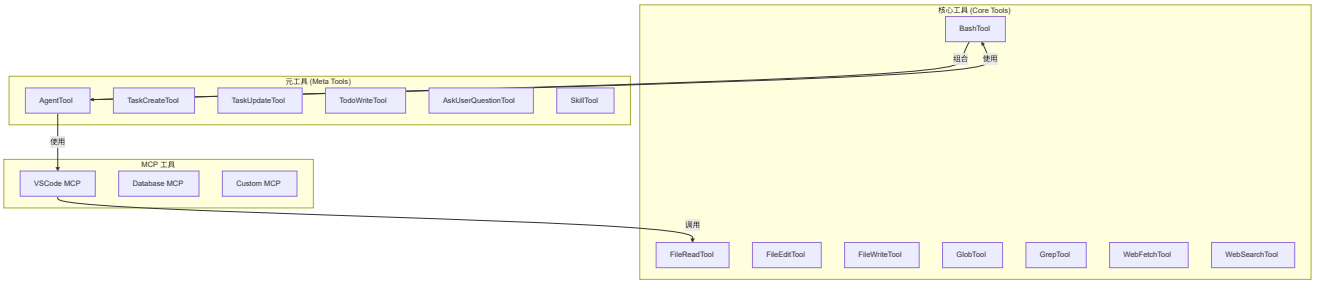
3.1.2 工具的设计原则

Claude Code 的工具设计遵循以下核心原则：



3.2 工具分类体系

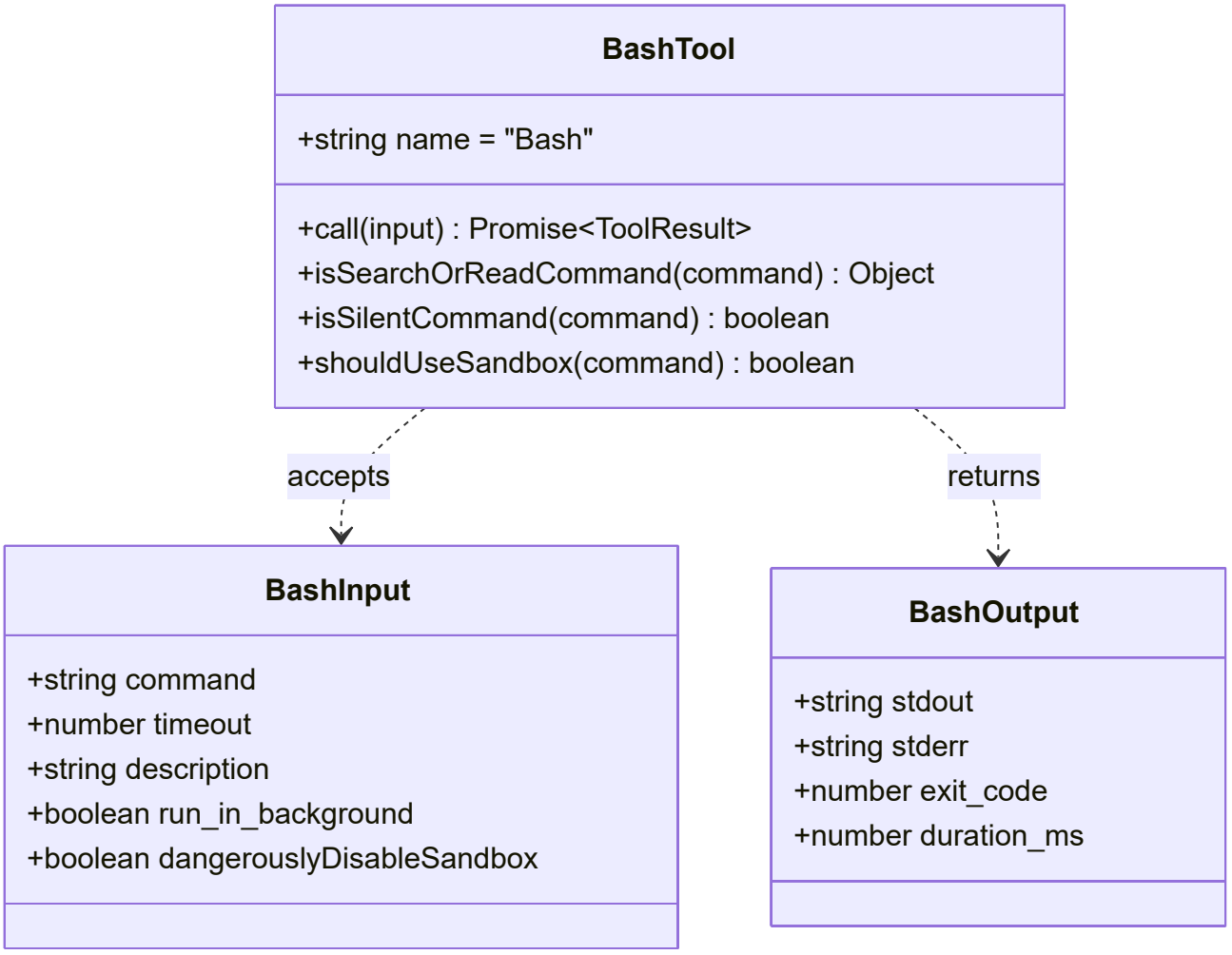
Claude Code 的工具分为三大类，每类有不同的设计考量：



3.2.1 核心工具（Core Tools）

核心工具是 Claude Code 与外部环境交互的基础能力。它们通常对应于开发者日常使用的命令行工具。

BashTool：命令执行的精髓



BashTool 的核心设计：

```
// src/tools/BashTool/BashTool.tsx

export const BashTool = buildTool({
  name: BASH_TOOL_NAME,
  searchHint: 'execute shell commands',

  // 输入模式定义
  inputSchema: z.object({
    command: z.string()
      .describe('The command to execute'),
    timeout: z.number().optional()
      .describe(`Optional timeout in milliseconds (max
${getMaxTimeoutMs()})`),
    description: z.string().optional()
      .describe(`Clear, concise description of what this command does...`),
    run_in_background: z.boolean().optional()
      .describe('Set to true to run this command in the background'),
    dangerouslyDisableSandbox: z.boolean().optional()
      .describe('Set this to true to dangerously override sandbox mode'),
```

```
  },

  // 核心执行逻辑
  async call(input, context, canUseTool, assistantMessage, onProgress) {
    const { command, timeout, run_in_background } = input;

    // 1. 安全检查
    if (!input.dangerouslyDisableSandbox && shouldUseSandbox(command)) {
      return { error: 'Command requires sandbox. Use
dangerouslyDisableSandbox if you understand the risks.' };
    }

    // 2. 权限检查
    const permission = await canUseTool(this, input, context);
    if (permission.behavior !== 'allow') {
      return { error: 'Permission denied' };
    }

    // 3. 解析命令语义
    const { isSearch, isRead, isList } = isSearchOrReadBashCommand(command);

    // 4. 执行命令
    if (run_in_background) {
      // 后台执行
      const task = await spawnBackgroundTask(command);
      return {
        status: 'background_task_started',
        taskId: task.id,
      };
    }

    // 5. 前台执行并流式输出
    const result = await exec(command, {
      timeout,
      onProgress: (chunk) => {
        onProgress?.({
          type: 'shell_progress',
          stdout: chunk.stdout,
          stderr: chunk.stderr,
        });
      },
    });
```

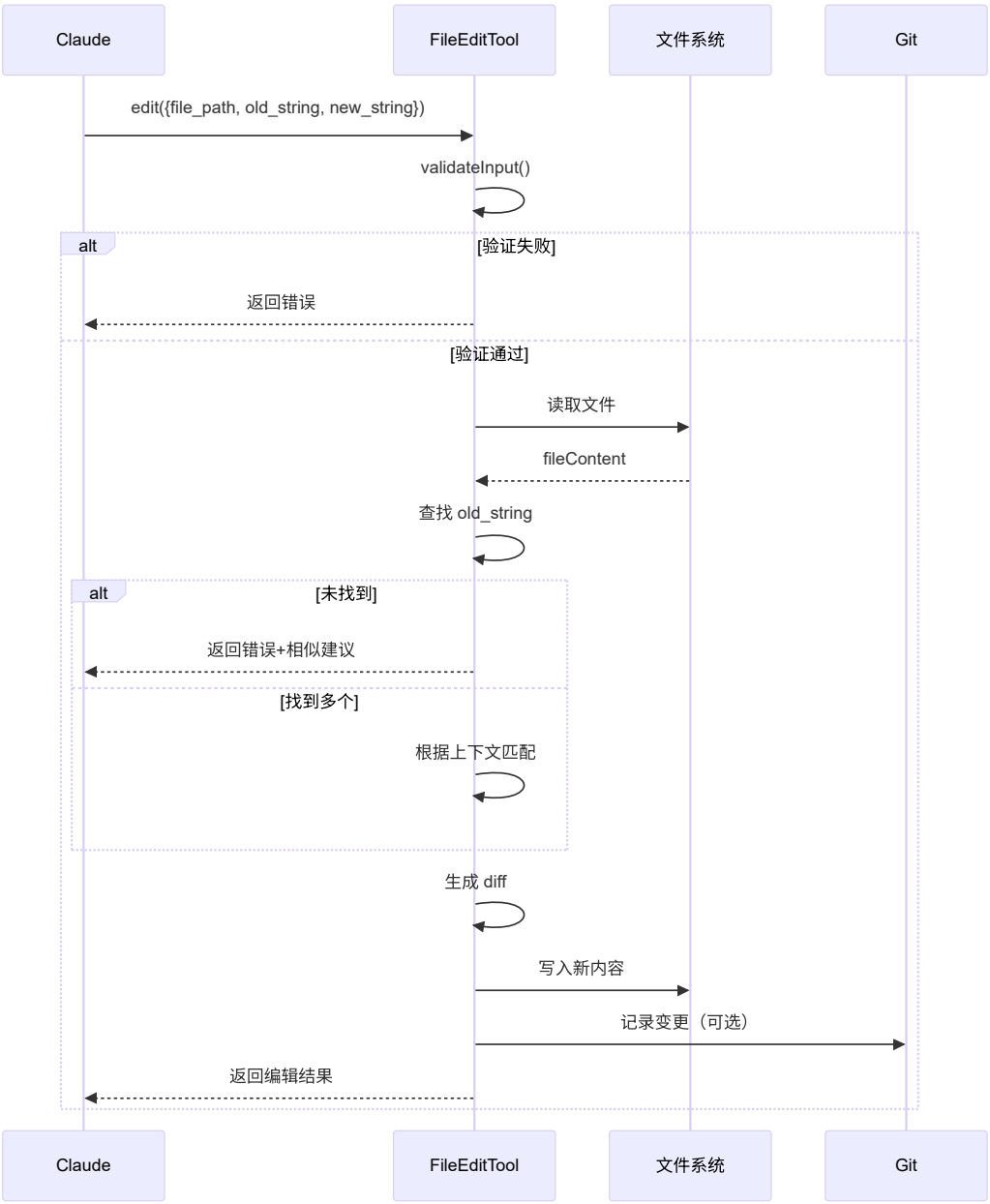
```
    return {
      stdout: result.stdout,
      stderr: result.stderr,
      exit_code: result.exitCode,
      duration_ms: result.duration,
    };
  },

  // 语义分析：判断命令类型
  isSearchOrReadCommand(command: string): {
    isSearch: boolean;
    isRead: boolean;
    isList: boolean;
  } {
    // 解析命令，识别是否为查询类操作
    const parts = splitCommandWithOperators(command);

    // 搜索命令：grep, find, rg, ag, etc.
    const searchCommands = new Set(['find', 'grep', 'rg', 'ag', 'ack']);

    // 读取命令：cat, head, tail, etc.
    const readCommands = new Set(['cat', 'head', 'tail', 'less', 'more']);

    // 列表命令：ls, tree, du
    const listCommands = new Set(['ls', 'tree', 'du']);

    // 分析命令中的每个部分
    return analyzeCommandParts(parts, { searchCommands, readCommands,
listCommands });
  },
});
```

BashTool 的精妙之处：

1. **语义感知**：不是盲目执行命令，而是理解命令的意图
2. **安全沙箱**：默认在受限环境中运行，防止意外破坏
3. **智能分类**：自动识别搜索/读取命令，优化 UI 展示
4. **后台支持**：长时间运行的命令可以放入后台

FileEditTool：结构化编辑的艺术

FileEditTool 是 Claude Code 最常用的工具之一，它实现了**结构化的 find-and-replace**：



FileEditTool 的核心实现：

```
// src/tools/FileEditTool/FileEditTool.ts

export const FileEditTool = buildTool({
  name: FILE_EDIT_TOOL_NAME,
  maxResultSizeChars: 100_000,
  strict: true,

  inputSchema: z.object({
    file_path: z.string()
      .describe('The absolute path to the file'),
    old_string: z.string()
      .describe('The existing text to replace'),
    new_string: z.string()
      .describe('The new text to insert'),
```

```
    replace_all: z.boolean().optional()
      .describe('Replace all occurrences'),
  }),

  async call(input, context) {
    const { file_path, old_string, new_string, replace_all } = input;

    // 1. 安全检查
    if (old_string === new_string) {
      return { error: 'No changes to make: old_string and new_string are the
same.' };
    }

    // 2. 读取文件（带编码检测）
    const fileBuffer = await fs.readFileBytes(file_path);
    const encoding = detectEncoding(fileBuffer);
    let content = fileBuffer.toString(encoding);

    // 3. 规范化换行符
    content = content.replaceAll('\r\n', '\n');

    // 4. 执行替换
    let newContent: string;
    if (replace_all) {
      newContent = content.replaceAll(old_string, new_string);
    } else {
      // 智能匹配：当 old_string 有多个匹配时，使用上下文确定位置
      const matches = findAllMatches(content, old_string);
      if (matches.length === 0) {
        // 未找到，尝试模糊匹配
        const suggestion = findSimilarString(content, old_string);
        return { error: `old_string not found. Did you mean: ${suggestion}?`
};
      }
      if (matches.length === 1) {
        newContent = content.replace(old_string, new_string);
      } else {
        // 多个匹配，需要更多上下文
        return {
          error: `Multiple matches found (${matches.length}). Please provide
more context.`,
          matches: matches.map(m => ({ line: m.line, preview: m.preview })),
        };
```

```
      }
    }

    // 5. 写入文件（保留原始换行符风格）
    await writeTextContent(file_path, newContent, { encoding, lineEnding:
detectLineEndings(fileBuffer) });

    // 6. 返回结构化结果
    return {
      path: file_path,
      diff: generateDiff(content, newContent),
      linesChanged: countLinesChanged(content, newContent),
    };
  },

  // 检查两个编辑是否等价（用于去重）
  inputsEquivalent(a: FileEditInput, b: FileEditInput): boolean {
    return a.file_path === b.file_path &&
           a.old_string === b.old_string &&
           a.new_string === b.new_string;
  },

  // 标记为破坏性操作（影响权限系统）
  isDestructive(): boolean {
    return true; // 文件编辑总是破坏性的
  },
});
```

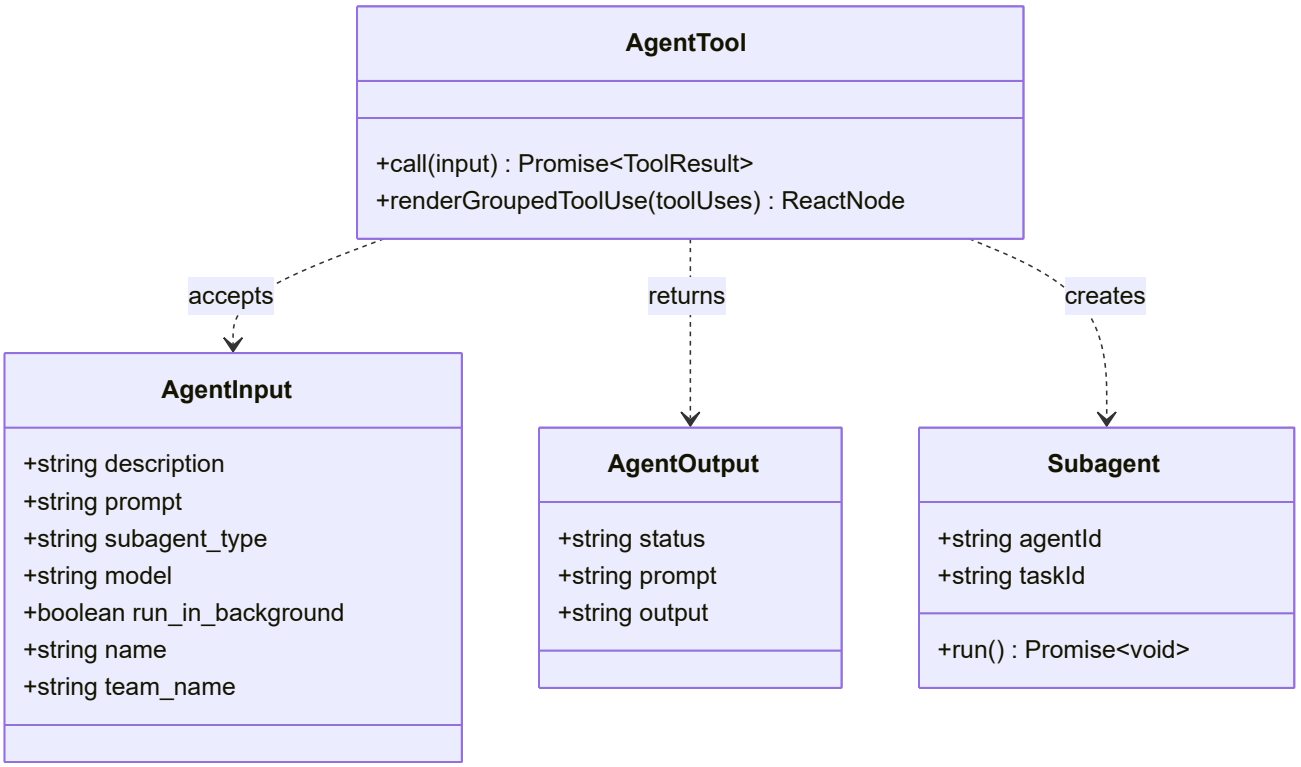
FileEditTool 的设计亮点：

特性	说明	好处
结构化编辑	old_string + new_string 模式	精确、可验证、可回滚
编码感知	自动检测 UTF-8/UTF-16	正确处理各种文件
换行符保留	保留原始换行符风格	避免不必要的变更
智能匹配	上下文感知的模糊匹配	减少因空白字符导致的失败
破坏性标记	明确标记文件变更	权限系统可以特殊处理

3.2.2 元工具（Meta Tools）

元工具是"关于工具的工具"，它们操作 Claude Code 本身的状态和行为。

AgentTool：多智能体的入口



AgentTool 的核心逻辑：

```
// src/tools/AgentTool/AgentTool.tsx

export const AgentTool = buildTool({
  name: AGENT_TOOL_NAME,
  aliases: [LEGACY_AGENT_TOOL_NAME],

  inputSchema: z.object({
    description: z.string()
      .describe('A short (3-5 word) description of the task'),
    prompt: z.string()
      .describe('The task for the agent to perform'),
    subagent_type: z.string().optional()
      .describe('The type of specialized agent to use'),
    model: z.enum(['sonnet', 'opus', 'haiku']).optional()
      .describe('Optional model override'),
    run_in_background: z.boolean().optional()
      .describe('Set to true to run this agent in the background'),
    name: z.string().optional()
      .describe('Name for the spawned agent'),
    team_name: z.string().optional()
      .describe('Team name for spawning'),
  }),
```

```
  async call(input, context) {
    const { description, prompt, subagent_type, model, run_in_background,
name } = input;

    // 1. 创建子智能体上下文
    const agentId = createAgentId();
    const agentContext = createSubagentContext({
      parentContext: context,
      agentId,
      model,
    });

    // 2. 构建系统提示词
    const systemPrompt = await buildSystemPrompt({
      tools: agentContext.tools,
      subagent_type,
    });

    // 3. 启动子智能体
    if (run_in_background) {
      // 后台模式：创建异步任务
      const task = await spawnAsyncAgent({
        agentId,
        description,
        prompt,
        systemPrompt,
        context: agentContext,
      });

      return {
        status: 'async_launched',
        agentId,
        description,
        prompt,
        outputFile: task.outputPath,
      };
    }

    // 4. 同步模式：直接执行并等待结果
    const result = await runAgent({
      agentId,
      description,
```

```
      prompt,
      systemPrompt,
      context: agentContext,
      onProgress: (progress) => {
        // 转发进度到父智能体
        context.onProgress?.(progress);
      },
    });

    return {
      status: 'completed',
      prompt: result.prompt,
      output: result.output,
    };
  },

  // 批量渲染多个子智能体的 UI
  renderGroupedToolUse(toolUses, options): React.ReactNode {
    // 优化：多个子智能体并行运行时，显示为紧凑的列表
    return (
      <AgentGroupProgress
        agents={toolUses.map(t => t.agent)}
        shouldAnimate={options.shouldAnimate}
      />
    );
  },
});
```

TaskCreate/Update 工具：任务管理

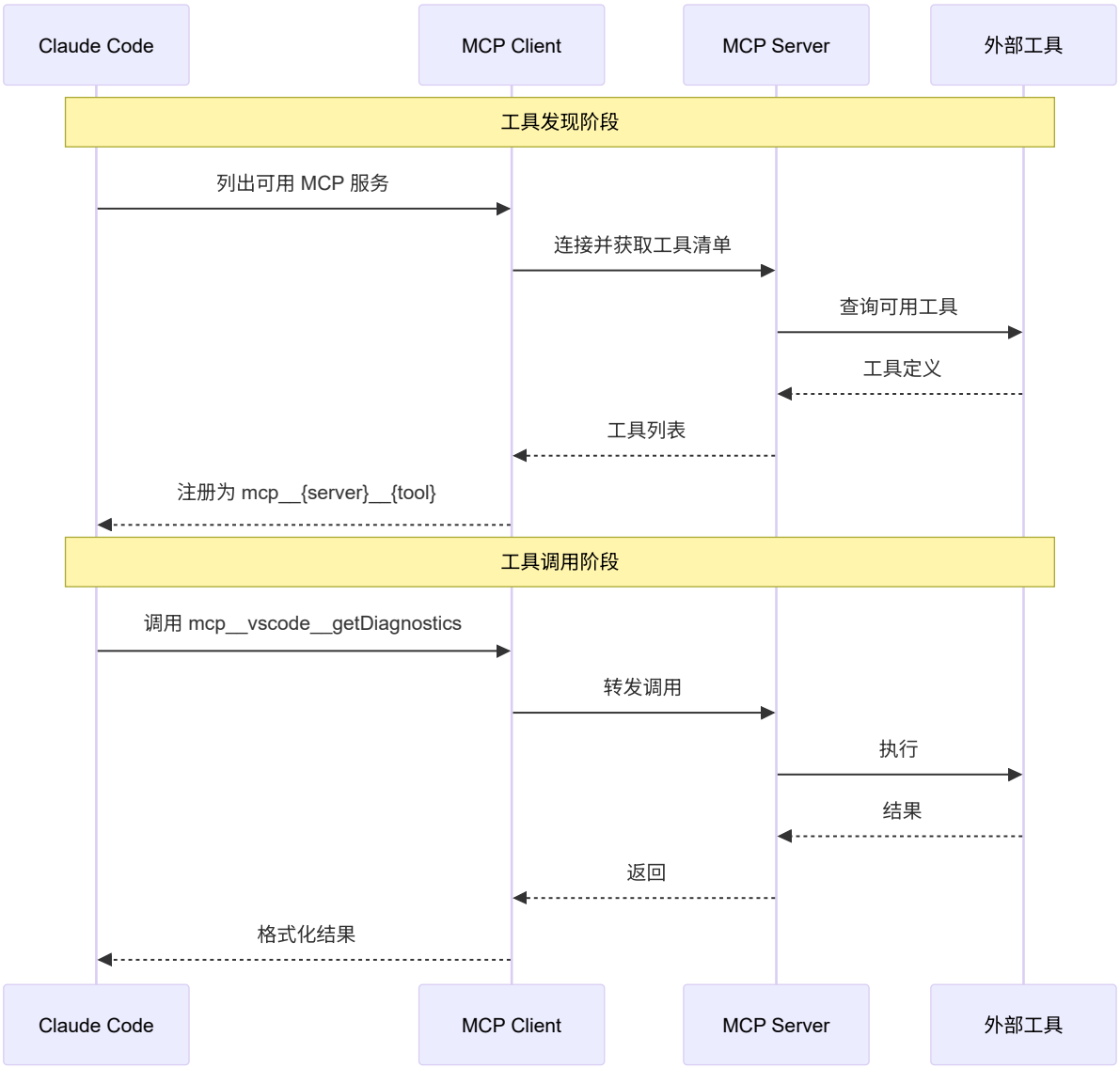
任务工具实现了轻量级的任务追踪：

```
// src/tools/TaskCreateTool/TaskCreateTool.ts

export const TaskCreateTool = buildTool({
  name: TASK_CREATE_TOOL_NAME,

  inputSchema: z.object({
    subject: z.string()
      .describe('A brief title for the task'),
    description: z.string()
      .describe('What needs to be done'),
    status: z.enum(['pending', 'in_progress', 'completed'])
```

```
      .default('pending'),
  }),

  async call(input, context) {
    const taskId = generateTaskId();
    const task: Task = {
      id: taskId,
      subject: input.subject,
      description: input.description,
      status: input.status,
      createdAt: Date.now(),
    };

    // 更新全局状态
    context.setAppState(prev => ({
      ...prev,
      tasks: {
        ...prev.tasks,
        [taskId]: task,
      },
    }));

    return {
      taskId,
      status: 'created',
      task,
    };
  },
});
```

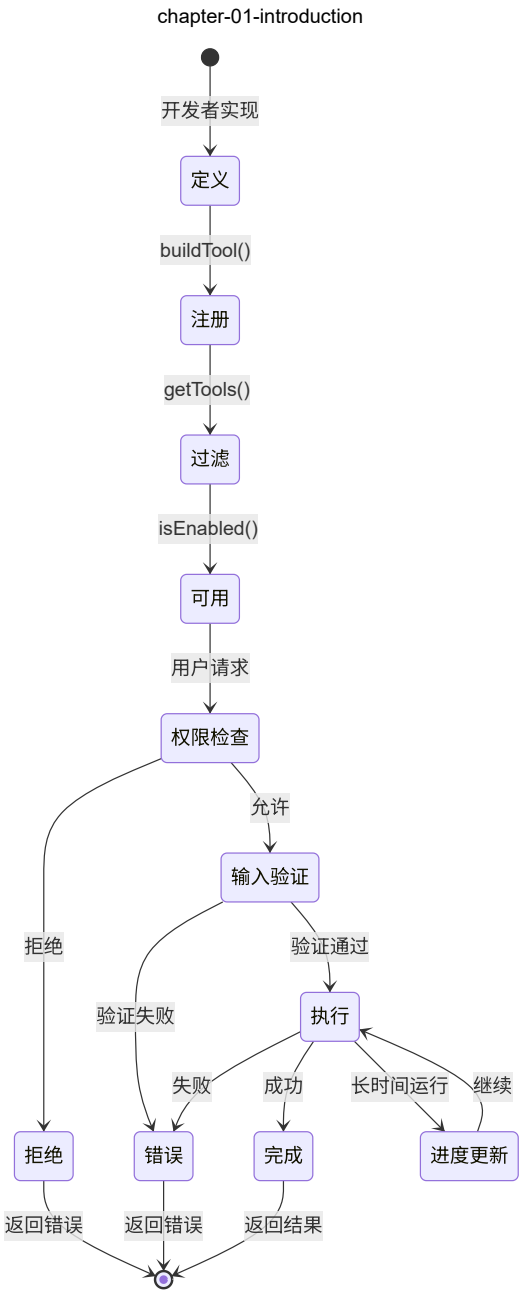
3.2.3 MCP 工具

MCP (Model Context Protocol) 工具是外部系统通过标准协议接入的能力。它们在设计上与原生工具有所不同：



3.3 工具生命周期

一个工具从定义到执行经历完整的生命周期：



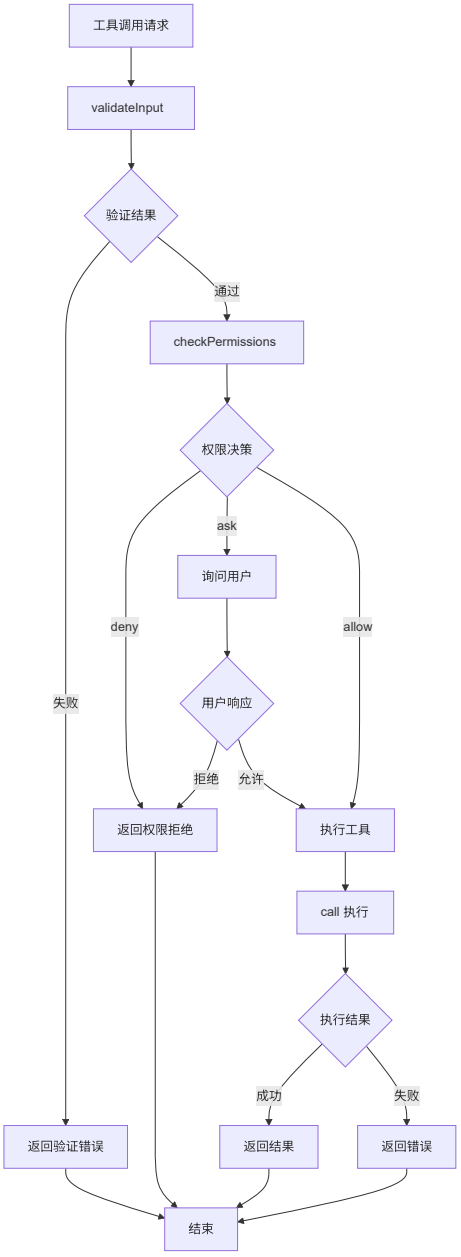
3.3.1 注册与发现

工具的注册发生在应用启动时：

```
// src/tools.ts

export function getAllBaseTools(): Tools {
  const tools = [
    // 核心工具
    AgentTool,
    TaskOutputTool,
    BashTool,
    FileEditTool,
    FileReadTool,
    FileWriteTool,
    GlobTool,
```

```
    GrepTool,
    WebFetchTool,
    WebSearchTool,

    // 条件编译的工具
    ...(feature('AGENT_TRIGGERS') ? cronTools : []),
    ...(feature('KAIROS') ? [SendUserFileTool] : []),

    // 延迟加载的工具
    getPowerShellTool(),

    // ... 更多工具
  ].filter(Boolean); // 过滤掉 null/undefined

  return tools.map(tool => {
    // 应用工具权限过滤
    if (shouldFilterTool(tool)) {
      return { ...tool, isEnabled: () => false };
    }
    return tool;
  });
}
```

3.3.2 权限检查流程



3.4 工具开发最佳实践

3.4.1 工具实现模板

```
// 1. 定义输入输出类型
// types.ts
import { z } from 'zod/v4';

export const inputSchema = z.object({
  // 定义你的输入字段
  param1: z.string().describe('参数1的描述'),
  param2: z.number().optional().describe('可选参数'),
});

export const outputSchema = z.object({
  // 定义输出字段
```

```
  result: z.string(),
  metadata: z.object({
    duration: z.number(),
  }),
});

export type MyToolInput = z.infer<typeof inputSchema>;
export type MyToolOutput = z.infer<typeof outputSchema>;

// 2. 实现工具逻辑
// index.ts
import { buildTool } from '../../Tool.js';
import { inputSchema, outputSchema, type MyToolInput, type MyToolOutput }
from './types.js';

export const MyTool = buildTool({
  name: 'MyTool',
  description: '工具的简短描述',

  inputSchema,
  outputSchema,

  // 权限检查
  async checkPermissions(input, context) {
    // 实现权限检查逻辑
    return { behavior: 'allow', updatedInput: input };
  },

  // 输入验证
  async validateInput(input, context) {
    // 实现额外的验证逻辑
    if (input.param1.length === 0) {
      return {
        result: false,
        message: 'param1 cannot be empty',
      };
    }
    return { result: true };
  },

  // 核心执行
  async call(input, context, canUseTool, assistantMessage, onProgress) {
    const startTime = Date.now();
```

```
    // 执行工具逻辑
    const result = await doSomething(input);

    // 报告进度（如果长时间运行）
    onProgress?.({
      type: 'my_tool_progress',
      percent: 50,
    });

    return {
      result: result.data,
      metadata: {
        duration: Date.now() - startTime,
      },
    };
  },

  // UI 渲染（可选但推荐）
  renderToolUseMessage(input) {
    return <MyToolUseView input={input} />;
  },

  renderToolResultMessage(output) {
    return <MyToolResultView output={output} />;
  },
});
```

3.4.2 测试策略

```
// MyTool.test.ts
import { describe, it, expect, vi } from 'vitest';
import { MyTool } from './index.js';

describe('MyTool', () => {
  it('should validate input correctly', async () => {
    const result = await MyTool.validateInput(
      { param1: '', param2: 42 },
      mockContext
    );
    expect(result.result).toBe(false);
    expect(result.message).toContain('cannot be empty');
  });
```

```
  it('should execute successfully', async () => {
    const mockCanUseTool = vi.fn().mockResolvedValue({ behavior: 'allow' });

    const result = await MyTool.call(
      { param1: 'test', param2: 42 },
      mockContext,
      mockCanUseTool
    );

    expect(result.result).toBeDefined();
    expect(result.metadata.duration).toBeGreaterThan(0);
  });

  it('should report progress for long operations', async () => {
    const onProgress = vi.fn();

    await MyTool.call(
      { param1: 'test' },
      mockContext,
      mockCanUseTool,
      undefined,
      onProgress
    );

    expect(onProgress).toHaveBeenCalled();
  });
});
```

3.5 本章小结

本章深入探讨了 Claude Code 的工具系统：

1. **工具哲学**：工具让 AI 从"谈论"到"做事"
2. **三大类别**：核心工具、元工具、MCP 工具各有侧重
3. **生命周期**：从注册到执行，每个环节精心设计
4. **开发实践**：标准化的实现模式和测试策略

工具系统是 Claude Code 的"能力层"，它让 AI 能够真正与外部世界交互。在下一章中，我们将探讨如何安全地管理这些能力——权限系统。

延伸阅读：

- Anthropic Tool Use Guide
- Model Context Protocol Spec
- Building CLI Tools with Node.js

← 上一章：核心架构 | 下一章：权限系统深度解析 →

chapter-04-permissions

第4章 权限系统深度解析

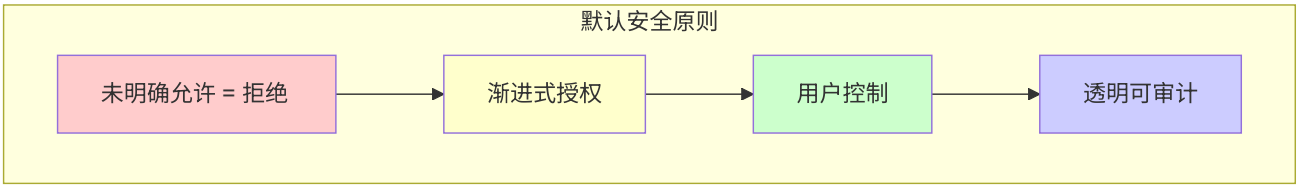
> "安全不是阻碍，而是信任的基石。"
> —— 《Claude Code 设计哲学》

权限系统是 Claude Code 的安全基石。它让用户能够在享受 AI 强大能力的同时，保持对系统的完全控制。本章将深入探讨权限系统的设计理念、实现机制和实践应用。

4.1 权限哲学

4.1.1 默认安全原则

Claude Code 的权限系统建立在"默认安全"原则之上：

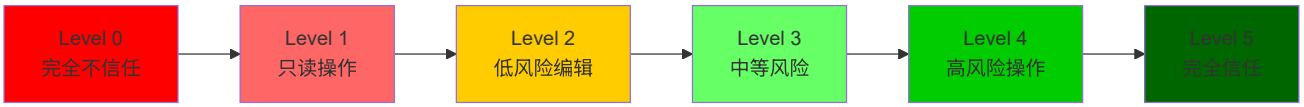


对比传统权限模型：

模型	默认行为	授权方式	适用场景
黑名单模型	全部允许，拒绝特定项	列出禁止项	开放环境
白名单模型	全部拒绝，允许特定项	列出允许项	安全环境
Claude Code 模型	交互式确认	规则+分类器+用户确认	AI 助手

4.1.2 渐进式信任

渐进式信任是 Claude Code 权限系统的核心设计思想：



信任级别的实际体现：

级别	权限模式	自动允许	需要确认	完全拒绝
0	`default`	无	所有操作	无
1	`default` + 规则	`FileRead *.md`	其他	危险命令
2	`plan`	无	所有操作（带计划）	危险命令
3	`auto`	安全操作	可疑操作	危险命令
4	`acceptEdits`	所有编辑	破坏性操作	危险命令
5	`bypass`	所有操作	无	无

4.2 权限模式详解

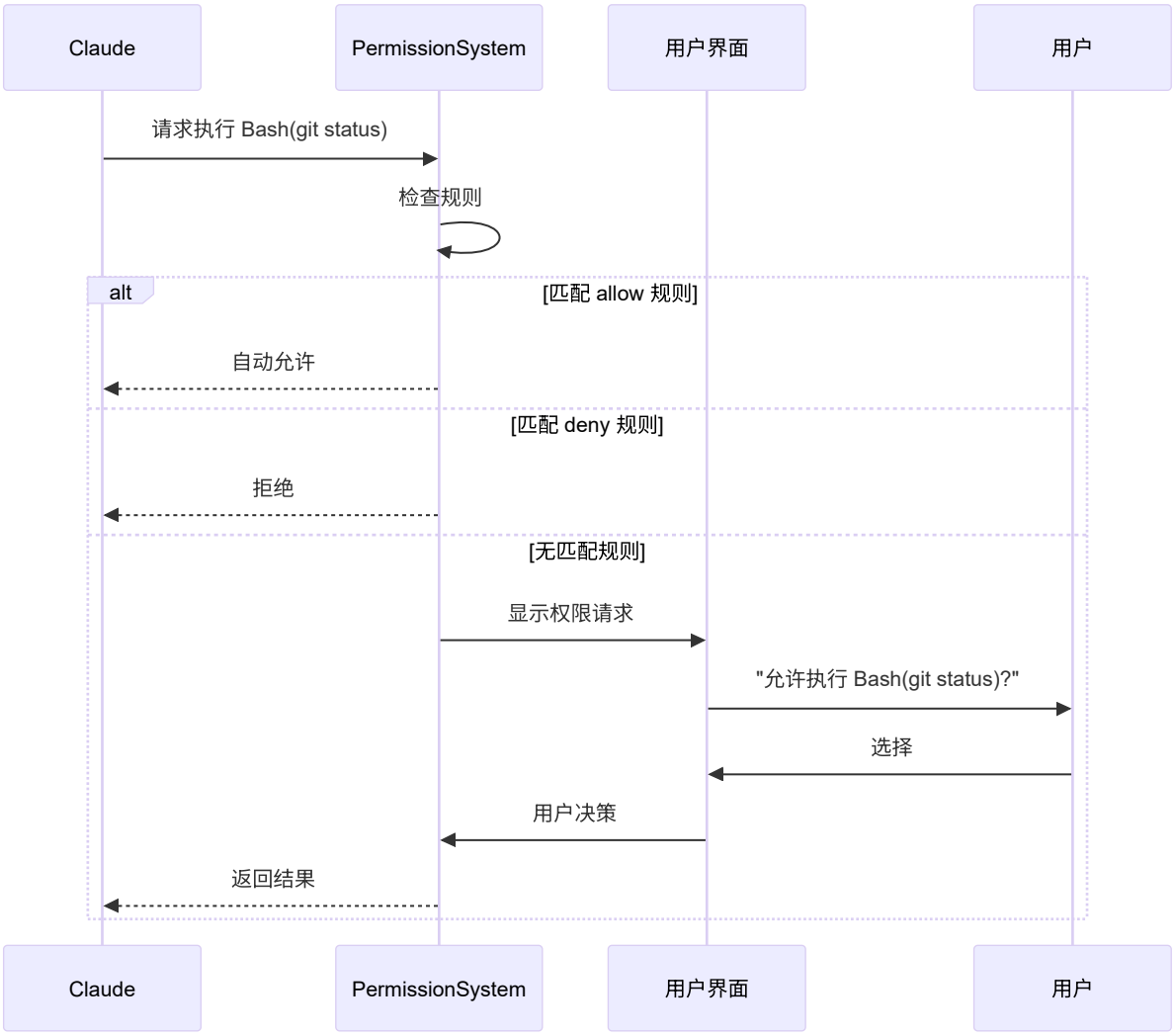
4.2.1 权限模式全景

```
// src/types/permissions.ts

export type PermissionMode =
  | 'default'       // 标准模式，每次询问
  | 'auto'        // 自动模式，AI 判断安全则允许
  | 'plan'        // 计划模式，先展示再执行
  | 'bypass'      // 绕过模式（危险）
  | 'acceptEdits' // 自动接受编辑
  | 'dontAsk'      // 静默执行

export const INTERNAL_PERMISSION_MODES = [
  ...EXTERNAL_PERMISSION_MODES,
  ...(feature('TRANSCRIPT_CLASSIFIER') ? (['auto'] as const) : ([] as
const)),
] as const satisfies readonly PermissionMode[]
```

4.2.2 default 模式

`default` 模式是 Claude Code 的默认权限模式，也是最适合新用户的模式。

工作流程：



配置示例：

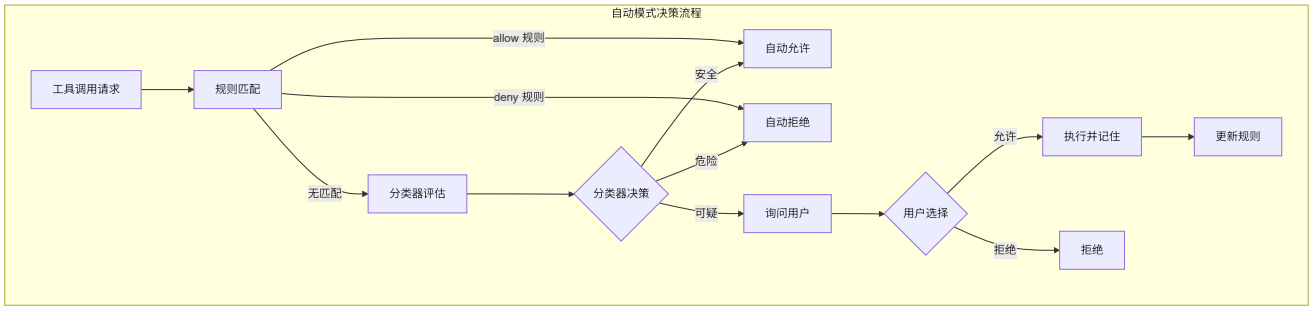
```
// ~/.claude/settings.json
{
  "permissions": {
    "mode": "default",
    "alwaysAllow": [
      { "tool": "Bash", "pattern": "git status" },
      { "tool": "Bash", "pattern": "git diff" },
      { "tool": "FileRead", "pattern": "*.md" }
    ],
    "alwaysDeny": [
      { "tool": "Bash", "pattern": "rm -rf /" }
    ]
```

```
  }
}
```

4.2.3 auto 模式与分类器

`auto` 模式是 Claude Code 最智能的权限模式。它使用 AI 分类器来判断操作是否安全。

分类器架构：



分类器实现：

```
// src/utils/permissions/yoloClassifier.ts

interface ClassifierInput {
  toolName: string;
  toolInput: unknown;
  command?: string;
  descriptions: string[];
  cwd: string;
}

interface ClassifierOutput {
  decision: 'allow' | 'ask' | 'deny';
  confidence: number;
```

```
  reason: string;
}

async function classifyYoloAction(
  input: ClassifierInput,
  context: ToolUseContext
): Promise<ClassifierOutput> {
  // 构建分类器提示词
  const prompt = buildClassifierPrompt(input);

  // 调用轻量级模型进行分类
  const response = await anthropic.messages.create({
    model: 'claude-3-haiku-20240307',
    messages: [{ role: 'user', content: prompt }],
    max_tokens: 150,
  });

  // 解析分类结果
  return parseClassifierResponse(response);
}

// 分类器提示词示例
function buildClassifierPrompt(input: ClassifierInput): string {
  return `
You are a security classifier for an AI coding assistant.
Analyze the following tool use and classify it as SAFE, SUSPICIOUS, or
DANGEROUS.

Tool: ${input.toolName}
Input: ${JSON.stringify(input.toolInput)}
Working Directory: ${input.cwd}

Consider:
1. Does this modify system files?
2. Does this access sensitive data?
3. Does this execute arbitrary code?
4. Is this a standard development operation?

Respond with JSON:
{
  "decision": "allow|ask|deny",
  "confidence": 0-1,
  "reason": "brief explanation"
```

```
}
`;
}
```

分类器的优势：

优势	说明
智能判断	理解操作语义，不只是模式匹配
上下文感知	考虑工作目录、项目类型等
学习适应	根据用户反馈改进
轻量快速	使用轻量级模型，延迟 < 100ms

4.2.4 plan 模式

`plan` 模式要求 AI 在执行前制定详细计划，用户可以审查后再执行。

工作流程：

```
Error parsing Mermaid diagram!

Parse error on line 14:
...            2. 分析依赖关系              3. 重命
-----------------------^
Expecting 'SOLID_OPEN_ARROW', 'DOTTED_OPEN_ARROW', 'SOLID_ARROW',
'BIDIRECTIONAL_SOLID_ARROW', 'DOTTED_ARROW', 'BIDIRECTIONAL_DOTTED_ARROW',
'SOLID_CROSS', 'DOTTED_CROSS', 'SOLID_POINT', 'DOTTED_POINT', got 'NEWLINE'
```

计划模式的价值：

1. **透明性**：用户清楚知道 AI 要做什么
2. **可控性**：可以修改或跳过某些步骤
3. **学习性**：用户可以从中学到重构策略
4. **安全性**：破坏性操作必须显式批准

4.3 规则系统

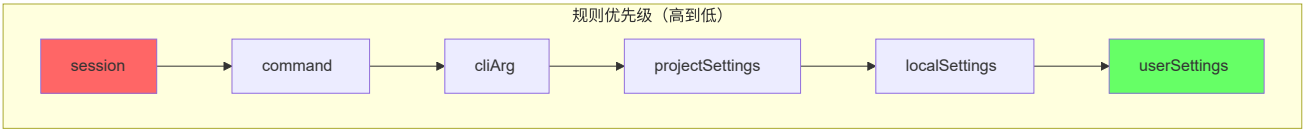
4.3.1 规则结构

```
// src/types/permissions.ts

export type PermissionRule = {
```

```
  source: PermissionRuleSource  // 规则来源
  ruleBehavior: PermissionBehavior  // allow | deny | ask
  ruleValue: PermissionRuleValue    // 规则内容
}

export type PermissionRuleSource =
  | 'userSettings'    // ~/.claude/settings.json
  | 'projectSettings' // .claude/settings.json
  | 'localSettings'   // 本地配置
  | 'cliArg'          // 命令行参数
  | 'command'         // 会话中动态添加
  | 'session'         // 会话级别

export type PermissionRuleValue = {
  toolName: string      // 工具名
  ruleContent?: string  // 可选的模式
}
```

4.3.2 规则优先级

当多个规则冲突时，Claude Code 使用优先级系统：



示例场景：

```
// ~/.claude/settings.json (userSettings)
{
  "permissions": {
    "alwaysAllow": [
      { "tool": "Bash", "pattern": "npm install" }
    ]
  }
}

// .claude/settings.json (projectSettings)
{
  "permissions": {
    "alwaysDeny": [
```

```
      { "tool": "Bash", "pattern": "npm install" }
    ]
  }
}
```

结果： `npm install` 被拒绝，因为 `projectSettings` 优先级更高。

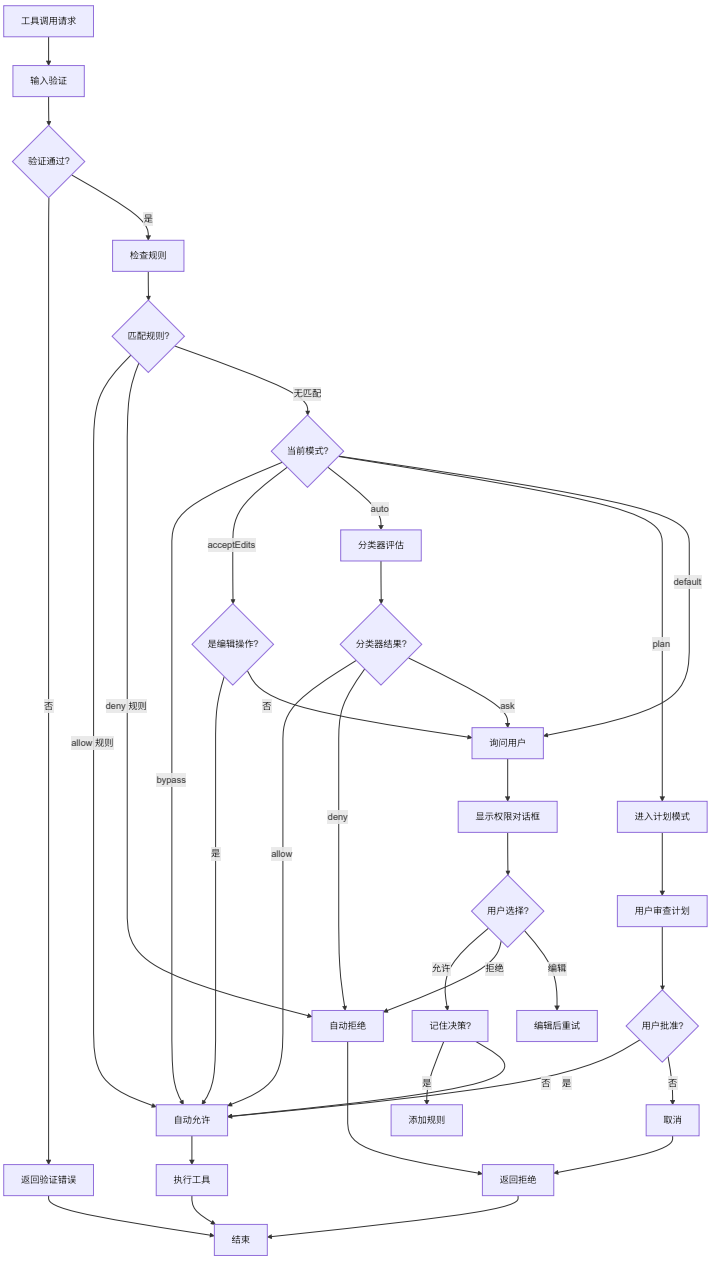
4.3.3 规则匹配算法

```
// src/utils/permissions/permissions.ts

function matchingRuleForInput(
  input: string,
  context: ToolPermissionContext,
  toolName: string,
  behavior: PermissionBehavior
): PermissionRule | null {
  // 按优先级顺序检查规则来源
  const sources: PermissionRuleSource[] = [
    'session', 'command', 'cliArg',
    'policySettings', 'projectSettings',
    'localSettings', 'userSettings'
  ];

  for (const source of sources) {
    const rules = context[`always${capitalize(behavior)}Rules`][source];
    if (!rules) continue;

    for (const ruleString of rules) {
      const rule = parseRule(ruleString);
      if (rule.toolName !== toolName) continue;

      // 检查内容模式匹配
      if (rule.ruleContent) {
        if (matchWildcardPattern(rule.ruleContent, input)) {
          return { source, ruleBehavior: behavior, ruleValue: rule };
        }
      } else {
        // 无内容规则匹配整个工具
        return { source, ruleBehavior: behavior, ruleValue: rule };
      }
    }
  }
```

```
  return null;
}

// 通配符匹配支持 * 和 ?
function matchWildcardPattern(pattern: string, input: string): boolean {
  const regex = new RegExp(
    '^' + pattern.replace(/\*/g, '.*').replace(/\?/g, '.') + '$'
  );
  return regex.test(input);
}
```

4.4 权限决策流程

4.4.1 完整决策流程图



4.4.2 权限上下文

```
// src/Tool.ts

export interface ToolPermissionContext {
  mode: PermissionMode

  // 按来源组织的规则
  alwaysAllowRules: PermissionRulesBySource
  alwaysDenyRules: PermissionRulesBySource
  alwaysAskRules: PermissionRulesBySource

  // 额外工作目录
  additionalWorkingDirectories: Map<string, AdditionalWorkingDirectory>

  // 是否可用绕过模式
  isBypassPermissionsModeAvailable: boolean
}

// 权限上下文随会话状态变化
export interface ToolUseContext {
  options: {
    tools: Tools
    commands: Command[]
    mainLoopModel: string
    // ...
  }

  getAppState(): AppState
  setAppState(f: (prev: AppState) => AppState): void

  // 工具权限上下文
  toolPermissionContext: ToolPermissionContext

  // 其他上下文...
}
```

4.5 拒绝追踪与防护

4.5.1 连续拒绝防护

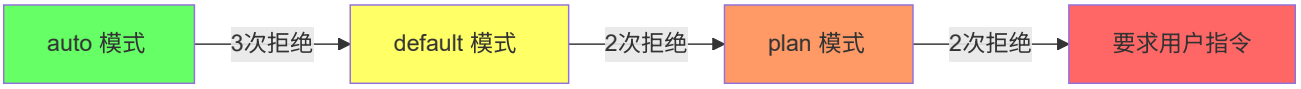
Claude Code 实现了连续拒绝追踪机制，防止 AI 在权限被拒绝后反复尝试相同操作。

```ts
// src/utils/permissions/denialTracking.ts

interface DenialTrackingState {
  // 最近拒绝的工具调用
  recentDenials: Array<{
    toolName: string
    input: unknown
    timestamp: number
  }>

  // 连续拒绝计数
  consecutiveDenialCount: number

  // 最后一次拒绝时间
  lastDenialTime: number
}

const DENIAL_LIMITS = {
  // 连续拒绝阈值
  MAX_CONSECUTIVE_DENIALS: 3,

  // 拒绝冷却时间（毫秒）
  DENIAL_COOLDOWN_MS: 5000,

  // 最大拒绝历史
  MAX_DENIAL_HISTORY: 10,
}

function shouldFallbackToPrompting(
  state: DenialTrackingState,
  toolName: string,
  input: unknown
): boolean {
  // 检查是否超过连续拒绝阈值
  if (state.consecutiveDenialCount >= DENIAL_LIMITS.MAX_CONSECUTIVE_DENIALS)
{
    return true
  }

  // 检查是否在冷却期内
  const timeSinceLastDenial = Date.now() - state.lastDenialTime
  if (timeSinceLastDenial < DENIAL_LIMITS.DENIAL_COOLDOWN_MS) {
    return true
```

```
  }

  // 检查是否是重复拒绝
  const isDuplicate = state.recentDenials.some(
    d => d.toolName === toolName && isEqual(d.input, input)
  )
  if (isDuplicate) {
    return true
  }

  return false
}
```

4.5.2 降级策略

当检测到连续拒绝时，系统会降级到更保守的策略：



4.6 本章小结

本章深入探讨了 Claude Code 的权限系统：

1. **默认安全原则**：未明确允许即拒绝
2. **渐进式信任**：从 default 到 bypass 的多级权限模式
3. **智能分类器**：auto 模式使用 AI 判断操作安全性
4. **灵活的规则系统**：支持通配符、多来源、优先级
5. **连续拒绝防护**：防止反复尝试相同操作

权限系统让 Claude Code 在强大与安全之间取得了平衡。用户可以根据自己的信任程度选择合适的权限模式，既能享受 AI 的便利，又能保持对系统的控制。

在下一章中，我们将探讨 Claude Code 如何管理复杂的应用状态——状态管理系统。

延伸阅读：

- Principle of Least Privilege
- OAuth 2.0 Authorization Framework
- RBAC vs ABAC

---

chapter-05-state

第5章 状态管理与响应式架构

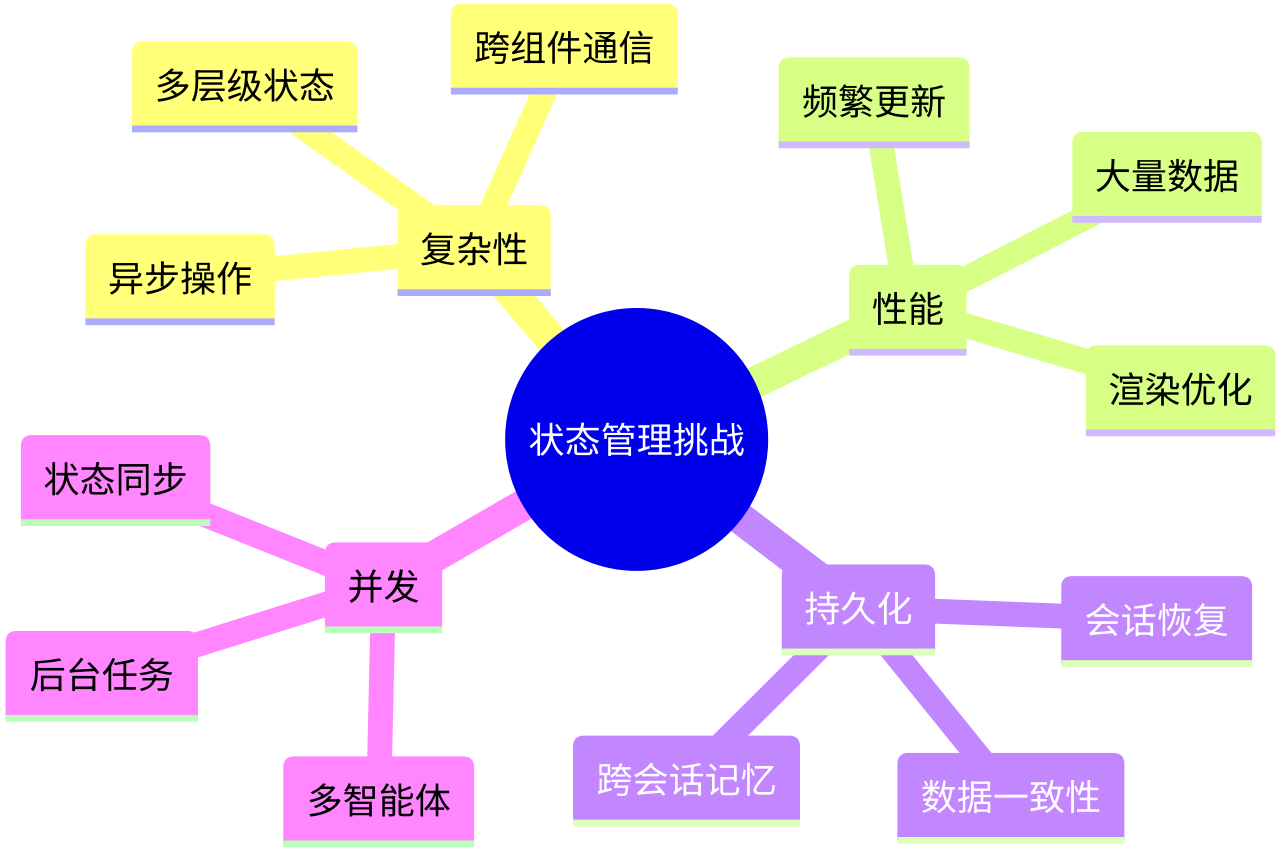
> "状态是应用的血液，良好的状态管理让系统流动起来。"
> —— 《Claude Code 设计哲学》

Claude Code 需要管理极其复杂的状态：对话历史、工具执行状态、用户偏好、MCP 连接、权限上下文、多智能体状态等。本章将深入探讨其状态管理的架构设计、实现机制和最佳实践。

5.1 状态管理的挑战

5.1.1 为什么状态管理困难？

传统应用的状态管理相对简单，但 Claude Code 面临特殊挑战：

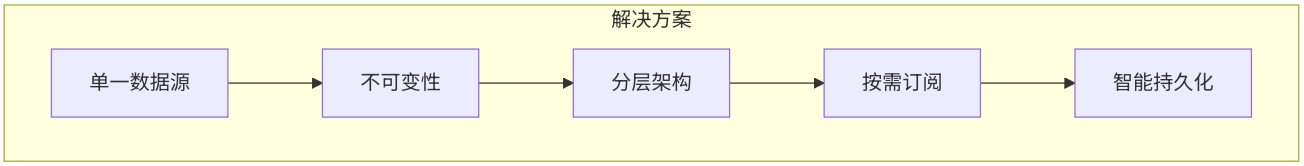


具体挑战：

挑战	描述	影响
状态规模	长对话可能有数千条消息	内存占用、渲染性能
更新频率	流式响应每秒多次更新	渲染优化、状态合并

挑战	描述	影响
异步操作	工具执行、MCP 调用都是异步	竞态条件、状态一致性
多智能体	子智能体有自己的状态	状态隔离、通信机制

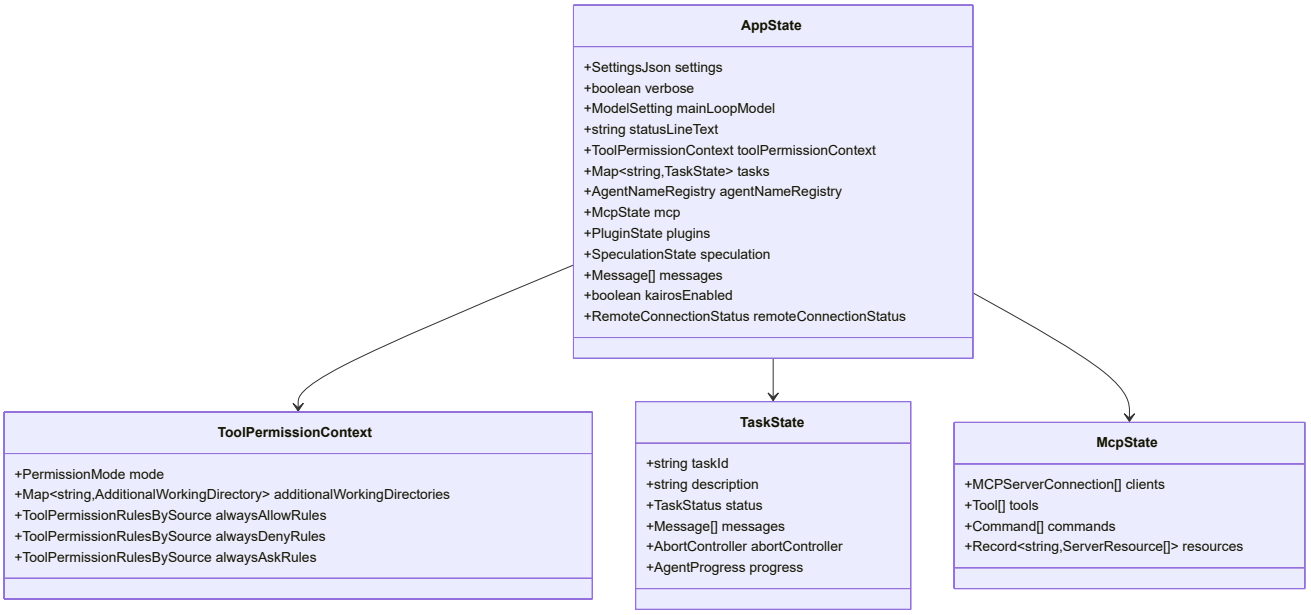
5.1.2 Claude Code 的解决方案



5.2 核心架构

5.2.1 AppState 全景

AppState 是 Claude Code 的单一数据源，包含所有应用状态：



核心状态字段详解：

```
// src/state/AppStateStore.ts

export type AppState = DeepImmutable<{
  // === 基础设置 ===
  settings: SettingsJson            // 用户设置
  verbose: boolean                  // 详细模式
  mainLoopModel: ModelSetting       // 当前模型
```

```
  // === UI 状态 ===
  statusLineText: string | undefined
  expandedView: 'none' | 'tasks' | 'teammates'
  isBriefOnly: boolean

  // === 权限状态 ===
  toolPermissionContext: ToolPermissionContext

  // === 任务状态 ===
  // Unified task state - excluded from DeepImmutable
  tasks: { [taskId: string]: TaskState }
  foregroundedTaskId?: string
  viewingAgentTaskId?: string

  // === MCP 状态 ===
  mcp: {
    clients: MCPServerConnection[]
    tools: Tool[]
    commands: Command[]
    resources: Record<string, ServerResource[]>
    pluginReconnectKey: number
  }

  // === 插件状态 ===
  plugins: {
    enabled: LoadedPlugin[]
    disabled: LoadedPlugin[]
    commands: Command[]
    errors: PluginError[]
  }

  // === 推测执行状态 ===
  speculation: SpeculationState

  // === 远程连接状态 ===
  remoteSessionUrl: string | undefined
  remoteConnectionStatus: 'connecting' | 'connected' | 'reconnecting' |
'disconnected'
  remoteBackgroundTaskCount: number

  // ... 更多字段
}>
```

5.2.2 不可变性的价值

Claude Code 使用 TypeScript 的类型系统强制不可变性：

```
// src/types/utils.ts

// DeepImmutable 递归地将所有属性设为只读
type DeepImmutable<T> = {
  readonly [K in keyof T]: T[K] extends object
    ? T[K] extends Function
      ? T[K]
      : DeepImmutable<T[K]>
    : T[K]
}

// 使用示例
interface User {
  name: string
  address: {
    city: string
  }
}

type ImmutableUser = DeepImmutable<User>
// 等价于：
// {
//   readonly name: string
//   readonly address: {
//     readonly city: string
//   }
// }
```

不可变性的好处：

好处	说明
可预测性	状态只能通过 `setState` 更新
比较优化	引用相等即可判断状态变化
时间旅行调试	容易实现撤销/重做
并发安全	状态不会被意外修改

5.3 Store 实现

5.3.1 极简 Store 设计

Claude Code 没有使用 Redux 或 MobX，而是实现了极简的 Store：

```
// src/state/store.ts

export interface Store<T> {
  /** 获取当前状态 */
  getState(): T

  /** 更新状态（函数式更新） */
  setState(updater: (prev: T) => T): void

  /** 订阅状态变化 */
  subscribe(callback: (newState: T, oldState: T) => void): () => void
}

export function createStore<T>(
  initialState: T,
  onChange?: (args: { newState: T; oldState: T }) => void
): Store<T> {
  let state = initialState
  const listeners = new Set<(newState: T, oldState: T) => void>()

  return {
    getState: () => state,

    setState: (updater) => {
      const oldState = state
      state = updater(state)

      // 调用全局 onChange 回调
      onChange?.({ newState: state, oldState })

      // 通知所有订阅者
      listeners.forEach(listener => listener(state, oldState))
    },

    subscribe: (callback) => {
      listeners.add(callback)
      // 返回取消订阅函数
      return () => listeners.delete(callback)
    },
```

```
  }
}
```

与 Redux 对比：

特性	Redux	Claude Code Store
Actions	必需	无（直接函数更新）
Reducers	必需	无（内联更新函数）
Middleware	支持	通过 onChange 回调
DevTools	丰富	基础日志
Bundle Size	~10KB	~100B

5.3.2 React 集成

使用 React 18 的 `useSyncExternalStore` 实现高效订阅：

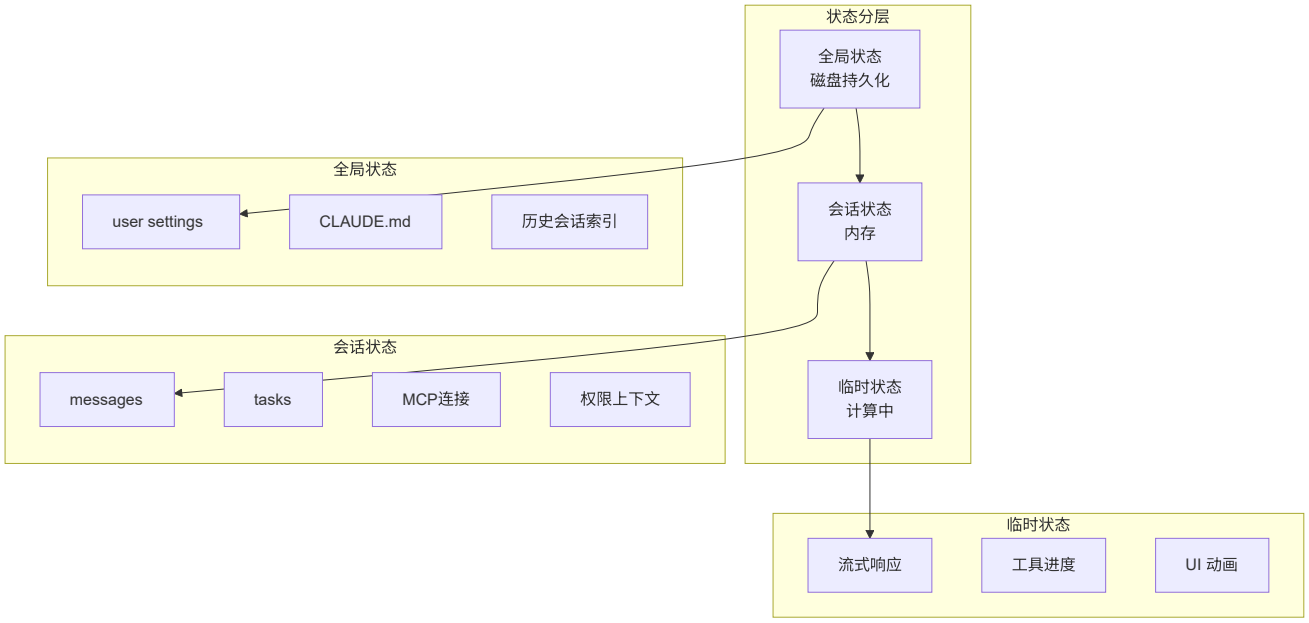
```
// src/state/AppState.tsx

const AppStoreContext = React.createContext<AppStateStore | null>(null)

export function useAppState<T>(
  selector: (state: AppState) => T
): T {
  const store = useContext(AppStoreContext)
  if (!store) {
    throw new Error('useAppState must be used within AppStateProvider')
  }

  // 使用 useCallback 缓存 selector 包装器
  const get = useCallback(() => {
    return selector(store.getState())
  }, [selector, store])

  // useSyncExternalStore 自动处理订阅和快照
  return useSyncExternalStore(
    store.subscribe,
    get,
    get  // server snapshot (same for SSR)
  )
}
```

性能优化的关键：

```
// ✅ 好的做法：选择原始值
const verbose = useAppState(s => s.verbose)

// ✅ 好的做法：选择稳定引用
const { text, promptId } = useAppState(s => s.promptSuggestion)

// ❌ 坏的做法：每次都创建新对象
const settings = useAppState(s => ({ ...s.settings }))
```

5.4 状态分层

5.4.1 三层状态架构



各层特性：

层级	存储	生命周期	恢复策略
全局状态	磁盘	永久	启动加载
会话状态	内存+磁盘	会话期间	崩溃恢复
临时状态	内存	短暂	无需恢复

5.4.2 状态选择器模式

```
// 基础选择器
export const selectSettings = (state: AppState) => state.settings
export const selectMessages = (state: AppState) => state.messages
export const selectTasks = (state: AppState) => state.tasks

// 派生选择器
export const selectTaskById = (taskId: string) =>
  (state: AppState) => state.tasks[taskId]

export const selectActiveTasks = (state: AppState) =>
  Object.values(state.tasks).filter(t => t.status === 'running')

export const selectMcpTools = (state: AppState) =>
  state.mcp.tools

// 使用选择器
function TaskList() {
  const tasks = useAppState(selectActiveTasks)
  // 只在 active tasks 变化时重渲染
  return <TaskListUI tasks={tasks} />
}
```

5.5 状态持久化

5.5.1 会话保存与恢复

```
// src/utils/sessionStorage.ts

interface SessionSnapshot {
  version: number
  timestamp: number
  sessionId: string
  messages: Message[]
  tasks: TaskState[]
```

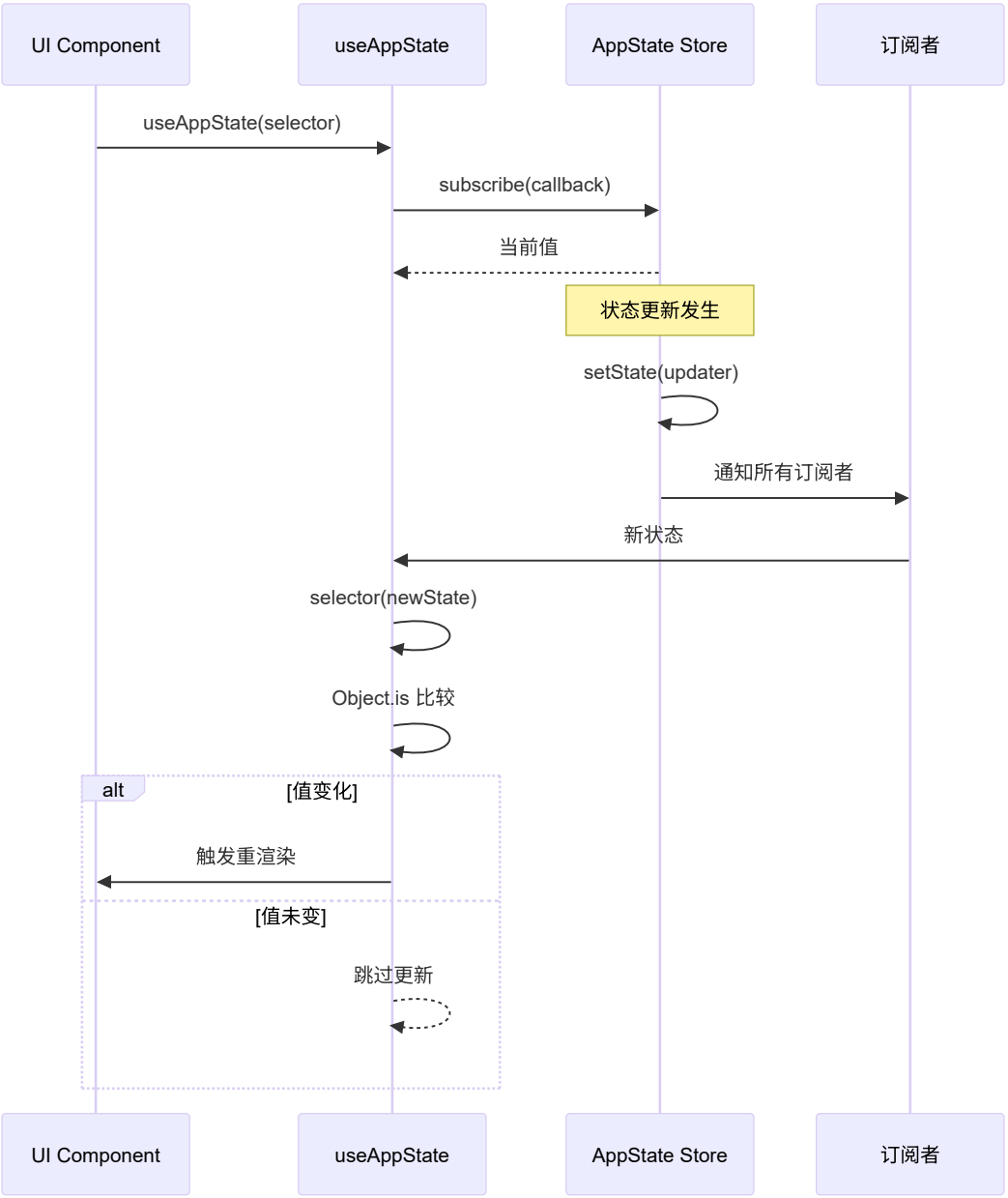
```
  fileStateCache: FileStateCache
  usage: Usage
}

export async function saveSessionSnapshot(
  state: AppState
): Promise<void> {
  const snapshot: SessionSnapshot = {
    version: SESSION_VERSION,
    timestamp: Date.now(),
    sessionId: getSessionId(),
    messages: state.messages,
    tasks: Object.values(state.tasks),
    fileStateCache: state.readFileState,
    usage: getTotalUsage(),
  }

  await writeFile(
    getSessionFilePath(),
    JSON.stringify(snapshot)
  )
}

export async function loadSessionSnapshot(
  sessionId: string
): Promise<SessionSnapshot | null> {
  try {
    const data = await readFile(getSessionFilePath(sessionId))
    const snapshot: SessionSnapshot = JSON.parse(data)

    // 版本兼容性检查
    if (snapshot.version !== SESSION_VERSION) {
      return migrateSnapshot(snapshot)
    }

    return snapshot
  } catch {
    return null
  }
}
```

5.5.2 自动保存策略

```
// 防抖保存（避免频繁写入）
const debouncedSave = debounce(saveSessionSnapshot, 5000)

// 关键事件触发保存
onChangeAppState((newState, oldState) => {
  // 消息变化时保存
  if (newState.messages.length !== oldState.messages.length) {
    debouncedSave(newState)
  }

  // 任务状态变化时立即保存
  if (hasTaskStatusChanged(newState.tasks, oldState.tasks)) {
    saveSessionSnapshot(newState)
  }
})
```

5.6 响应式更新

5.6.1 更新流程



5.6.2 批量更新

```
// 批量更新多个字段
setAppState(prev => ({
  ...prev,
  messages: [...prev.messages, newMessage],
  tasks: {
    ...prev.tasks,
    [taskId]: updatedTask
  }
}))

// 或使用 immer 风格（实际使用展开操作符）
setAppState(prev => produce(prev, draft => {
  draft.messages.push(newMessage)
```

```
  draft.tasks[taskId].status = 'completed'
}))
```

5.7 本章小结

本章深入探讨了 Claude Code 的状态管理：

1. **挑战**：复杂状态、高频更新、异步操作、多智能体
2. **单一数据源**：AppState 包含所有状态
3. **不可变性**：DeepImmutable 类型确保状态安全
4. **极简 Store**：自定义 Store，无 Redux 开销
5. **分层架构**：全局/会话/临时三层状态
6. **持久化**：自动保存和会话恢复
7. **响应式**：useSyncExternalStore 实现高效订阅

状态管理是 Claude Code 的"隐形骨架"，支撑着复杂功能的实现。在下一章中，我们将探讨多智能体架构。

← 上一章：权限系统 | 下一章：多智能体 →

chapter-06-multi-agent

第6章 多智能体架构

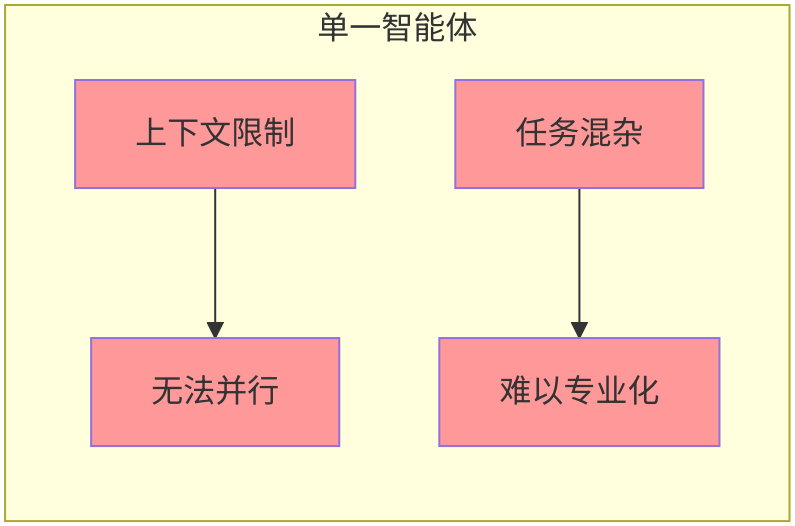
> "单个大脑有极限，多个大脑有无限可能。"
> —— 《Claude Code 设计哲学》

多智能体架构是 Claude Code 最强大的特性之一。它允许一个 AI 实例（Agent）生成多个子智能体（Subagent），并行处理复杂任务。本章深入探讨多智能体的设计哲学、实现机制和实践应用。

6.1 为什么需要多智能体?

6.1.1 单一智能体的局限性

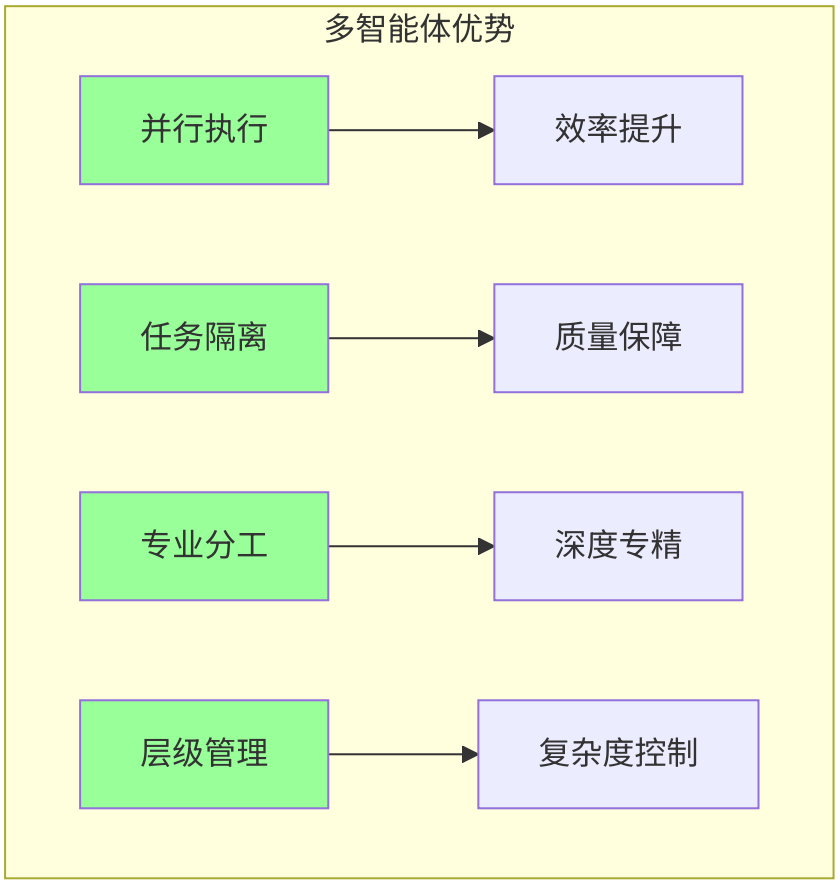
传统 AI 对话系统只有一个智能体处理所有任务，这在面对复杂场景时存在明显局限：



具体局限：

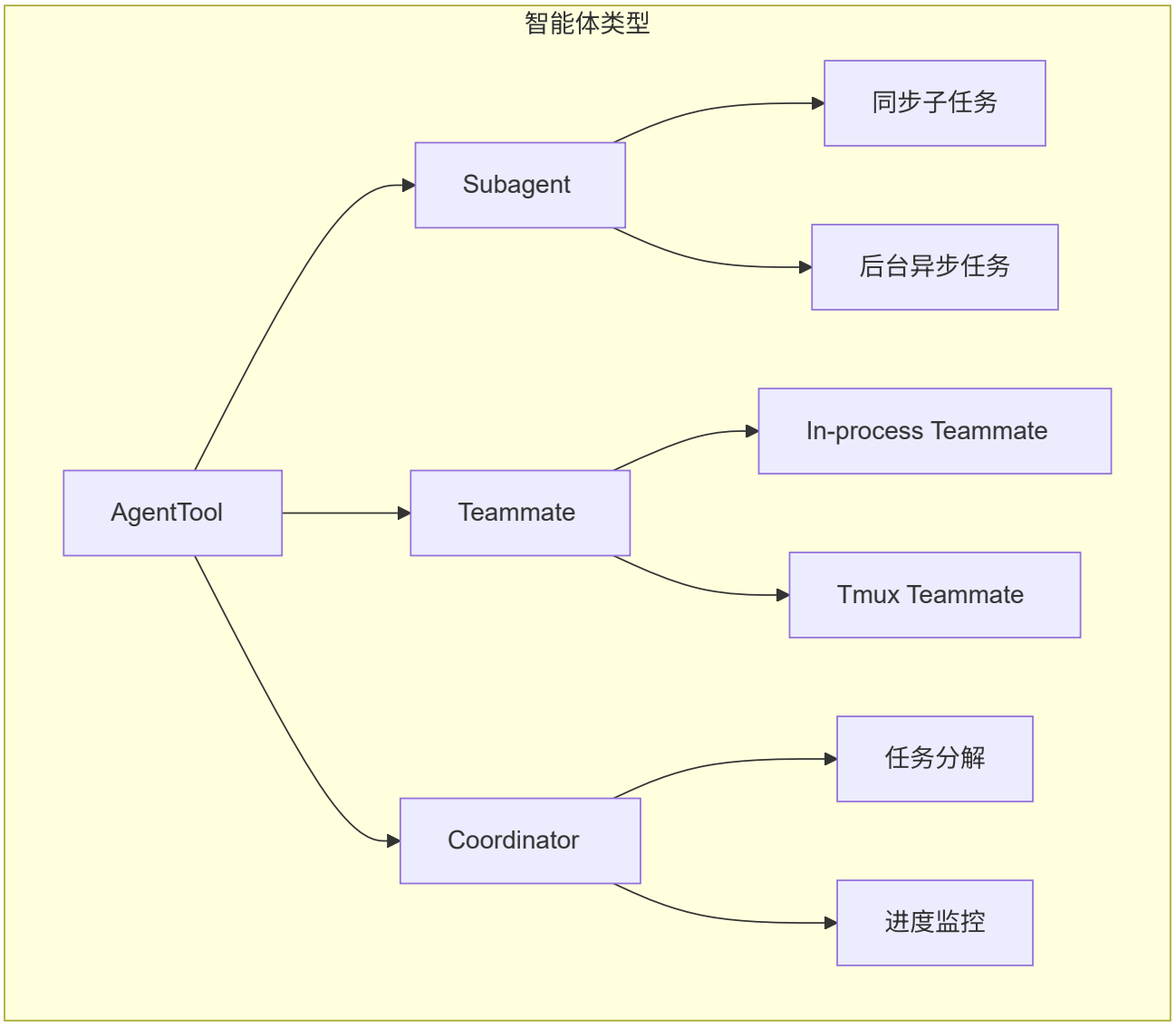
局限	说明	影响
上下文限制	单一会话有 Token 上限	大项目无法全部装入
串行处理	任务只能一个接一个执行	效率低下
上下文污染	不同任务相互干扰	输出质量下降
难以专业化	一个智能体身兼多职	无法深度专精

6.1.2 多智能体的优势



6.2 智能体类型

Claude Code 支持多种智能体类型，每种有不同的适用场景：



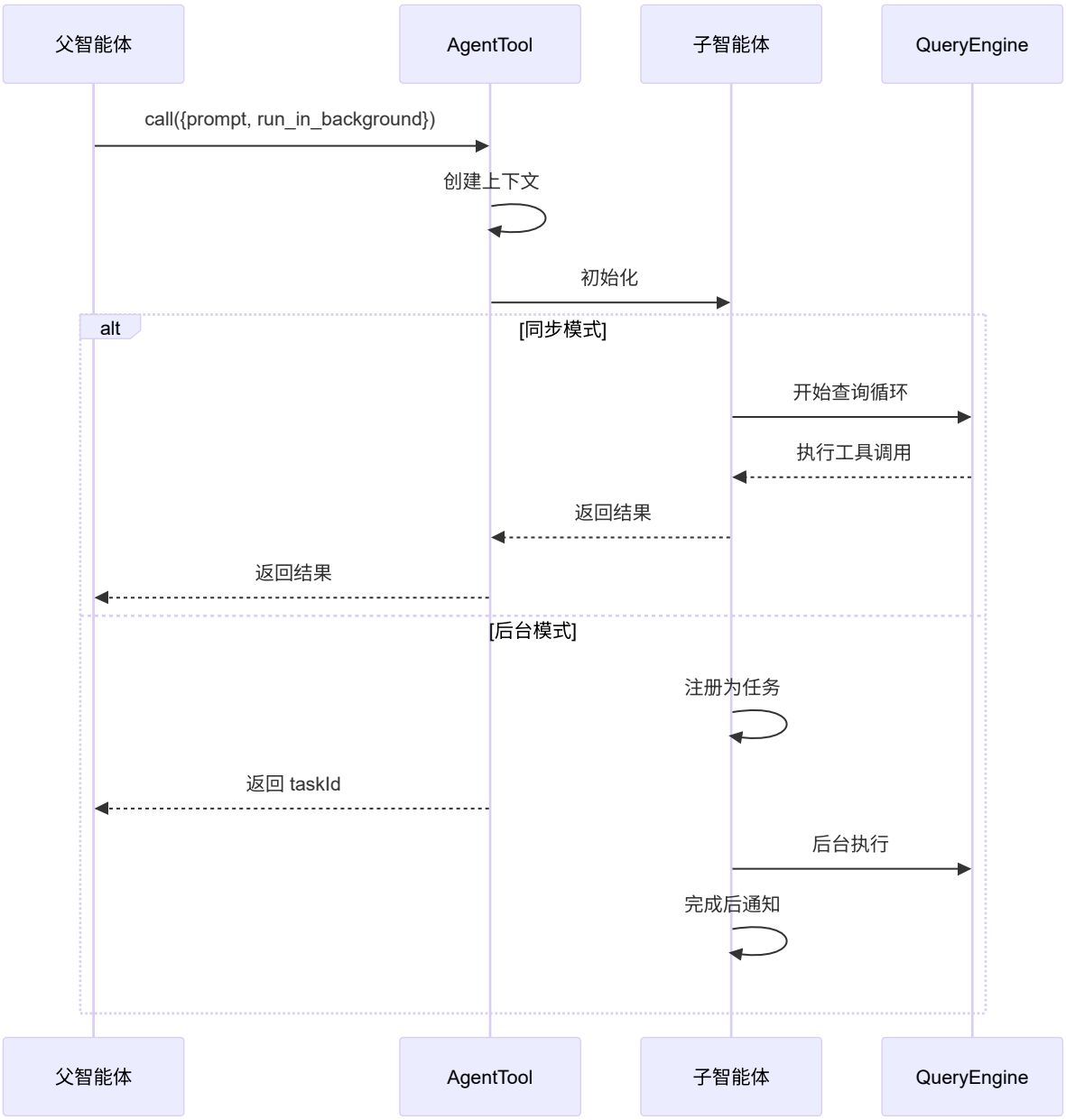
6.2.1 子智能体（Subagent）

子智能体是最基本的并行单元，通过 `AgentTool` 创建：

```
// 同步子智能体
const result = await AgentTool.call({
  description: "分析测试失败",
  prompt: "请分析 tests/login.test.ts 失败的原因...",
  subagent_type: "code-reviewer",
  model: "haiku"
});

// 后台异步子智能体
```

```
const task = await AgentTool.call({
  description: "批量重命名",
  prompt: "将所有 Button 组件重命名为 UIButton...",
  run_in_background: true
});
```

子智能体的生命周期：



6.2.2 队友（Teammate）

队友是更高级的协作实体，支持双向通信：

```
// 创建队友
await AgentTool.call({
  description: "创建数据库专家",
  prompt: "你是 PostgreSQL 专家...",
```

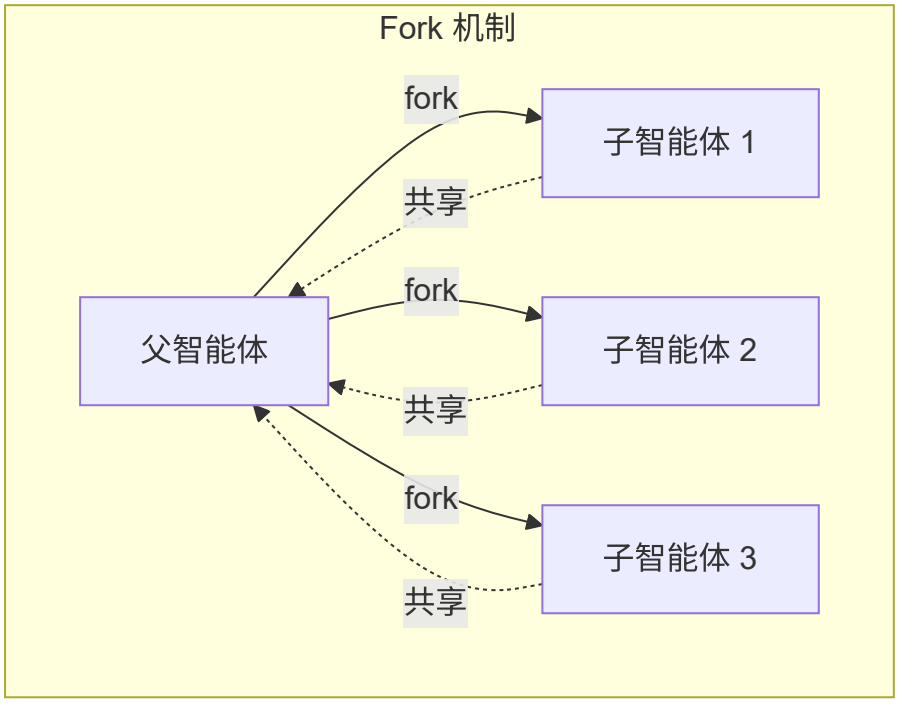
```
  name: "db-expert",
  team_name: "my-team",
  run_in_background: true
});

// 发送消息
await SendMessageTool.call({
  to: "db-expert",
  content: "优化这个查询..."
});
```

队友的实现方式：

类型	实现	通信方式	适用场景
In-process	同进程不同线程	内存共享	快速协作
Tmux	独立 tmux 窗口	IPC	可视化并行
Remote	远程 CCR 环境	WebSocket	计算密集型

6.2.3 Fork 子智能体

Fork 是 Claude Code 的特殊子智能体，它继承父智能体的完整上下文：



Fork 的核心特性：

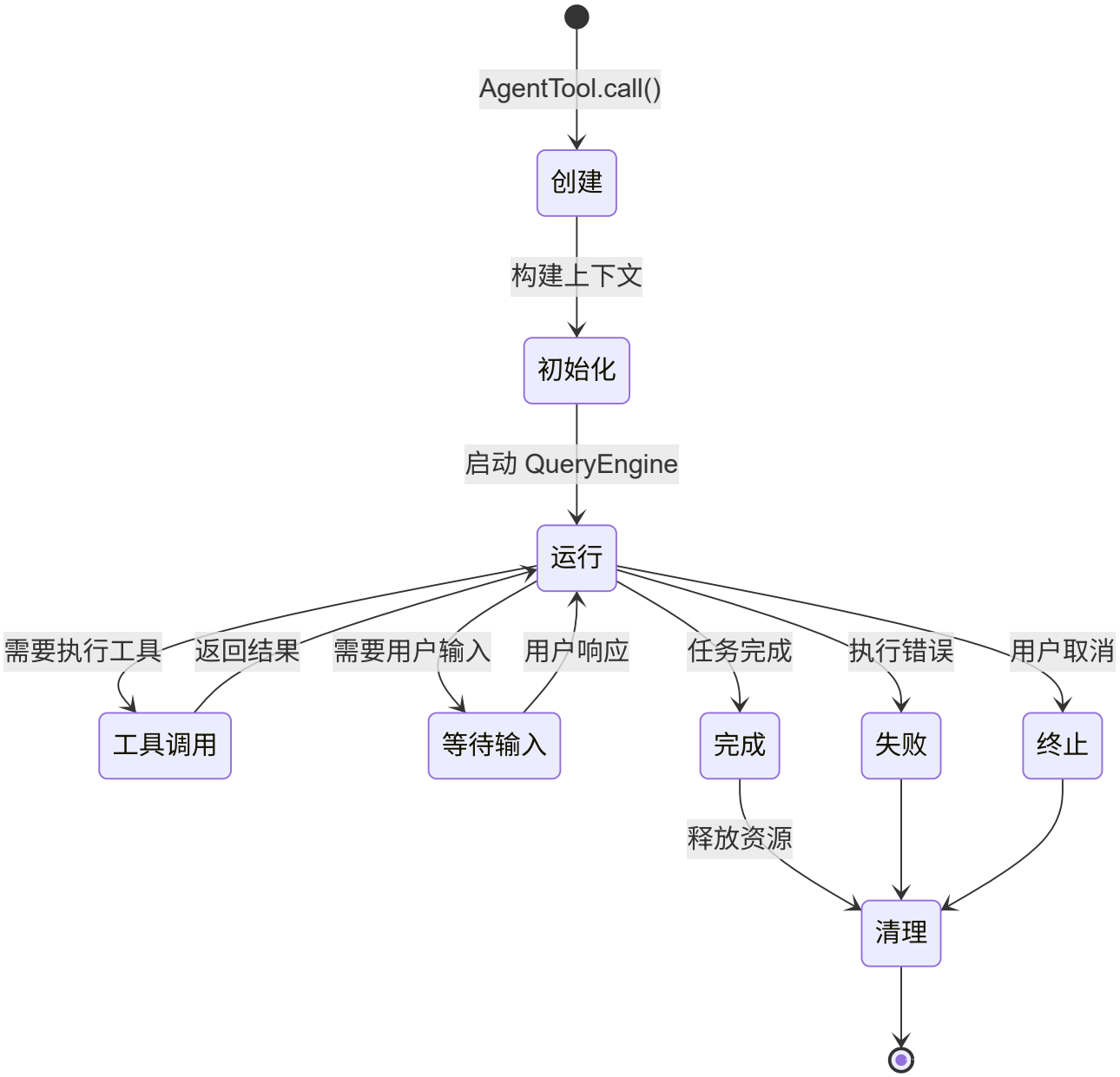
1. **上下文继承**：子智能体看到父智能体的完整历史
2. **并行探索**：多个子智能体可以同时探索不同方案

3. **结果汇总**：父智能体收集并比较各子智能体的结果

```
// Fork 子智能体示例
await AgentTool.call({
  description: "探索方案 A",
  prompt: "基于当前上下文，探索实现方案 A..."
  // subagent_type 省略，使用 Fork 路径
});

await AgentTool.call({
  description: "探索方案 B",
  prompt: "基于当前上下文，探索实现方案 B..."
});
```

6.3 智能体生命周期

6.3.1 完整生命周期图



6.3.2 状态管理

每个智能体都有其状态，存储在全局 AppState 中：

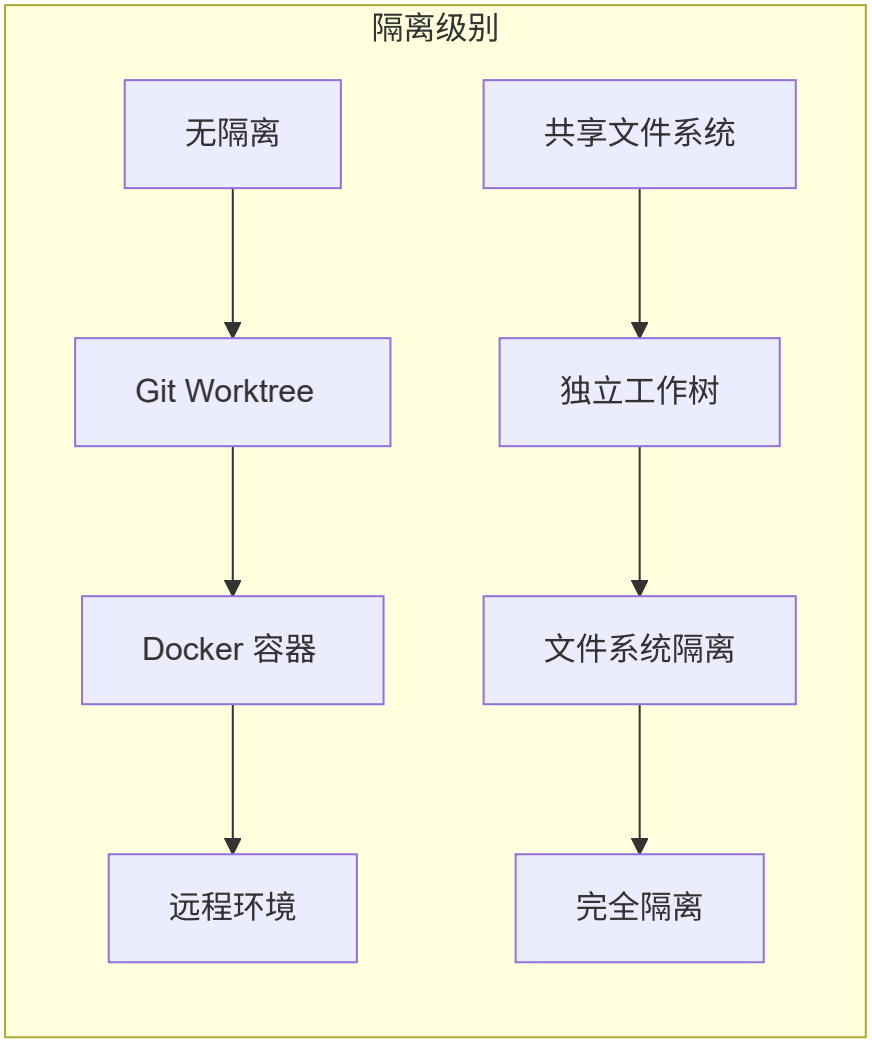
```
// src/tasks/LocalAgentTask/LocalAgentTask.tsx

interface LocalAgentTaskState {
  type: 'local_agent';
  agentId: string;
  prompt: string;
  agentType: string;
  model?: string;

  // 执行状态
  status: 'pending' | 'running' | 'completed' | 'failed' | 'killed';
  progress?: AgentProgress;
```

```
  messages?: Message[];

  // 控制
  abortController?: AbortController;

  // 后台任务专用
  isBackgrounded: boolean;
  pendingMessages: string[];
}
```

6.4 隔离机制

6.4.1 隔离级别

Claude Code 提供多种隔离级别，满足不同安全需求：



隔离级别	文件系统	进程	网络	适用场景
无隔离	共享	共享	共享	快速子任务
Worktree	部分隔离	共享	共享	并行开发
Docker	隔离	隔离	可控	敏感操作

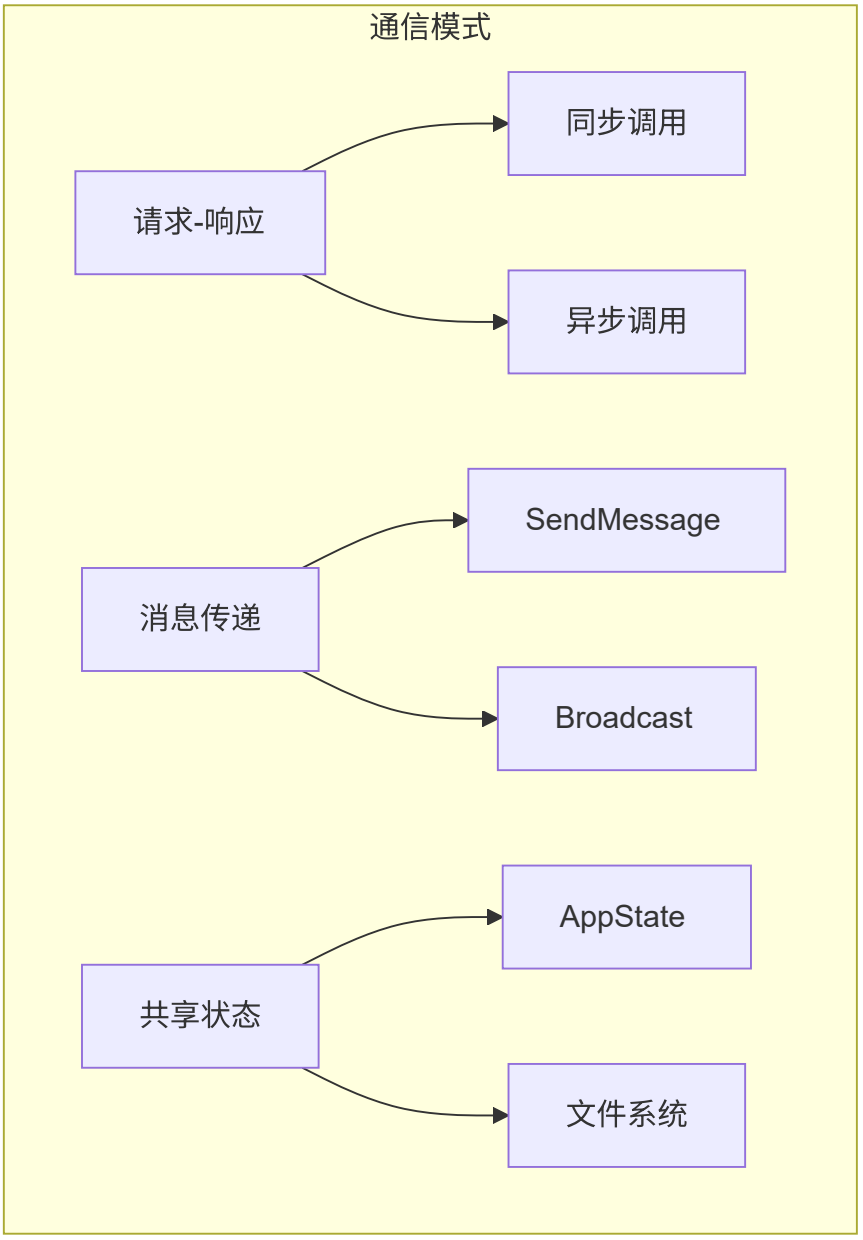
隔离级别	文件系统	进程	网络	适用场景
Remote	完全隔离	完全隔离	隔离	不可信代码

6.4.2 Worktree 隔离详解

```
// src/utils/worktree.ts

export async function createAgentWorktree(
  agentId: string,
  baseBranch: string
): Promise<WorktreeInfo> {
  // 1. 创建临时分支
  const worktreeBranch = `agent-${agentId}`;
  await exec(`git checkout -b ${worktreeBranch}`);

  // 2. 创建 worktree
  const worktreePath = `/tmp/claude-worktrees/${agentId}`;
  await exec(`git worktree add ${worktreePath} ${worktreeBranch}`);

  // 3. 设置隔离环境
  return {
    path: worktreePath,
    branch: worktreeBranch,
    baseBranch,
  };
}

export async function removeAgentWorktree(worktree: WorktreeInfo):
Promise<void> {
  // 清理 worktree
  await exec(`git worktree remove ${worktree.path}`);
  await exec(`git branch -D ${worktree.branch}`);
}
```

6.5 智能体间通信

6.5.1 通信模式



6.5.2 SendMessage 实现

```
// src/tools/SendMessageTool/SendMessageTool.ts

export const SendMessageTool = buildTool({
  name: 'SendMessage',

  inputSchema: z.object({
    to: z.string().describe('目标智能体名称'),
    content: z.string().describe('消息内容'),
  }),

  async call(input, context) {
    const { to, content } = input;

    // 1. 查找目标智能体
```

```
    const appState = context.getAppState();
    const targetAgentId = appState.agentNameRegistry.get(to);

    if (!targetAgentId) {
      return { error: `Agent '${to}' not found` };
    }

    // 2. 将消息加入目标智能体的待处理队列
    queuePendingMessage(targetAgentId, content, context.setAppState);

    // 3. 如果目标正在运行，立即通知
    notifyAgent(targetAgentId);

    return { status: 'sent' };
  },
});
```

6.6 AgentTool 核心实现

6.6.1 智能体选择逻辑

```
// src/tools/AgentTool/AgentTool.tsx (简化示意)

export const AgentTool = buildTool({
  async call(input, context) {
    const {
      prompt,
      subagent_type,
      run_in_background,
      name,
      team_name,
      isolation,
    } = input;

    // 1. 判断是否为 Teammate 创建
    if (team_name && name) {
      return spawnTeammate({
        name,
        prompt,
        team_name,
        // ...
      });
    }
```

```
    // 2. 判断是否为 Fork 路径
    const isForkPath = !subagent_type && isForkSubagentEnabled();

    // 3. 选择智能体定义
    const selectedAgent = isForkPath
      ? FORK_AGENT
      : context.options.agentDefinitions.activeAgents.find(
          a => a.agentType === subagent_type
        );

    // 4. 检查 MCP 依赖
    if (selectedAgent.requiredMcpServers) {
      const hasRequiredServers = checkMcpServers(
        selectedAgent.requiredMcpServers,
        context.getAppState().mcp.clients
      );
      if (!hasRequiredServers) {
        throw new Error(`Missing required MCP servers`);
      }
    }

    // 5. 根据隔离模式创建执行环境
    switch (isolation) {
      case 'remote':
        return launchRemoteAgent({ prompt, selectedAgent });
      case 'worktree':
        return launchWorktreeAgent({ prompt, selectedAgent });
      default:
        return launchLocalAgent({
          prompt,
          selectedAgent,
          run_in_background,
        });
    }
  },
});
```

6.6.2 进度追踪

```
// src/tasks/LocalAgentTask/LocalAgentTask.tsx

export interface AgentProgress {
```

```
  toolUseCount: number;
  tokenCount: number;
  lastActivity?: ToolActivity;
  recentActivities?: ToolActivity[];
  summary?: string;
}

export function updateProgressFromMessage(
  tracker: ProgressTracker,
  message: Message,
  resolveActivityDescription?: ActivityDescriptionResolver
): void {
  if (message.type !== 'assistant') return;

  // 更新 Token 计数
  tracker.latestInputTokens =
    message.message.usage.input_tokens +
    (message.message.usage.cache_creation_input_tokens ?? 0) +
    (message.message.usage.cache_read_input_tokens ?? 0);
  tracker.cumulativeOutputTokens += message.message.usage.output_tokens;

  // 记录工具调用
  for (const content of message.message.content) {
    if (content.type === 'tool_use') {
      tracker.toolUseCount++;
      tracker.recentActivities.push({
        toolName: content.name,
        input: content.input as Record<string, unknown>,
        activityDescription: resolveActivityDescription?.(
          content.name,
          content.input
        ),
      });
    }
  }

  // 保持最近 5 个活动
  while (tracker.recentActivities.length > MAX_RECENT_ACTIVITIES) {
    tracker.recentActivities.shift();
  }
}
```

6.7 实践案例

6.7.1 并行代码审查

```
// 使用多个子智能体并行审查不同方面

async function parallelCodeReview(prFiles: string[]) {
  // 创建多个审查智能体
  const reviewers = await Promise.all([
    AgentTool.call({
      description: "安全性审查",
      prompt: "审查以下代码的安全问题：检查 SQL 注入、XSS、敏感数据泄露...",
      subagent_type: "security-reviewer",
    }),
    AgentTool.call({
      description: "性能审查",
      prompt: "审查以下代码的性能问题：检查算法复杂度、N+1 查询、内存泄漏...",
      subagent_type: "performance-reviewer",
    }),
    AgentTool.call({
      description: "架构审查",
      prompt: "审查以下代码的架构问题：检查模块化、依赖关系、设计模式...",
      subagent_type: "architecture-reviewer",
    }),
  ]);

  // 汇总结果
  return consolidateReviews(reviewers);
}
```

6.7.2 探索式重构

```
// 使用 Fork 子智能体探索多种重构方案

async function exploratoryRefactoring(targetFile: string) {
  // 创建多个 Fork 探索不同方案
  const explorations = await Promise.all([
    AgentTool.call({
      description: "方案 A：提取类",
      prompt: `将 ${targetFile} 中的逻辑提取为独立类...`,
      // 使用 Fork 路径
    }),
    AgentTool.call({
      description: "方案 B：函数组合",
```

```
      prompt: `将 ${targetFile} 重构为纯函数组合...`,
    }),
    AgentTool.call({
      description: "方案 C: 策略模式",
      prompt: `将 ${targetFile} 重构为策略模式...`,
    }),
  ]);

  // 比较各方案的优缺点
  return compareApproaches(explorations);
}
```

6.8 本章小结

本章深入探讨了 Claude Code 的多智能体架构：

1. **为什么需要多智能体**：突破单一智能体的上下文、并行、专业化限制
2. **三种智能体类型**：Subagent（子智能体）、Teammate（队友）、Coordinator（协调者）
3. **生命周期管理**：从创建到清理的完整状态机
4. **隔离机制**：从 Worktree 到 Remote 的多级隔离
5. **通信模式**：请求-响应、消息传递、共享状态
6. **实践案例**：并行审查、探索式重构

多智能体架构让 Claude Code 能够处理复杂的多步骤任务，是其实现"AI 成为真正搭档"愿景的关键。

在下一章中，我们将探讨 Claude Code 的开放生态——MCP 集成。

延伸阅读：

- Multi-Agent Reinforcement Learning
- Actor Model
- Git Worktree Documentation

← **上一章：状态管理** | **下一章：MCP 集成 →**

chapter-07-mcp

第7章 MCP 集成与开放生态

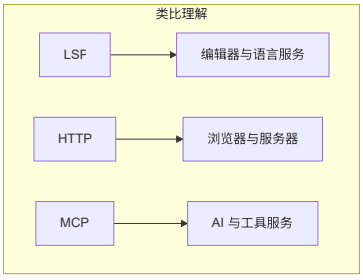
> "开放的生态让能力无限延伸。"
> —— MCP 设计哲学

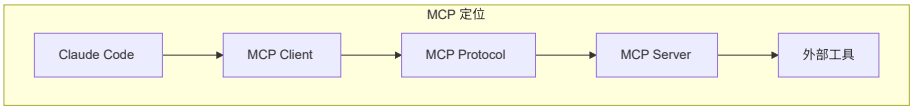
Model Context Protocol (MCP) 是 Claude Code 的开放生态基石。它定义了标准协议，让外部工具能够无缝集成到 Claude Code 中。本章将深入探讨 MCP 的设计理念、架构实现和集成实践。

7.1 MCP 概述

7.1.1 什么是 MCP?

MCP 是 Anthropic 提出的开放协议，用于标准化 AI 与外部工具的交互方式。它类似于 LSP (Language Server Protocol)，但是面向 AI 工具而非代码编辑器。

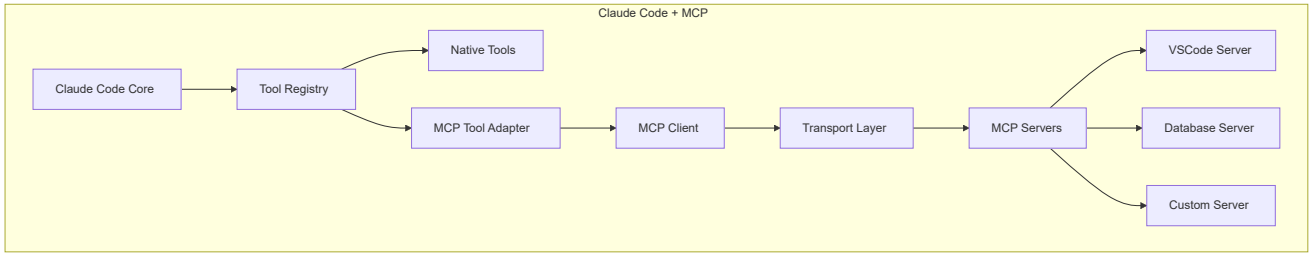




MCP 的核心价值：

价值	说明
标准化	统一的工具接入协议
开放性	第三方可自由扩展
隔离性	外部工具在沙箱运行
可发现	自动发现可用工具

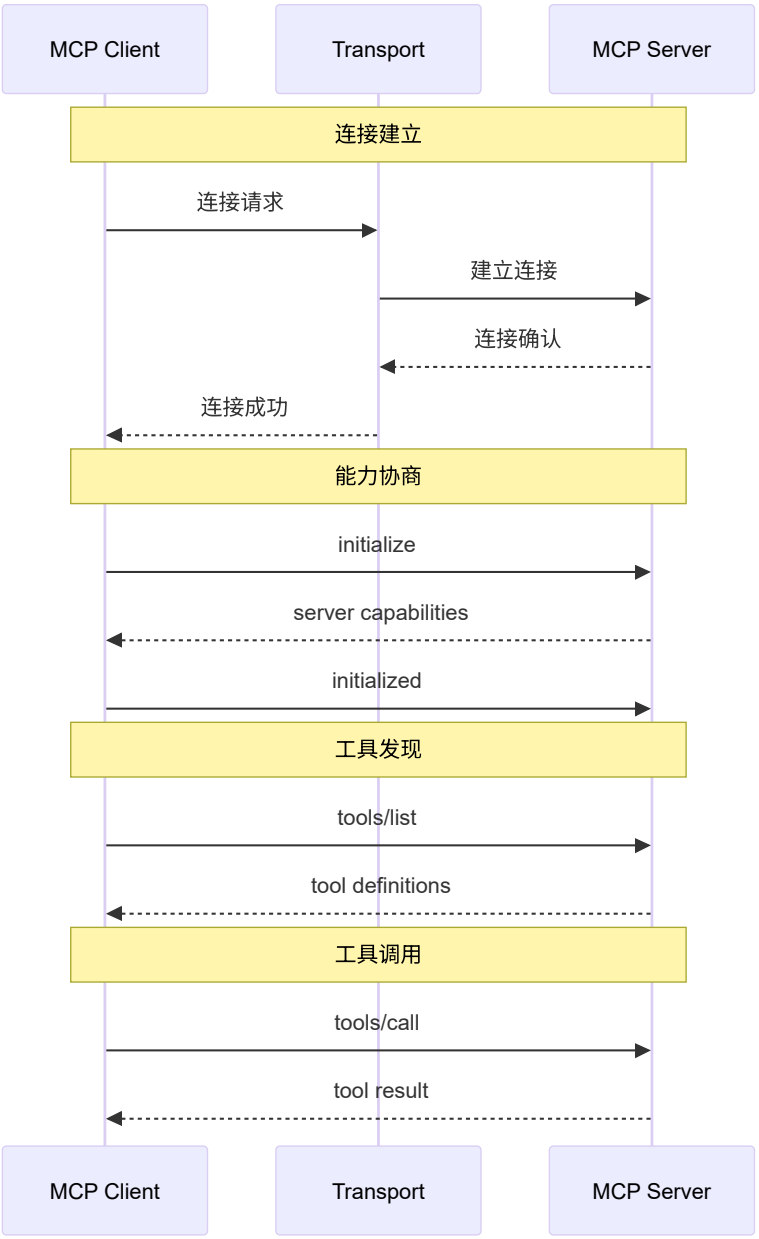
7.1.2 MCP 与 Claude Code 的关系



7.2 MCP 协议详解

7.2.1 协议架构

MCP 基于 JSON-RPC 2.0 构建，支持多种传输方式：



7.2.2 传输层支持

Claude Code 支持多种 MCP 传输方式：

```
// src/services/mcp/client.ts

import { StdioClientTransport } from
'@modelcontextprotocol/sdk/client/stdio.js'
import { SSEClientTransport } from '@modelcontextprotocol/sdk/client/sse.js'
import { StreamableHTTPClientTransport } from
'@modelcontextprotocol/sdk/client/streamableHttp.js'
import { WebSocketTransport } from '../../utils/mcpWebSocketTransport.js'

type TransportType =
  | 'stdio'        // 本地子进程
  | 'sse'          // Server-Sent Events
```

```
  | 'http'        // HTTP Streaming
  | 'websocket'   // WebSocket

async function createTransport(
  type: TransportType,
  config: TransportConfig
): Promise<Transport> {
  switch (type) {
    case 'stdio':
      return new StdioClientTransport({
        command: config.command,
        args: config.args,
        env: { ...process.env, ...config.env },
      })

    case 'sse':
      return new SSEClientTransport(config.url, {
        eventSourceInit: { headers: config.headers },
      })

    case 'http':
      return new StreamableHTTPClientTransport(config.url, {
        headers: config.headers,
      })

    case 'websocket':
      return new WebSocketTransport(config.url, {
        headers: config.headers,
      })

    default:
      throw new Error(`Unknown transport type: ${type}`)
  }
}
```

7.2.3 消息格式

MCP 消息遵循 JSON-RPC 2.0：

```
// Request
interface MCPRequest {
  jsonrpc: '2.0'
  id: string | number
```

```
  method: string
  params?: unknown
}

// Response
interface MCPResponse {
  jsonrpc: '2.0'
  id: string | number
  result?: unknown
  error?: {
    code: number
    message: string
    data?: unknown
  }
}

// Notification（无需响应）
interface MCPNotification {
  jsonrpc: '2.0'
  method: string
  params?: unknown
}
```

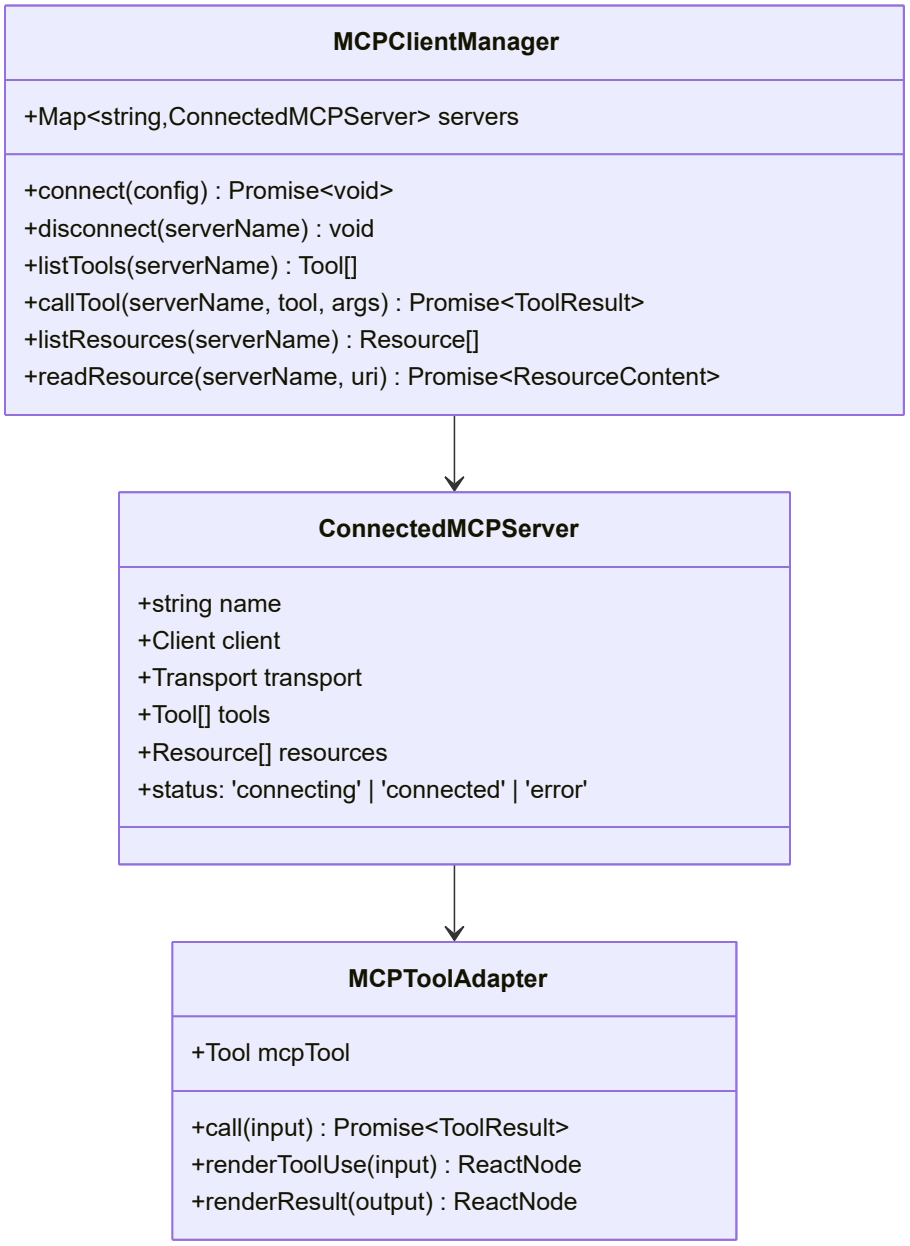
工具调用示例：

```
// Request
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "get_diagnostics",
    "arguments": {
      "file": "src/main.ts"
    }
  }
}

// Response
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
```

```
    "content": [
      {
        "type": "text",
        "text": "Found 3 errors..."
      }
    ],
    "isError": false
  }
}
```

7.3 MCP Client 实现

7.3.1 Client 架构



7.3.2 连接管理

```
// src/services/mcp/client.ts

export class MCPClientManager {
  private servers = new Map<string, ConnectedMCPServer>()

  async connect(config: McpServerConfig): Promise<void> {
    const { name, transport: transportType, ...transportConfig } = config

    try {
      // 1. 创建传输层
      const transport = await createTransport(transportType,
transportConfig)

      // 2. 创建 MCP Client
      const client = new Client({ name: 'claude-code', version: VERSION })

      // 3. 建立连接
      await client.connect(transport)

      // 4. 协商能力
      const initResult = await client.initialize({
        protocolVersion: '2024-11-05',
        capabilities: {
          tools: {},
          resources: {},
          prompts: {},
        },
        clientInfo: { name: 'claude-code', version: VERSION },
      })

      // 5. 获取工具列表
      const toolsResult = await client.listTools()

      // 6. 注册服务器
      const server: ConnectedMCPServer = {
        name,
        client,
        transport,
        tools: toolsResult.tools,
        resources: [],
        status: 'connected',
      }
```

```
      this.servers.set(name, server)

      // 7. 更新 AppState
      setAppState(prev => ({
        ...prev,
        mcp: {
          ...prev.mcp,
          clients: [...prev.mcp.clients, { name, type: 'connected' }],
          tools: [...prev.mcp.tools, ...toolsResult.tools.map(t =>
            createMcpToolAdapter(server, t)
          )],
        },
      }))

    } catch (error) {
      logMCPError(`Failed to connect to MCP server ${name}:`, error)
      throw error
    }
  }

  async callTool(
    serverName: string,
    toolName: string,
    args: Record<string, unknown>
  ): Promise<ToolResult> {
    const server = this.servers.get(serverName)
    if (!server) {
      throw new Error(`MCP server ${serverName} not found`)
    }

    const result = await server.client.callTool({
      name: toolName,
      arguments: args,
    })

    return normalizeMcpResult(result)
  }
}
```

7.3.3 工具适配器

MCP 工具需要适配为 Claude Code 的 Tool 接口：

```
// src/tools/MCPTool/MCPTool.ts

export function createMcpToolAdapter(
  server: ConnectedMCPServer,
  mcpTool: MCPToolDefinition
): Tool {
  return buildTool({
    // MCP 工具名格式: mcp__{serverName}__{toolName}
    name: buildMcpToolName(server.name, mcpTool.name),

    // 从 MCP 定义转换
    description: mcpTool.description,
    inputSchema: mcpTool.inputSchema,

    // 调用转发到 MCP Server
    async call(input, context) {
      const result = await mcpClientManager.callTool(
        server.name,
        mcpTool.name,
        input
      )

      // 处理 MCP 结果
      if (result.isError) {
        return {
          error: result.content.map(c => c.text).join(''),
        }
      }

      // 转换内容为 Claude Code 格式
      return {
        data: result.content.map(c => ({
          type: c.type,
          content: c.text || c.data,
        })),
      }
    },

    // 自定义 UI 渲染
    renderToolUseMessage(input) {
      return <McpToolUseView server={server.name} tool={mcpTool.name} />
    },
```

```
    renderToolResultMessage(output) {
      return <McpResultView content={output.data} />
    },
  })
}
```

7.4 配置与发现

7.4.1 配置文件

MCP 服务器通过配置文件声明：

```
// .claude/mcp.json
{
  "mcpServers": {
    "vscode": {
      "command": "node",
      "args": ["/path/to/vscode-mcp-server/dist/index.js"],
      "transport": "stdio",
      "env": {
        "VSCODE_PID": "${env:VSCODE_PID}"
      }
    },
    "postgres": {
      "url": "http://localhost:3000/mcp",
      "transport": "http",
      "headers": {
        "Authorization": "Bearer ${env:PG_TOKEN}"
      }
    },
    "github": {
      "url": "https://api.github.com/mcp",
      "transport": "sse",
      "oauth": {
        "provider": "github",
        "scopes": ["repo", "read:user"]
      }
    }
  }
}
```

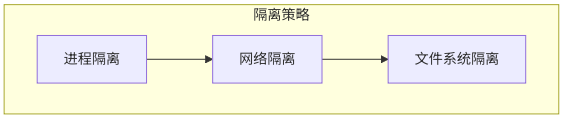
7.4.2 自动发现

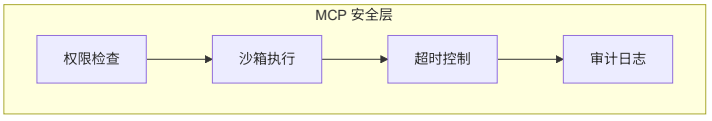
Claude Code 支持自动发现 MCP 服务器：

```
// src/services/mcp/discovery.ts

export async function discoverMcpServers(): Promise<McpServerConfig[]> {
  const servers: McpServerConfig[] = []

  // 1. 从配置文件
  const configPath = path.join(getCwd(), '.claude', 'mcp.json')
  if (await fileExists(configPath)) {
    const config = await readJson(configPath)
    servers.push(...Object.entries(config.mcpServers).map(([name, cfg]) =>
      ({ name, ...cfg })
    ))
  }

  // 2. 从环境变量
  if (process.env.MCP_SERVERS) {
    const envServers = JSON.parse(process.env.MCP_SERVERS)
    servers.push(...envServers)
  }

  // 3. 从插件声明
  const plugins = await loadPlugins()
  for (const plugin of plugins) {
    if (plugin.mcpServers) {
      servers.push(...plugin.mcpServers)
    }
  }

  return servers
}
```

7.5 安全与隔离

7.5.1 安全架构





7.5.2 安全实现

```
// src/services/mcp/security.ts

interface MCPSecurityPolicy {
  // 允许调用的工具白名单
  allowedTools?: string[]

  // 禁止的工具黑名单
  deniedTools?: string[]

  // 超时限制
  timeoutMs: number

  // 是否允许网络访问
  allowNetwork: boolean

  // 允许的文件系统访问路径
  allowedPaths?: string[]
}

async function executeWithSandbox<T>(
  server: ConnectedMCPServer,
  operation: () => Promise<T>,
  policy: MCPSecurityPolicy
): Promise<T> {
  // 1. 设置超时
  const timeoutPromise = sleep(policy.timeoutMs).then(() => {
    throw new Error(`MCP operation timed out after ${policy.timeoutMs}ms`)
  })

  // 2. 执行操作
  const resultPromise = operation()

  // 3. 竞争超时
  return Promise.race([resultPromise, timeoutPromise])
}

// 权限检查
function checkMcpToolPermission(
  toolName: string,
  context: ToolPermissionContext
): PermissionResult {
```

```
  // 检查 MCP 工具权限规则
  const mcpRule = context.mcpToolRules.find(r =>
    r.pattern.test(toolName)
  )

  if (mcpRule?.behavior === 'deny') {
    return { behavior: 'deny', reason: 'MCP tool denied by policy' }
  }

  return { behavior: 'allow' }
}
```

7.6 实践案例

7.6.1 VSCode 集成

```
// 配置 VSCode MCP Server
{
  "mcpServers": {
    "vscode": {
      "command": "code",
      "args": ["--mcp-server"],
      "transport": "stdio"
    }
  }
}

// 使用示例
const result = await MCPTool.call({
  server: "vscode",
  tool: "get_diagnostics",
  args: { file: "src/main.ts" }
})
```

7.6.2 数据库集成

```
// PostgreSQL MCP Server
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
```

```
      "env": {
        "DATABASE_URL": "postgresql://..."
      }
    }
  }
}

// 使用示例
const schema = await MCPTool.call({
  server: "postgres",
  tool: "get_schema",
  args: { table: "users" }
})
```

7.7 本章小结

本章深入探讨了 Claude Code 的 MCP 集成：

1. **MCP 概述**：标准化、开放、隔离的协议
2. **协议架构**：JSON-RPC 2.0 基础，多传输支持
3. **Client 实现**：连接管理、工具适配、错误处理
4. **配置与发现**：配置文件、自动发现、环境变量
5. **安全隔离**：权限控制、沙箱执行、超时限制
6. **实践案例**：VSCode、数据库等集成

MCP 让 Claude Code 的能力无限延伸，构建真正的开放生态。

在下一章中，我们将探讨 Claude Code 的性能优化策略。

← 上一章：多智能体 | 下一章：性能优化 →

chapter-08-performance

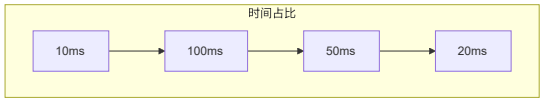
第8章 性能优化策略

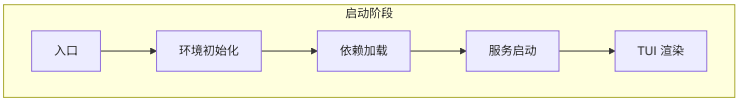
> "性能不是特性，而是用户体验的基础。"
> —— 《Claude Code 设计哲学》

Claude Code 需要处理复杂的 AI 对话、工具执行和状态管理。本章深入探讨其性能优化策略，涵盖启动优化、运行时优化和内存管理。

8.1 启动优化

8.1.1 启动流程分析





Claude Code 启动时间分解：

阶段	耗时	主要操作
环境初始化	~10ms	特性标志、配置加载
依赖加载	~100ms	模块导入、代码编译
服务启动	~50ms	MCP 连接、LSP 启动
TUI 渲染	~20ms	React 渲染、首次绘制

8.1.2 并行加载

```
// src/main.tsx

async function initializeServices() {
  // ❌ 串行加载（慢）
  // await loadPlugins()
  // await initializeMcp()
  // await loadSkills()

  // ✅ 并行加载（快）
  const [plugins, mcp, skills] = await Promise.all([
    loadPlugins(),
    initializeMcp(),
    loadSkills(),
  ])

  return { plugins, mcp, skills }
}
```

8.1.3 延迟加载

```
// src/main.tsx
```

```
// 延迟加载避免循环依赖和减少初始 bundle
const getTeammateUtils = () => require('./utils/teammate.js')

// 在需要时调用
function someFunction() {
  const { getTeammateId } = getTeammateUtils()
  // ...
}

// 特性标志控制的延迟加载
const coordinatorModeModule = feature('COORDINATOR_MODE')
  ? require('./coordinator/coordinatorMode.js')
  : null
```

8.1.4 预取策略

```
// src/main.tsx

// 在模块加载阶段就开始异步操作
import { startMdmRawRead } from './utils/settings/mdm/rawRead.js'
startMdmRawRead() // 后台启动 MDM 读取

import { startKeychainPrefetch } from
'./utils/secureStorage/keychainPrefetch.js'
startKeychainPrefetch() // 后台预取 keychain

// 主流程继续，异步操作并行执行
```

8.1.5 编译时优化

使用 Bun 的 `feature()` 实现死代码消除：

```
// src/main.tsx

import { feature } from 'bun:bundle'

// 特性标志控制的代码在编译时确定
const assistantModule = feature('KAIROS')
  ? require('./assistant/index.js')
  : null
```

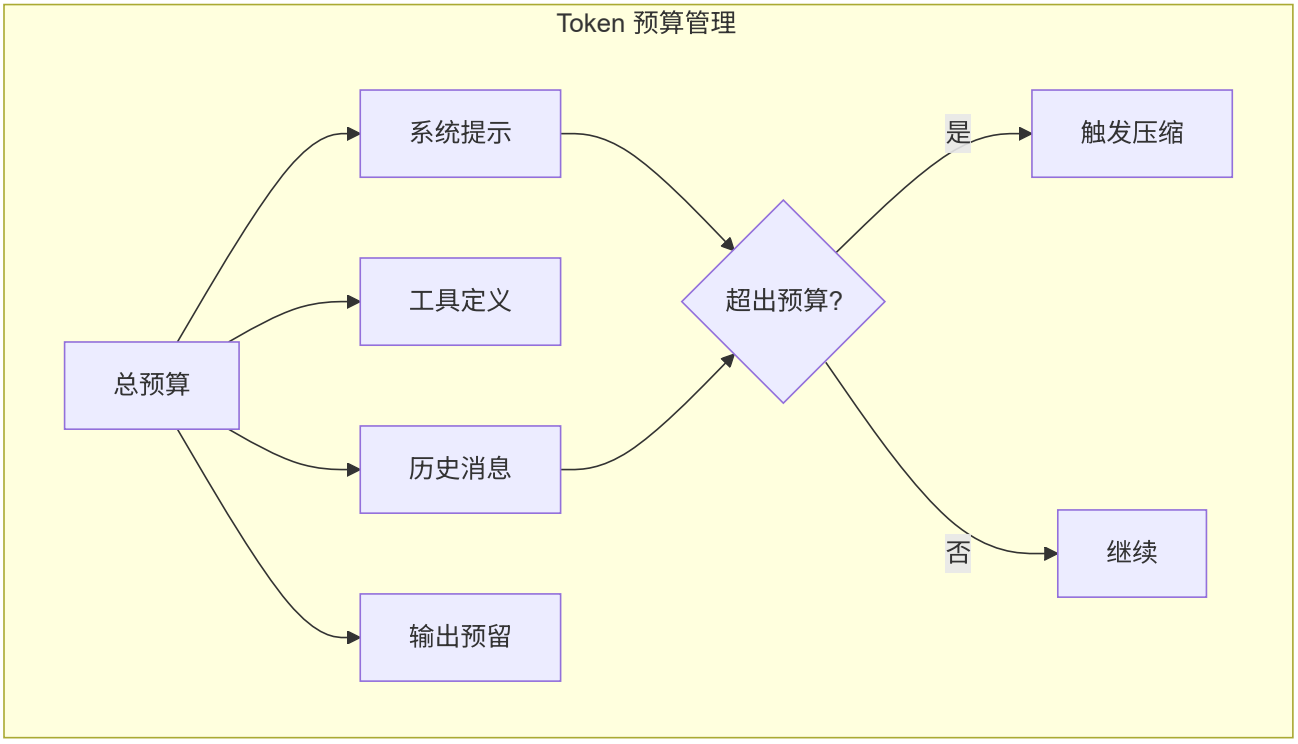
```
// 当 feature('KAIROS') 为 false 时，require 语句被完全移除
// 不会出现在最终 bundle 中
```

编译时优化的好处：

好处	说明
零运行时开销	条件判断在编译期完成
更小的 bundle	未使用的代码被移除
更快的启动	无需加载未使用的模块

8.2 运行时优化

8.2.1 Token 预算管理




```
// src/utils/context.ts

interface TokenBudget {
  maxTokens: number       // 模型最大 Token
  usedTokens: number      // 已使用 Token
  reservedTokens: number // 预留输出 Token
  availableTokens: number // 可用 Token
```

```
}

function calculateTokenBudget(
  messages: Message[],
  tools: Tool[]
): TokenBudget {
  const systemTokens = countTokens(getSystemPrompt())
  const toolTokens = countTokens(tools.map(t => t.description).join('\n'))
  const messageTokens = messages.reduce(
    (sum, m) => sum + countTokens(m.content),
    0
  )

  const maxTokens = getContextWindowForModel(getMainLoopModel())
  const reservedTokens = maxTokens * 0.15  // 预留 15% 给输出

  return {
    maxTokens,
    usedTokens: systemTokens + toolTokens + messageTokens,
    reservedTokens,
    availableTokens: maxTokens - systemTokens - toolTokens - messageTokens -
reservedTokens,
  }
}

// 接近限制时触发压缩
if (budget.availableTokens < COMPACT_THRESHOLD) {
  await compactHistory(messages)
}
```

8.2.2 上下文压缩

```
// src/services/compact/compact.ts

interface CompactStrategy {
  name: string
  canCompact(messages: Message[]): boolean
  compact(messages: Message[]): CompactResult
}

// 策略 1: 摘要压缩
const summaryStrategy: CompactStrategy = {
  name: 'summary',
```

```
  canCompact: (msgs) => msgs.length > 10,
  compact: async (msgs) => {
    const oldMessages = msgs.slice(0, msgs.length - 5)
    const recentMessages = msgs.slice(-5)

    const summary = await generateSummary(oldMessages)

    return {
      messages: [
        { role: 'system', content: `Previous conversation summary:
${summary}` },
        ...recentMessages,
      ],
      summary,
    }
  },
}

// 策略 2：截断压缩
const truncationStrategy: CompactStrategy = {
  name: 'truncation',
  canCompact: () => true,
  compact: (msgs) => ({
    messages: msgs.slice(-MAX_MESSAGES),
    removed: msgs.length - MAX_MESSAGES,
  }),
}

// 策略 3：Snip 压缩（标记可跳过部分）
const snipStrategy: CompactStrategy = {
  name: 'snip',
  canCompact: (msgs) => hasLargeToolResults(msgs),
  compact: (msgs) => ({
    messages: msgs.map(m =>
      hasLargeToolResult(m)
        ? { ...m, content: snipContent(m.content) }
        : m
    ),
  }),
}
```

8.2.3 智能缓存

```
// src/utils/fileStateCache.ts

import { LRUCache } from 'lru-cache'

interface FileStateCache {
  content: string
  timestamp: number
  isPartialView: boolean
  offset?: number
  limit?: number
}

// LRU 缓存文件状态
const fileCache = new LRUCache<string, FileStateCache>({
  maxSize: 1000,
  ttl: 60000,
  sizeCalculation: (value) => value.content.length,
})

export function getCachedFileState(path: string): FileStateCache | undefined
{
  return fileCache.get(path)
}

export function setCachedFileState(
  path: string,
  state: FileStateCache
): void {
  fileCache.set(path, state)
}

// 权限决策缓存
const permissionCache = new Map<string, { result: PermissionResult;
timestamp: number }>()

export function getCachedPermission(
  toolName: string,
  input: unknown
): PermissionResult | undefined {
  const key = `${toolName}:${hashInput(input)}`
  const cached = permissionCache.get(key)

  if (cached && Date.now() - cached.timestamp < PERMISSION_CACHE_TTL) {
```

```
    return cached.result
  }

  return undefined
}
```

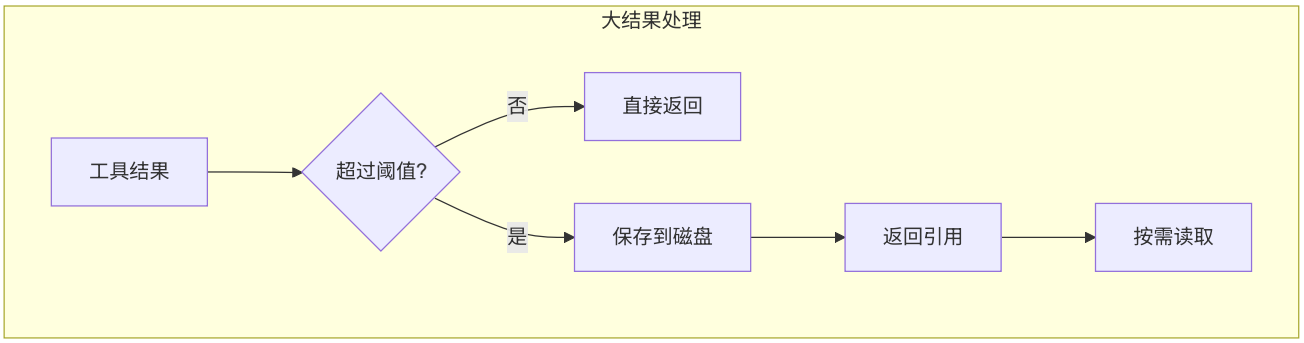
8.2.4 防抖与节流

```
// src/utils/debounce.ts

export function debounce<T extends (...args: unknown[]) => unknown>(
  fn: T,
  delay: number
): (...args: Parameters<T>) => void {
  let timeoutId: ReturnType<typeof setTimeout>

  return (...args) => {
    clearTimeout(timeoutId)
    timeoutId = setTimeout(() => fn(...args), delay)
  }
}

// 应用：状态保存
const debouncedSave = debounce(saveSessionSnapshot, 5000)

onChangeAppState((newState) => {
  debouncedSave(newState)
})

// 节流：限制更新频率
export function throttle<T extends (...args: unknown[]) => unknown>(
  fn: T,
  limit: number
): (...args: Parameters<T>) => void {
  let inThrottle = false

  return (...args) => {
    if (!inThrottle) {
      fn(...args)
      inThrottle = true
      setTimeout(() => inThrottle = false, limit)
    }
  }
```

```
}

// 应用：UI 更新
const throttledUpdate = throttle(updateProgressBar, 100)
```

8.3 内存管理

8.3.1 大结果处理




```
// src/utils/toolResultStorage.ts

const MAX_RESULT_SIZE_CHARS = 100_000

export async function handleLargeToolResult(
  result: ToolResult
): Promise<ToolResult> {
  const resultSize = JSON.stringify(result).length

  if (resultSize < MAX_RESULT_SIZE_CHARS) {
    return result  // 小结果直接返回
  }

  // 大结果保存到磁盘
  const resultPath = await persistToolResult(result)

  // 生成预览
  const preview = generatePreview(result, PREVIEW_SIZE)

  return {
    type: 'file_reference',
```

```
    path: resultPath,
    preview,
    fullSize: resultSize,
  }
}

async function persistToolResult(result: ToolResult): Promise<string> {
  const path = generateTempFilePath('tool-result-', '.json')
  await writeFile(path, JSON.stringify(result))
  return path
}
```

8.3.2 消息历史管理

```
// src/utils/messages.ts

const MAX_MESSAGES_IN_MEMORY = 1000

export function trimMessageHistory(messages: Message[]): Message[] {
  if (messages.length <= MAX_MESSAGES_IN_MEMORY) {
    return messages
  }

  // 保留系统消息和最近的对话
  const systemMessages = messages.filter(m => m.role === 'system')
  const recentMessages = messages.slice(-MAX_MESSAGES_IN_MEMORY / 2)

  return [...systemMessages, ...recentMessages]
}
```

8.3.3 垃圾回收优化

```
// src/utils/cleanupRegistry.ts

interface CleanupEntry {
  id: string
  cleanup: () => void
  priority: number
}

const cleanupRegistry: CleanupEntry[] = []
```

```
export function registerCleanup(
  cleanup: () => void,
  priority = 0
): () => void {
  const id = generateId()
  cleanupRegistry.push({ id, cleanup, priority })

  // 返回取消注册函数
  return () => {
    const index = cleanupRegistry.findIndex(e => e.id === id)
    if (index !== -1) {
      cleanupRegistry.splice(index, 1)
    }
  }
}

export async function runCleanup(): Promise<void> {
  // 按优先级排序执行清理
  const sorted = [...cleanupRegistry].sort((a, b) => b.priority -
a.priority)

  for (const entry of sorted) {
    try {
      entry.cleanup()
    } catch (error) {
      logError(`Cleanup failed for ${entry.id}:`, error)
    }
  }

  cleanupRegistry.length = 0
}
```

8.4 渲染优化

8.4.1 React 优化

```
// src/components/Messages.tsx

import { memo, useMemo, useCallback } from 'react'

// 使用 memo 避免不必要的重渲染
const MessageItem = memo(function MessageItem({ message }: { message:
Message }) {
```

```
  return (
    <Box>
      <Text>{message.content}</Text>
    </Box>
  )
}, (prev, next) => {
  // 自定义比较函数
  return prev.message.id === next.message.id
})

// 使用 useMemo 缓存计算结果
function MessageList({ messages }: { messages: Message[] }) {
  const groupedMessages = useMemo(() => {
    return groupMessagesByDate(messages)
  }, [messages])

  // 使用 useCallback 缓存回调
  const handleRetry = useCallback((messageId: string) => {
    retryMessage(messageId)
  }, [])

  return (
    <Box>
      {groupedMessages.map(group => (
        <MessageGroup
          key={group.date}
          group={group}
          onRetry={handleRetry}
        />
      ))}
    </Box>
  )
}
```

8.4.2 虚拟滚动

```
// src/components/VirtualMessageList.tsx

interface VirtualListProps {
  items: Message[]
  itemHeight: number
  windowHeight: number
}
```

```
export function VirtualMessageList({
  items,
  itemHeight,
  windowHeight,
}: VirtualListProps) {
  const [scrollTop, setScrollTop] = useState(0)

  // 计算可见范围
  const startIndex = Math.floor(scrollTop / itemHeight)
  const endIndex = Math.min(
    startIndex + Math.ceil(windowHeight / itemHeight),
    items.length
  )

  const visibleItems = items.slice(startIndex, endIndex)
  const offsetY = startIndex * itemHeight

  return (
    <Box
      height={windowHeight}
      onScroll={e => setScrollTop(e.scrollTop)}
    >
      <Box height={items.length * itemHeight}>
        <Box marginTop={offsetY}>
          {visibleItems.map((item, index) => (
            <MessageItem
              key={item.id}
              message={item}
              height={itemHeight}
            />
          ))}
        </Box>
      </Box>
    </Box>
  )
}
```

8.5 性能监控

8.5.1 启动性能分析

```
// src/utils/startupProfiler.ts

interface Checkpoint {
  name: string
  timestamp: number
  elapsed: number
}

const checkpoints: Checkpoint[] = []

export function profileCheckpoint(name: string): void {
  const now = performance.now()
  const elapsed = checkpoints.length > 0
    ? now - checkpoints[0].timestamp
    : 0

  checkpoints.push({ name, timestamp: now, elapsed })
}

export function profileReport(): void {
  console.log('Startup Profile:')
  checkpoints.forEach((cp, i) => {
    const duration = i > 0
      ? cp.timestamp - checkpoints[i - 1].timestamp
      : 0
    console.log(`  ${cp.name}: ${duration.toFixed(2)}ms (total:
${cp.elapsed.toFixed(2)}ms)`)
  })
}

// 使用
profileCheckpoint('main_tsx_entry')
// ... 加载模块 ...
profileCheckpoint('imports_loaded')
// ... 初始化服务 ...
profileCheckpoint('services_ready')
// ...
profileReport()
```

8.5.2 运行时性能指标

```typescript
// src/utils/performance.ts

interface PerformanceMetrics {
  // API 调用
  apiCalls: number
  apiDuration: number
  apiTokens: number

  // 工具执行
  toolCalls: number
  toolDuration: number

  // 渲染
  renderCount: number
  renderDuration: number
}

const metrics: PerformanceMetrics = {
  apiCalls: 0,
  apiDuration: 0,
  apiTokens: 0,
  toolCalls: 0,
  toolDuration: 0,
  renderCount: 0,
  renderDuration: 0,
}

export function recordApiCall(duration: number, tokens: number): void {
  metrics.apiCalls++
  metrics.apiDuration += duration
  metrics.apiTokens += tokens
}

export function recordToolCall(duration: number): void {
  metrics.toolCalls++
  metrics.toolDuration += duration
}

export function getPerformanceReport(): string {
  return `
Performance Report:
  API: ${metrics.apiCalls} calls, ${metrics.apiDuration}ms total,
${metrics.apiTokens} tokens
```

```
    Tools: ${metrics.toolCalls} calls, ${metrics.toolDuration}ms total
    Renders: ${metrics.renderCount}, ${metrics.renderDuration}ms total
    `
  }
```

8.6 本章小结

本章深入探讨了 Claude Code 的性能优化策略：

1. **启动优化**：并行加载、延迟加载、预取、编译时优化
2. **运行时优化**：Token 预算、上下文压缩、智能缓存、防抖节流
3. **内存管理**：大结果处理、消息历史管理、垃圾回收
4. **渲染优化**：React memo、虚拟滚动
5. **性能监控**：启动分析、运行时指标

这些优化策略共同确保了 Claude Code 的流畅体验。

在下一章中，我们将总结设计模式和最佳实践。

← 上一章：MCP 集成 | 下一章：设计模式 →

chapter-10-future

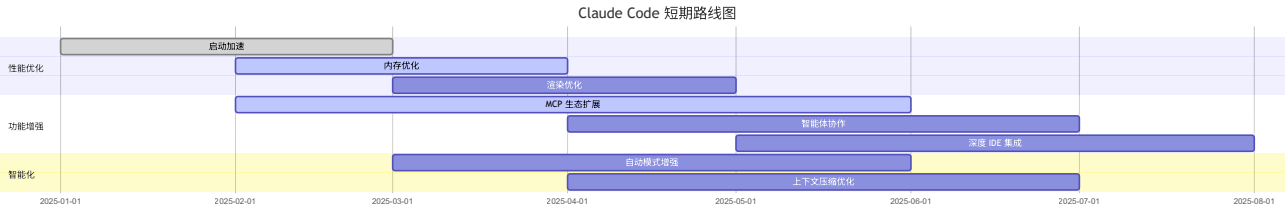
第10章 未来展望与附录

> "预测未来的最好方式就是创造它。"
> —— 《Claude Code 设计哲学》（化用 Alan Kay 观点）

本书的前九章深入探讨了 Claude Code 的架构与设计。本章展望其未来发展，并提供实用的附录信息。

10.1 演进方向

10.1.1 短期路线图

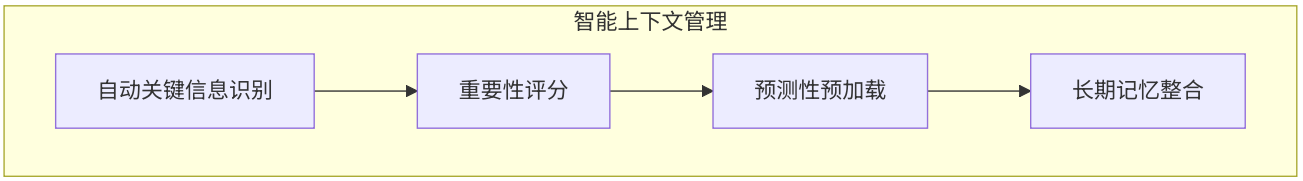


10.1.2 更智能的上下文管理

当前挑战：

- 长对话的上下文压缩会丢失细节
- 无法自动识别最重要的上下文

未来方向：



技术方向：

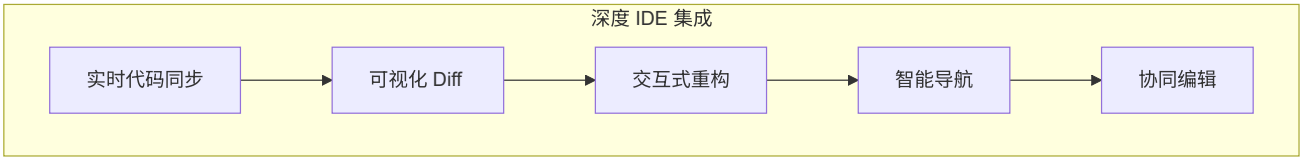
1. **关键信息识别**：自动识别对话中的关键决策点和重要信息
2. **重要性评分**：为每条消息计算重要性分数，压缩时优先保留高分内容
3. **预测性预加载**：根据用户工作模式预测需要的上下文
4. **长期记忆整合**：跨会话学习用户偏好和项目知识

10.1.3 更深度的 IDE 集成

当前集成：

- MCP 协议支持与 IDE 通信
- 文件读写操作

未来愿景：



可能的实现：

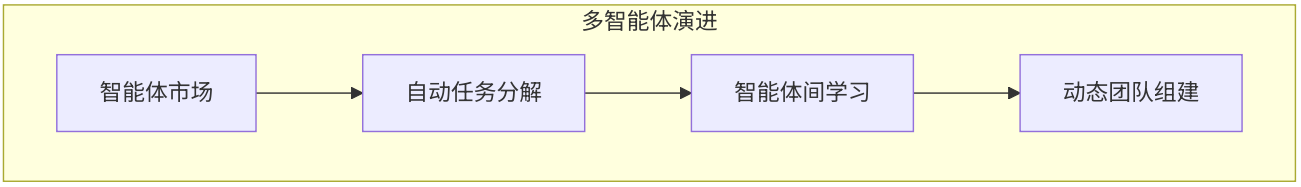
功能	描述	技术方案
实时代码同步	代码变化实时反映	WebSocket 双向同步
可视化 Diff	图形化展示变更	集成 Monaco Diff
交互式重构	可视化重构选项	IDE 插件 API
智能导航	代码结构可视化	语义分析 + 图形
协同编辑	多人同时编辑	CRDT 算法

10.1.4 更强大的多智能体

当前能力：

- 子智能体并行执行
- Teammate 双向通信

未来演进：



技术方向：

1. **智能体市场**：社区共享专业智能体（代码审查专家、安全审计专家等）
2. **自动任务分解**：AI 自动将复杂任务分解给合适的子智能体
3. **智能体间学习**：子智能体的经验可以被其他智能体学习
4. **动态团队组建**：根据任务特点自动组建最优智能体团队

10.1.5 自然语言编程

愿景： 让自然语言成为一等编程语言

```
用户："创建一个处理用户注册的 API 端点，
       需要验证邮箱、检查重复、发送确认邮件"

Claude Code：
1. 设计 API 接口
2. 实现验证逻辑
3. 集成邮件服务
4. 编写测试
5. 生成文档
```

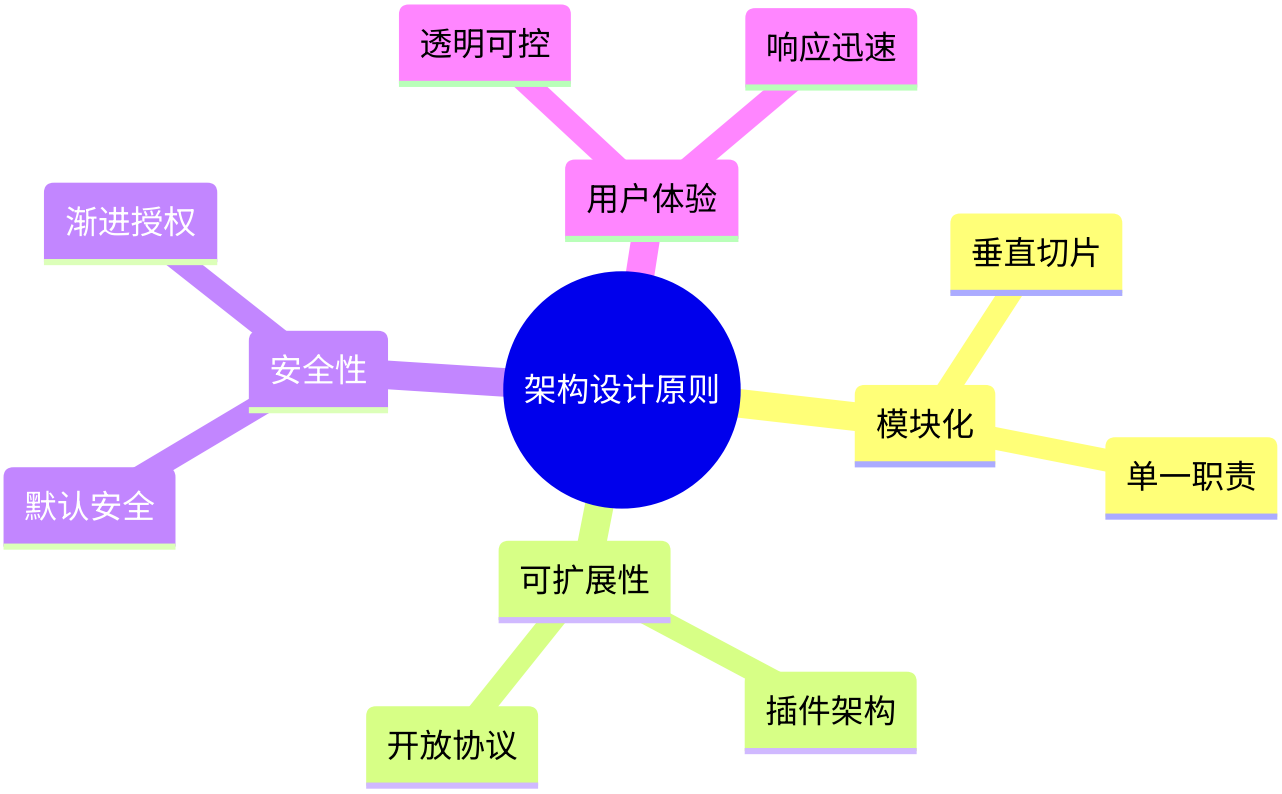
技术挑战：

- 自然语言的歧义性
- 需求完整性验证
- 生成的代码质量
- 可维护性保障

10.2 对开发者的启示

10.2.1 架构设计原则

从 Claude Code 学到的核心原则：



10.2.2 AI 应用设计要点

要点	说明	Claude Code 实践
能力透明	用户清楚 AI 能做什么	工具即界面
行为可控	用户可以控制 AI 行为	权限系统
错误可恢复	错误不是终点	会话恢复
渐进学习	随使用变得更智能	规则记忆
人机协作	AI 辅助而非替代	多智能体

10.2.3 技术选型建议

如果你要构建类似的 AI 应用：

1. **运行时**：Bun (快速、内置打包) 或 Node.js (生态丰富)
2. **语言**：TypeScript (类型安全)
3. **状态管理**：轻量级 Store (自定义或 Zustand)
4. **UI**：Ink (TUI) 或 Electron (GUI)
5. **协议**：MCP (开放生态)

10.3 附录

10.3.1 术语表

术语	定义
Agent	AI 实例，可以是主智能体或子智能体
MCP	Model Context Protocol，模型上下文协议
QueryEngine	Claude Code 的核心查询引擎
Tool	AI 可调用的能力单元
Skill	可复用的工作流定义
CLAUDE.md	项目级别的 AI 上下文文件
Subagent	由 AgentTool 创建的子智能体
Teammate	可双向通信的长期智能体
Fork	继承完整上下文的子智能体
Permission Mode	权限模式（default/auto/plan等）
Feature Flag	特性标志，控制功能开关

10.3.2 推荐阅读

架构设计：

- Clean Architecture - Robert C. Martin
- Vertical Slice Architecture - Jimmy Bogard
- Building Microservices - Sam Newman

AI 与 LLM：

- Language Models are Few-Shot Learners - GPT-3 论文
- ReAct: Synergizing Reasoning and Acting - 推理与行动结合
- The Rise and Potential of Large Language Model Based Agents - 智能体综述

相关协议：

- MCP Specification - 官方文档
- Language Server Protocol - LSP 规范
- JSON-RPC 2.0 - RPC 协议

10.3.3 源码导航

```
ClaudeCodesrc/
├── main.tsx                    # 应用入口
├── QueryEngine.ts              # 查询引擎核心
```

```
├── Tool.ts                    # 工具抽象基类
├── tools.ts                   # 工具注册
├── state/
│   ├── AppState.tsx          # React 状态管理
│   ├── AppStateStore.ts      # 状态定义
│   └── store.ts              # Store 实现
├── tools/
│   ├── BashTool/             # Bash 工具
│   ├── FileEditTool/         # 文件编辑工具
│   ├── AgentTool/            # 智能体工具
│   └── MCPTool/              # MCP 工具适配器
├── services/
│   └── mcp/
│       ├── client.ts         # MCP 客户端
│       ├── config.ts         # MCP 配置
│       └── types.ts          # MCP 类型
├── utils/
│   └── permissions/
│       ├── permissions.ts    # 权限核心
│       ├── PermissionMode.ts # 权限模式
│       └── yoloClassifier.ts # 自动分类器
└── types/
    └── permissions.ts        # 权限类型定义
```

10.3.4 调试技巧

启动性能分析：

```bash
# 启用启动性能分析
CLAUDE_CODE_PROFILE=1 claude

# 查看详细日志
claude --verbose
```

工具调试：

```typescript
// 在工具中添加调试日志
logForDebugging('MyTool input:', input)

// 查看状态变化
onChangeAppState((newState, oldState) => {
```

```
  console.log('State changed:', diff(oldState, newState))
})
```

MCP 调试：

```
# 查看 MCP 通信日志
CLAUDE_CODE_MCP_DEBUG=1 claude

# 测试 MCP 服务器
npx @modelcontextprotocol/inspector <server-command>
```

10.3.5 社区资源

- **官方文档**：https://docs.anthropic.com/en/docs/claude-code
- **GitHub 仓库**：https://github.com/anthropics/claude-code
- **Discord 社区**：Anthropic Community
- **Stack Overflow**：#claude-code 标签

10.4 结语

Claude Code 代表了 AI 辅助开发的未来方向。它不是要替代开发者，而是成为开发者的真正搭档。

通过本书的学习，我们深入理解了：

1. **设计哲学**：渐进式信任、工具即界面、上下文即代码
2. **核心架构**：分层设计、QueryEngine、状态管理
3. **工具系统**：可扩展的能力层
4. **权限系统**：安全与便利的平衡
5. **多智能体**：并行处理复杂任务
6. **MCP 集成**：开放的生态系统
7. **性能优化**：启动、运行时、内存
8. **设计模式**：可复用的架构经验

希望本书能够帮助你：

- 深入理解 Claude Code 的设计理念
- 在自己的项目中应用这些模式
- 为 Claude Code 生态做出贡献
- 构建更好的 AI 辅助工具

最后的话：

> AI 不是要取代人类，而是要增强人类。
> Claude Code 只是一个开始，真正的魔法发生在你使用它创造的时候。

感谢阅读。

← 上一章：设计模式

全书完

---

chapter-09-patterns

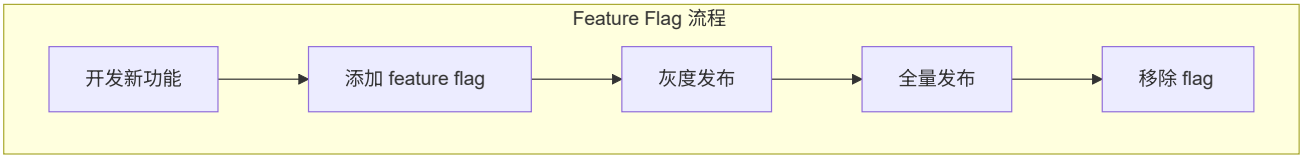
第9章 设计模式与最佳实践

> "好的代码不是写出来的，是经过反复提炼的。"
> —— 《Claude Code 设计哲学》

本章总结 Claude Code 中反复出现的设计模式和最佳实践，帮助读者在自己的项目中应用这些经验。

9.1 核心设计模式

9.1.1 特性标志驱动开发




```
// 使用 Bun 的 feature() 实现编译时特性标志
import { feature } from 'bun:bundle'

// 特性标志控制的代码
const coordinatorMode = feature('COORDINATOR_MODE')
  ? require('./coordinator/coordinatorMode.js')
  : null

// 运行时检查
if (feature('TRANSCRIPT_CLASSIFIER')) {
```

```
  enableAutoMode()
}
```

特性标志的优势：

优势	说明
零运行时开销	编译时确定，无运行时检查
死代码消除	未使用的代码不会打包
渐进发布	可以控制功能上线节奏
快速回滚	发现问题立即关闭

9.1.2 延迟加载模式

```
// 避免循环依赖和减少初始加载

// ❌ 直接导入（可能造成循环依赖）
import { teammateUtils } from './utils/teammate.js'

// ✅ 延迟加载
const getTeammateUtils = () => require('./utils/teammate.js')

// 在函数内部使用
function processTeammate() {
  const { getTeammateId } = getTeammateUtils()
  return getTeammateId()
}
```

9.1.3 依赖注入

```
// ❌ 全局导入（紧耦合）
import { getAppState } from './state/AppState.js'

function doSomething() {
  const state = getAppState() // 隐式依赖
}

// ✅ 依赖注入（松耦合）
interface ToolUseContext {
  getAppState(): AppState
  setAppState(f: (prev: AppState) => AppState): void
```

```
}

function doSomething(context: ToolUseContext) {
  const state = context.getAppState() // 显式依赖
}
```

9.1.4 Builder 模式

```
// src/Tool.ts

export function buildTool<D extends AnyToolDef>(def: D): BuiltTool<D> {
  const TOOL_DEFAULTS = {
    isEnabled: () => true,
    isConcurrencySafe: () => false,
    isReadOnly: () => false,
    isDestructive: () => false,
    checkPermissions: async (input) => ({
      behavior: 'allow',
      updatedInput: input,
    }),
  }

  return {
    ...TOOL_DEFAULTS,
    ...def,
  } as BuiltTool<D>
}

// 使用
const MyTool = buildTool({
  name: 'MyTool',
  description: 'My tool',
  // 只需要覆盖需要自定义的方法
  async call(input) {
    // ...
  },
})
```

9.1.5 策略模式

```
// src/services/compact/compact.ts
```

```typescript
interface CompactStrategy {
  name: string
  canCompact(messages: Message[]): boolean
  compact(messages: Message[]): CompactResult
}

const strategies: CompactStrategy[] = [
  summaryStrategy,
  truncationStrategy,
  snipStrategy,
]

export function compactMessages(messages: Message[]): Message[] {
  for (const strategy of strategies) {
    if (strategy.canCompact(messages)) {
      return strategy.compact(messages).messages
    }
  }
  return messages
}
```

9.2 类型安全实践

9.2.1 Zod + TypeScript

```typescript
import { z } from 'zod/v4'

// 定义 Schema
const inputSchema = z.object({
  name: z.string(),
  age: z.number().optional(),
  email: z.string().email(),
})

// 推断 TypeScript 类型
type Input = z.infer<typeof inputSchema>

// 运行时验证
function process(input: unknown): Input {
  return inputSchema.parse(input) // 验证失败会抛出错误
}
```

9.2.2 品牌类型（Branded Types）

```typescript
// src/types/ids.ts

type Brand<K, T> = K & { __brand: T }

export type SessionId = Brand<string, 'SessionId'>
export type AgentId = Brand<string, 'AgentId'>
export type TaskId = Brand<string, 'TaskId'>

// 使用
function createSessionId(): SessionId {
  return randomUUID() as SessionId
}

// 类型安全
const sessionId: SessionId = createSessionId()
const agentId: AgentId = createAgentId()

// ❌ 编译错误：不能混用
// const wrong: SessionId = agentId
```

9.2.3 不可变性

```typescript
// 使用 readonly 和 DeepImmutable
type DeepImmutable<T> = {
  readonly [K in keyof T]: T[K] extends object
    ? T[K] extends Function
      ? T[K]
      : DeepImmutable<T[K]>
    : T[K]
}

// 状态更新模式
setAppState(prev => ({
  ...prev,
  messages: [...prev.messages, newMessage],
}))
```

9.3 错误处理

9.3.1 错误即值

```
// ❌ 使用异常控制流程
try {
  const result = riskyOperation()
  return result
} catch (e) {
  return null
}

// ✅ 使用 Result 类型
interface Result<T, E> {
  success: boolean
  data?: T
  error?: E
}

function riskyOperation(): Result<Data, Error> {
  if (somethingWrong) {
    return { success: false, error: new Error('...') }
  }
  return { success: true, data: ... }
}
```

9.3.2 分层错误处理

```
// 工具层错误
type ToolError =
  | { type: 'validation'; message: string }
  | { type: 'permission'; message: string }
  | { type: 'execution'; message: string; cause: Error }

// 应用层错误
type AppError =
  | { type: 'tool'; toolName: string; error: ToolError }
  | { type: 'api'; message: string }
  | { type: 'system'; message: string }

// 统一错误处理
function handleError(error: AppError): void {
  switch (error.type) {
    case 'tool':
      logToolError(error)
      showToolErrorUI(error)
```

```
      break
    case 'api':
      logApiError(error)
      showRetryDialog(error)
      break
    // ...
  }
}
```

9.4 测试策略

9.4.1 单元测试

```
// MyTool.test.ts
import { describe, it, expect, vi } from 'vitest'
import { MyTool } from './MyTool.js'

describe('MyTool', () => {
  it('should validate input', async () => {
    const result = await MyTool.validateInput(
      { name: '' },
      mockContext
    )
    expect(result.result).toBe(false)
  })

  it('should execute successfully', async () => {
    const mockCanUseTool = vi.fn().mockResolvedValue({ behavior: 'allow' })

    const result = await MyTool.call(
      { name: 'test' },
      mockContext,
      mockCanUseTool
    )

    expect(result.data).toBeDefined()
  })
})
```

9.4.2 集成测试

```
// QueryEngine.integration.test.ts
describe('QueryEngine', () => {
  it('should handle tool calls', async () => {
    const engine = createTestEngine()

    await engine.submitMessage('List files')

    await waitFor(() =>
      expect(engine.messages).toContainToolCall('Glob')
    )
  })
})
```

9.4.3 Mock 策略

```
// 使用依赖注入便于测试
const createTestTool = (deps: {
  fs: typeof import('fs/promises')
  exec: typeof import('child_process').exec
}) => {
  return buildTool({
    name: 'TestTool',
    async call(input) {
      // 使用注入的依赖
      const content = await deps.fs.readFile(input.path)
      return { content }
    },
  })
}
```

9.5 代码组织

9.5.1 垂直切片

```
tools/
  MyTool/
    ├── index.ts      # 工具逻辑
    ├── types.ts      # 类型定义
    ├── UI.tsx        # UI 组件
    ├── utils.ts      # 辅助函数
    └── test.ts       # 测试
```

9.5.2 导入顺序

```
// 1. 外部库
import React from 'react'
import { z } from 'zod'

// 2. 内部模块
import { buildTool } from '../../Tool.js'

// 3. 本地文件
import { renderToolUse } from './UI.js'
import type { Input, Output } from './types.js'
```

9.6 常见陷阱

9.6.1 循环依赖

```
// ❌ A.ts 导入 B.ts，B.ts 导入 A.ts
// A.ts
import { B } from './B.js'
export const A = { useB: () => B }

// B.ts
import { A } from './A.js'  // 循环！
export const B = { useA: () => A }

// ✅ 解决方案：提取公共类型到单独文件
// types.ts
export interface A { ... }
export interface B { ... }

// A.ts
import type { B } from './types.js'

// B.ts
import type { A } from './types.js'
```

9.6.2 闭包陷阱

```
// ❌ 错误的闭包使用
for (const item of items) {
  setTimeout(() => {
```

```
    console.log(item) // 可能都是最后一个
  }, 1000)
}

// ✅ 使用 let 或闭包
for (const item of items) {
  const capturedItem = item
  setTimeout(() => {
    console.log(capturedItem)
  }, 1000)
}

// ✅ 使用 forEach
items.forEach(item => {
  setTimeout(() => {
    console.log(item)
  }, 1000)
})
```

9.6.3 异步竞态

```
// ❌ 竞态条件
let value = 0
async function increment() {
  const current = value
  await sleep(100)
  value = current + 1
}

// ✅ 使用原子操作
let value = 0
async function increment() {
  value++ // 原子操作
}

// ✅ 使用锁
const lock = new AsyncLock()
async function increment() {
  await lock.acquire(async () => {
    const current = value
    await sleep(100)
    value = current + 1
```

```
  })
}
```

9.7 本章小结

本章总结了 Claude Code 的核心设计模式：

1. **特性标志驱动**：编译时特性控制
2. **延迟加载**：避免循环依赖和减少初始加载
3. **依赖注入**：松耦合，易测试
4. **Builder 模式**：合理的默认值
5. **策略模式**：可插拔的算法
6. **类型安全**：Zod + TypeScript
7. **错误处理**：错误即值，分层处理
8. **测试策略**：单元测试 + 集成测试
9. **代码组织**：垂直切片
10. **常见陷阱**：循环依赖、闭包陷阱、异步竞态

在最后一章中，我们将展望 Claude Code 的未来发展。

← 上一章：性能优化 | 下一章：未来展望 →